In the name

of

Allah

Database Management Systems

Lecture - 2

Overview of lecture-1

- Database definitions
- File processing systems
- Advantages of database

Today's Lecture

- More advantages
- Some costs
- Levels of data
- Database users

Data & Information

Company: Super Soft Dept: Sales

Emp Name	Age	Salary
Malik Sharif	**23**	**55**
Sh. M. Akmal	**24**	**55**
M. A. Butt	**20**	**40**
Malik Junaid	**19**	**20**

- **Schema**
- **Database Applications**
- **Database Management System (DBMS)**

Other Advantages

- Data consistency
- Better data security
- Faster development of new applications

They also provide

- Economy of scale
- Better concurrency control
- Better backup and recovery procedures

BUT

Its not always just the

SUGAR

Disadvantages

- Higher costs
- Conversion cost
- More difficult recovery

Data As Resource

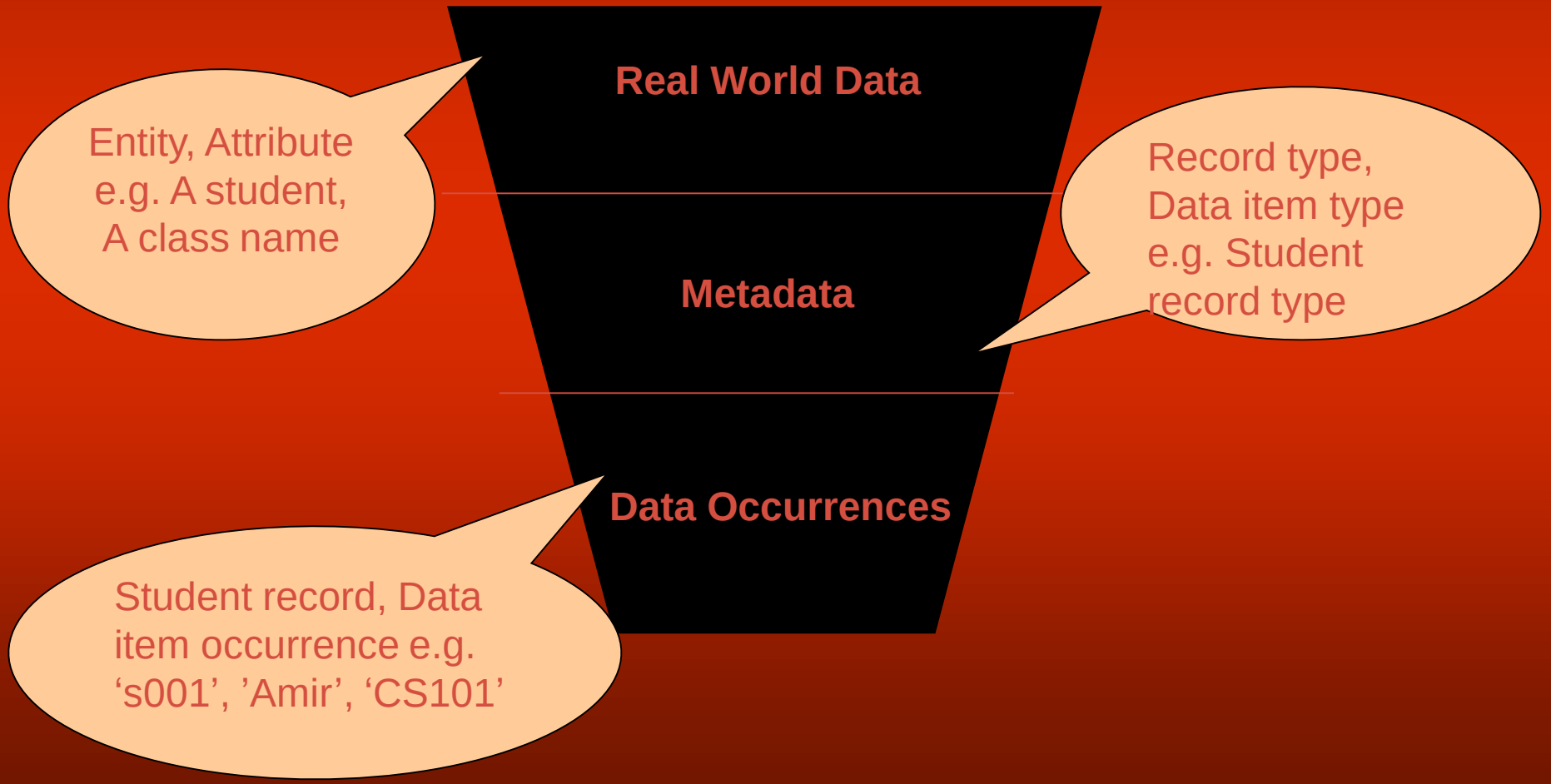
Resource

Any asset that is of value to an organization and that incurs cost

Is data a resource ?

YES

Levels of Data, Rizwan prepare ppt for front



Employee

name, age, qual, sal



Emp

Name text

Age number

Sal number

Malik Sharif	**23**	**55**
Sh. M. Akmal	**24**	**55**
M. A. Butt	**20**	**40**
Malik Junaid	**19**	**20**

Levels of Data

- Real-world data
- Metadata
- Data Occurrence

Database Users

- Application Programmers
- End Users
  - Naïve
  - Sophisticated

Database Users

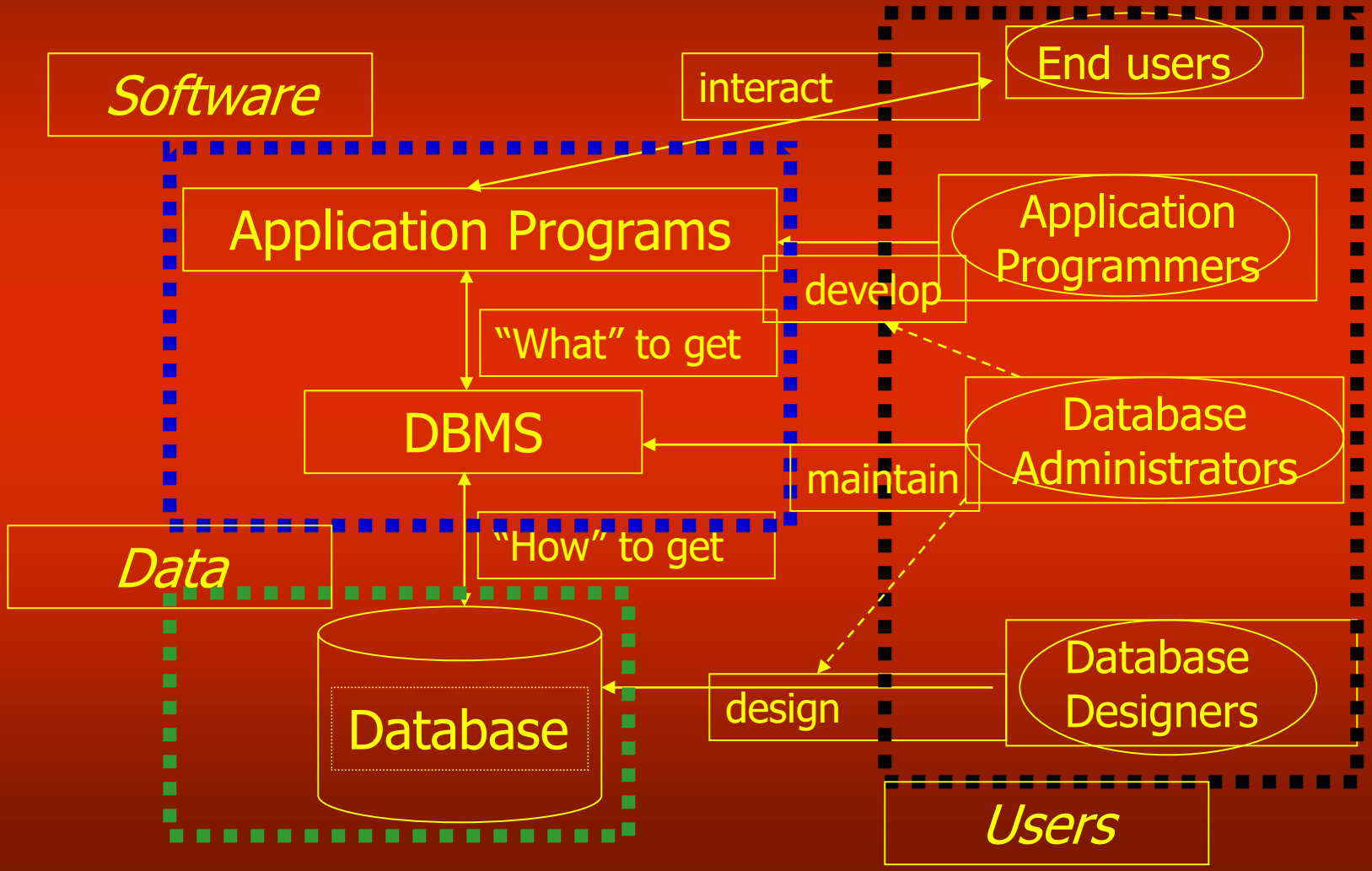
- **Database Administrator (DBA)**

  A person who has central control over data and programs that access this data

Functions of DBA

- Schema definition
- Granting data access
- Routine Maintenance
  - Backups
  - Monitoring disk space
  - Monitoring jobs running

Typical Components



Database Architecture

History

- Initiates in 1960s; IMS, IDS
- Conference on DAta Systems Languages (CODASYL)
- Data Base Task Group (DBTG)
- General architecture in 1971

History

- DBTG's two layered architecture having
  - System View- Schema
  - User View- Sub rschemas

History

- American National Standards Institute (ANSI)
- Standards Planning and Requirements Committee (SPARC)
- ANSI-SPARC's 3-layered Architecture

Summary

- Basic Terminology
- Pros n Cons of DB Env
- Levels of Data
- Types of Users
- History of 3-L Architecture

Thanks

Database Management Systems

Lecture - 2

Database Management Systems

Lecture - 3

Lecture Overview

- Database Architecture
- Data Independence

Database Architecture

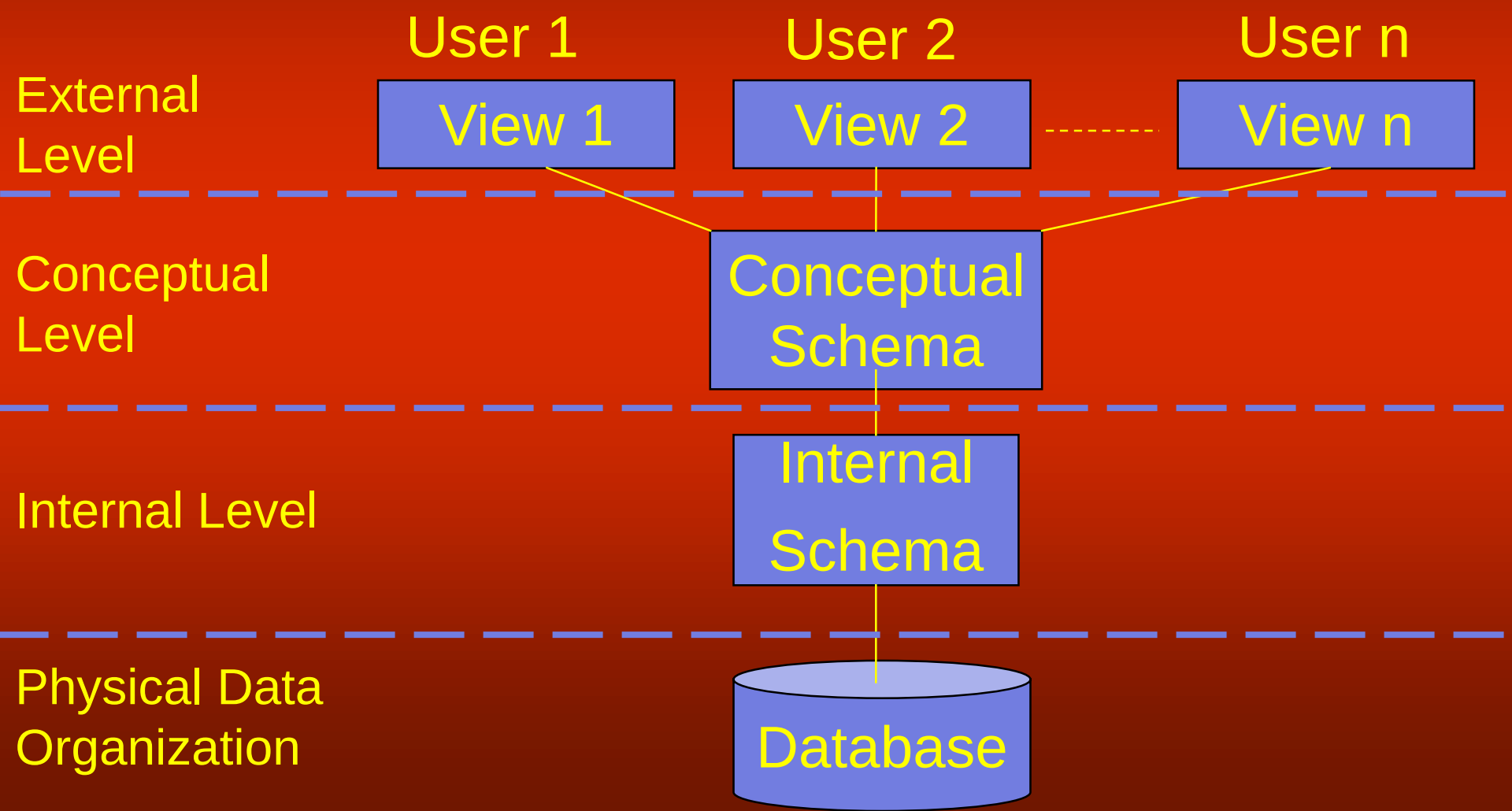
Three Level Architecture

- A basis for understanding DBMS functionalities
- Three levels at which data can be described

Objective

- Separate users view from the physical representation
- Why?
  - Different views of same data
  - Consolidated representation
  - Both ways easy change

The Three-Level Architecture



The Architecture

- **Depicted by three *schemas* or three models**
- **Refers to permanent structure or *intention of database***

Level 1
External View

The way users think about data

External View

Each user has a view of the database limited to the appropriate portion of the user's *perspective of reality.*

External View

Users may have *different* views of the same data e.g. date, time etc.

External View

Virtual/calculated data: that is not actually stored in the database but is created when needed e.g. age, statistical data etc.

External View

DBMS uses external views to create user interface for different users which is both the *facility and barrier*

External View

User’s external view is created after considering data access, reports, and the transactions needs.

External View

External schema evolves as user needs are modified over time

Employee Data

First Name: Rana
Last Name: Aslam
Date of Birth:
12 Sep, 1970

Saleema



Workers

Name: R. Aslam
Age: 25y,10d
Dept: Sales

Saleem

External Layer

Lower Layers

Level 2
Logical or Conceptual View

A complete description of the information content of the database

Conceptual Schema

- The entire information structure of the database, as seen by the DBA
- The community view of data

Conceptual Schema

All entities, attributes and their relationships are represented here

Conceptual Schema

Contains record types representing entities, data item types with their attributes, relationships and constraints on data.

Conceptual Schema

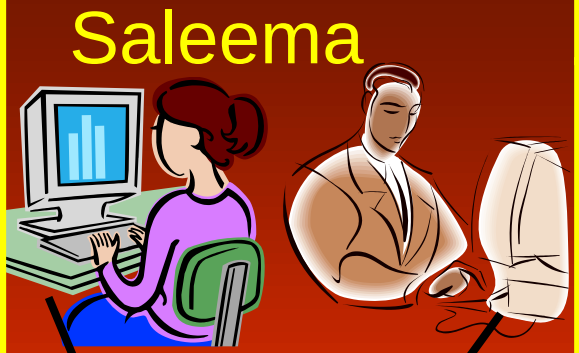
Contains Semantic information about the data meaning, security and integrity information

Conceptual Schema

Relatively constant: **designed with the present as well as future needs of an organization**

Employee Data

First Name: Rana
Last Name: Aslam
Date of Birth:
12 Sep, 1970



Workers

Name: R. Aslam
Age: 24y,10d
Dept: Sales

External Layer

Logical Record Interface

Conceptual Layer

Name	DoB	Deps	DepId
Rana Aslam	12/09/70	5	D001
Marya Wasti	29/02/80	0	D005

Database Management Systems

Lecture - 3

Database Management Systems

Lecture - 4

Today's Review

- Continue 3-L architecture
- Data Independence
- Different aspect of DBMS

Level 3
Internal or Physical View

Concerns about the physical implementation of the database

Internal View

- DBMS chooses type of data structures
- lays out data on storage devices with operating system access methods

Internal View

- ***Internal record*: a single stored record**
- **Does not just contain what we see at the conceptual level**
- **DBMS adds other data**

Physical Level

- Generally same as Internal
- Actual representation of data on the storage device
- In the binary format
- OS responsibility

Inter-Schema Mappings

- Also a part of 3-level architecture
- External/conceptual mapping
- Conceptual/Internal

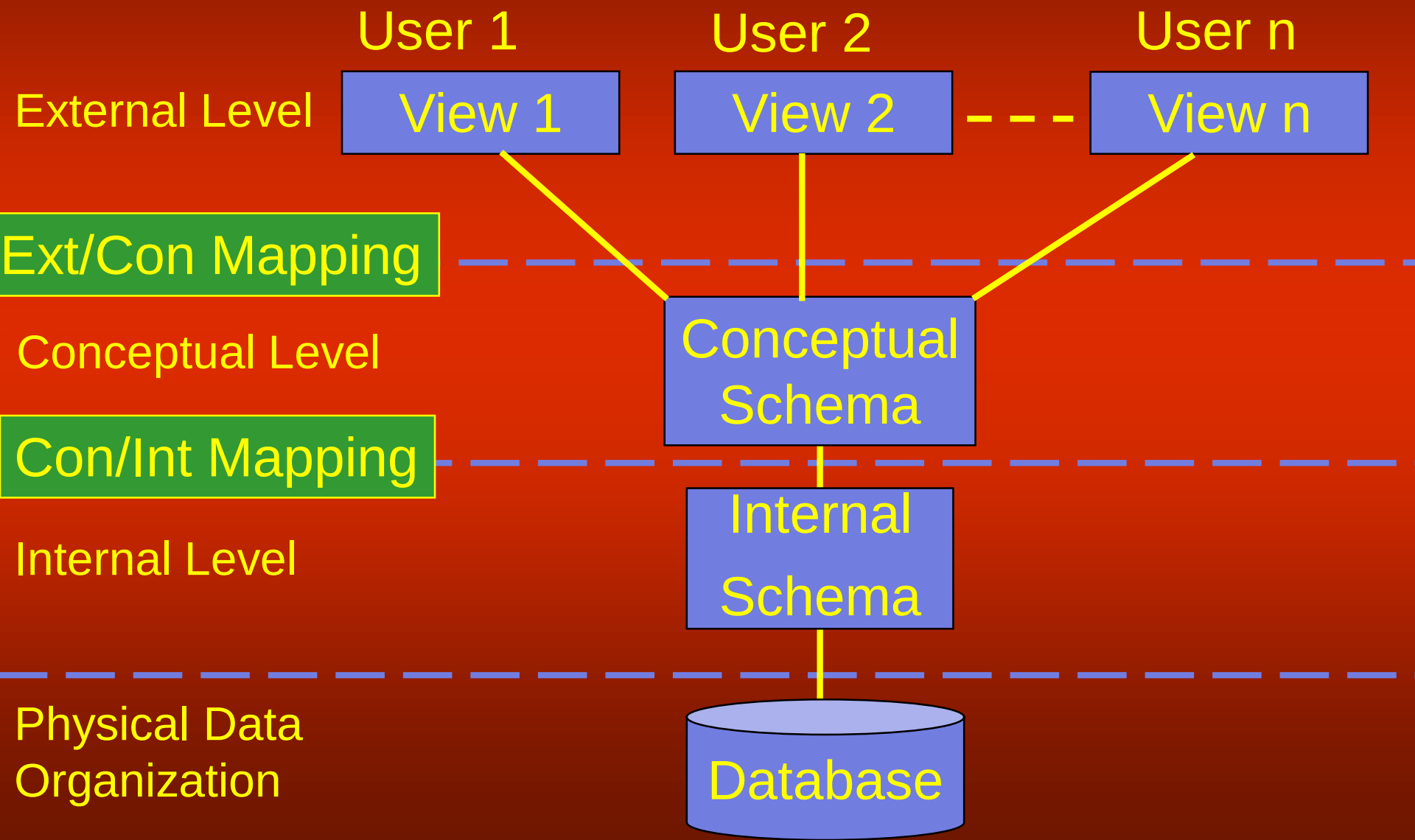
Ext/Con Mapping

Specifies mapping between objects in the external view to those in the logical model

Con/Int Mapping

Specifies mapping between objects in the logical model to those in the physical model-data independence

3-Level Architecture



First Name: Rana
Last Name: Aslam
Date of Birth:
12 Sep, 1970

Saleema



Name: R. Aslam
Age: 24y,10d
Dept: Sales

Saleem

Name	DoB	Deps	DepId
Rana Aslam	12/09/70	5	D001
Marya Wasti	29/02/80	0	D005

BH	RH	Rana Aslam 120970 5 D001	RH	Marya Wasti…

01110011010011100101001010100101010010101…..

Data Independence

Data Independence

- A major outcome of 3-L Arch
- The immunity of applications to change in storage structure and access strategy

Data Independence

- Changes in lower level do not affect the upper levels
- Don’t take it word to word
- Mind the direction please



Data Independence Types

- Logical Data Independence
- Physical Data Independence

Logical Data Independence

- Changes in conceptual model do not affect the external views
- Immunity of external level from changes at conceptual level

Types of Changes

- Adding a new file/index etc.
- Adding a new field in a file
- Changing type/size
- Deleting an attribute

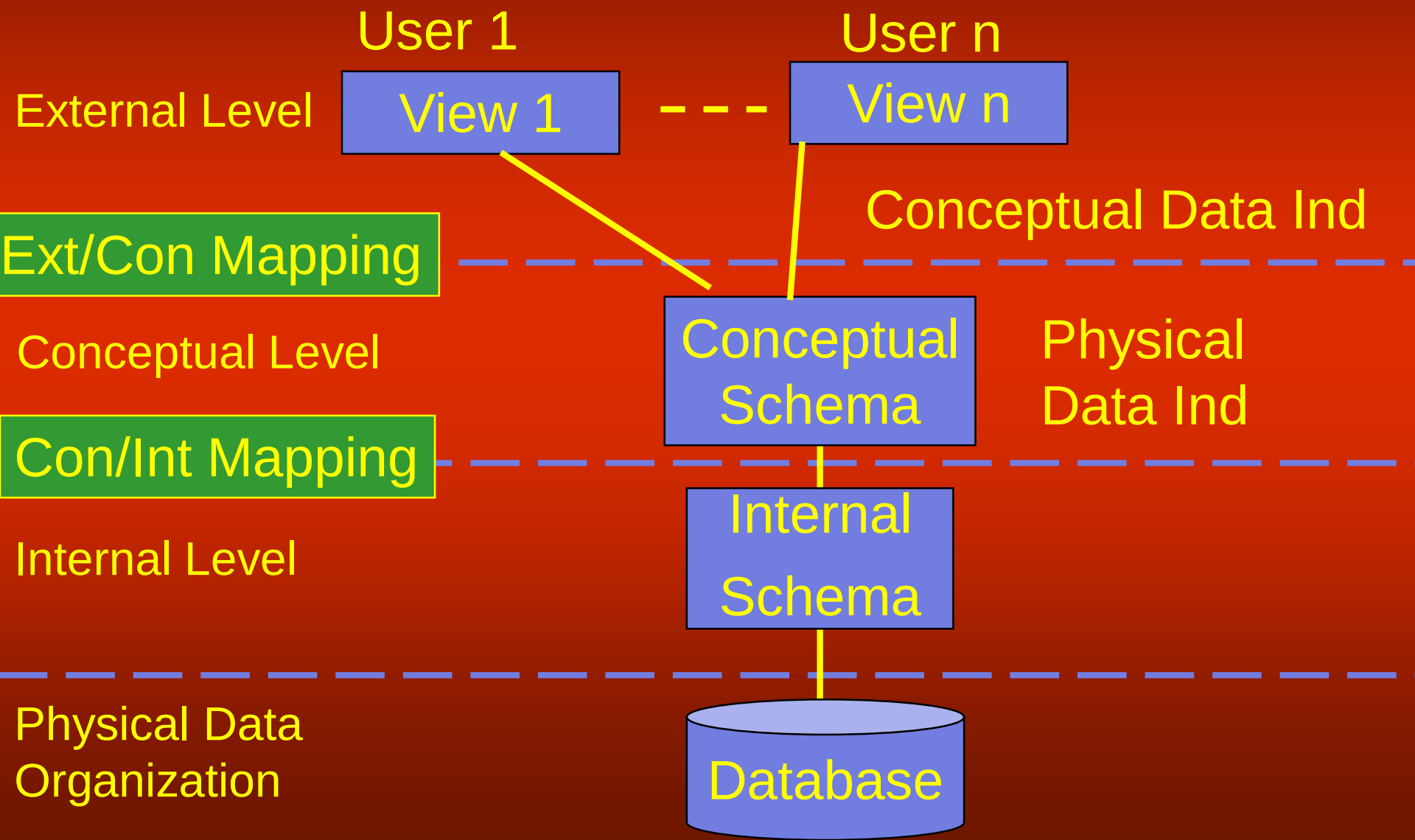
Physical Data Independence

- Changes in the internal model do not affect the conceptual model
- Immunity of Conceptual level from changes at Internal level

Changes Examples

- Changing file organization
- Index implementation, hash, tree etc.
- Changing storage medium

3-Level Architecture



Functions of DBMS

- Data Processing
- A User Accessible Catalog
- Transaction Support
- Concurrency Control Services

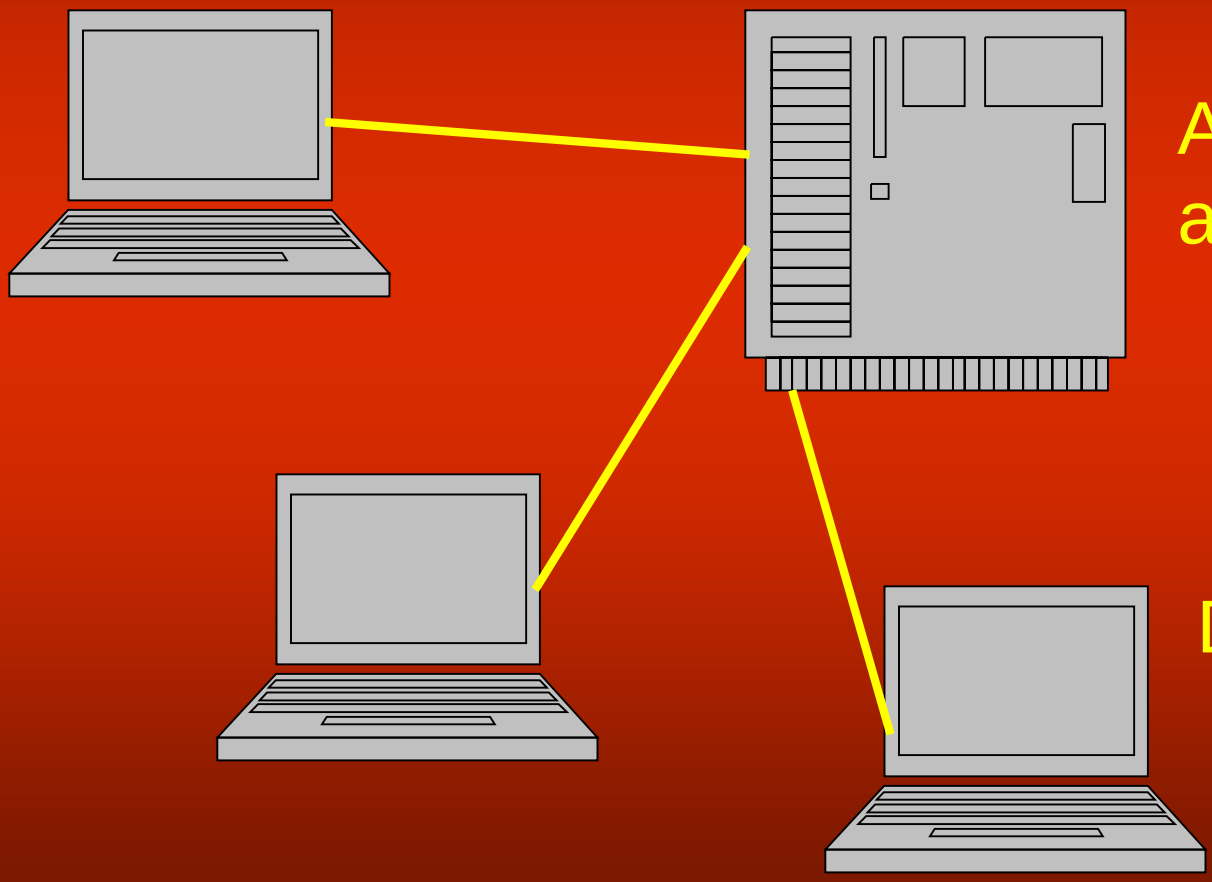
Functions of DBMS

- Recovery Services
- Authorization Services
- Support for Data Communication
- Integrity Services

DBMS Environments

- Single User
- Multi-user
  - Teleprocessing
  - File Servers
  - Client-Server

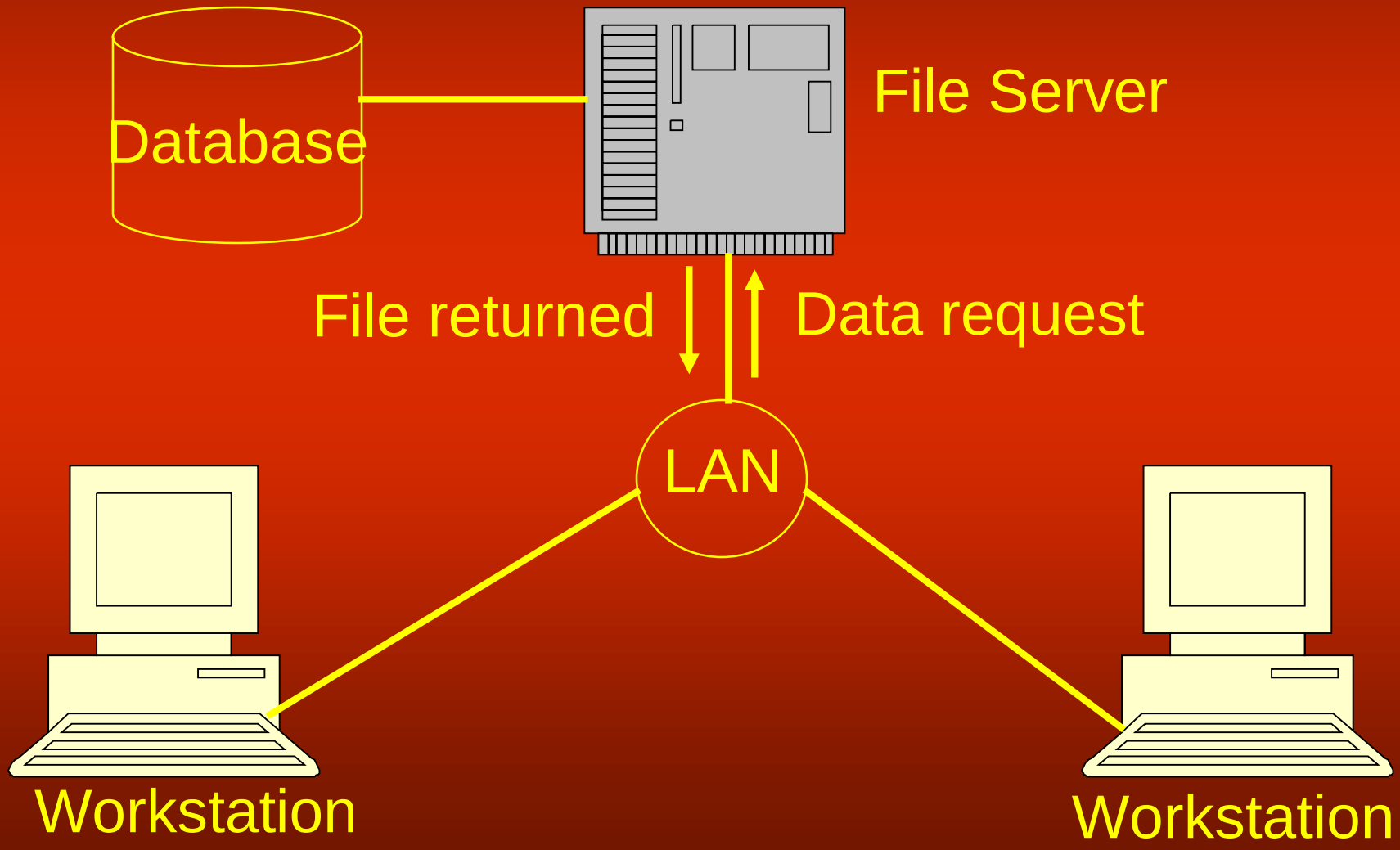
Teleprocessing



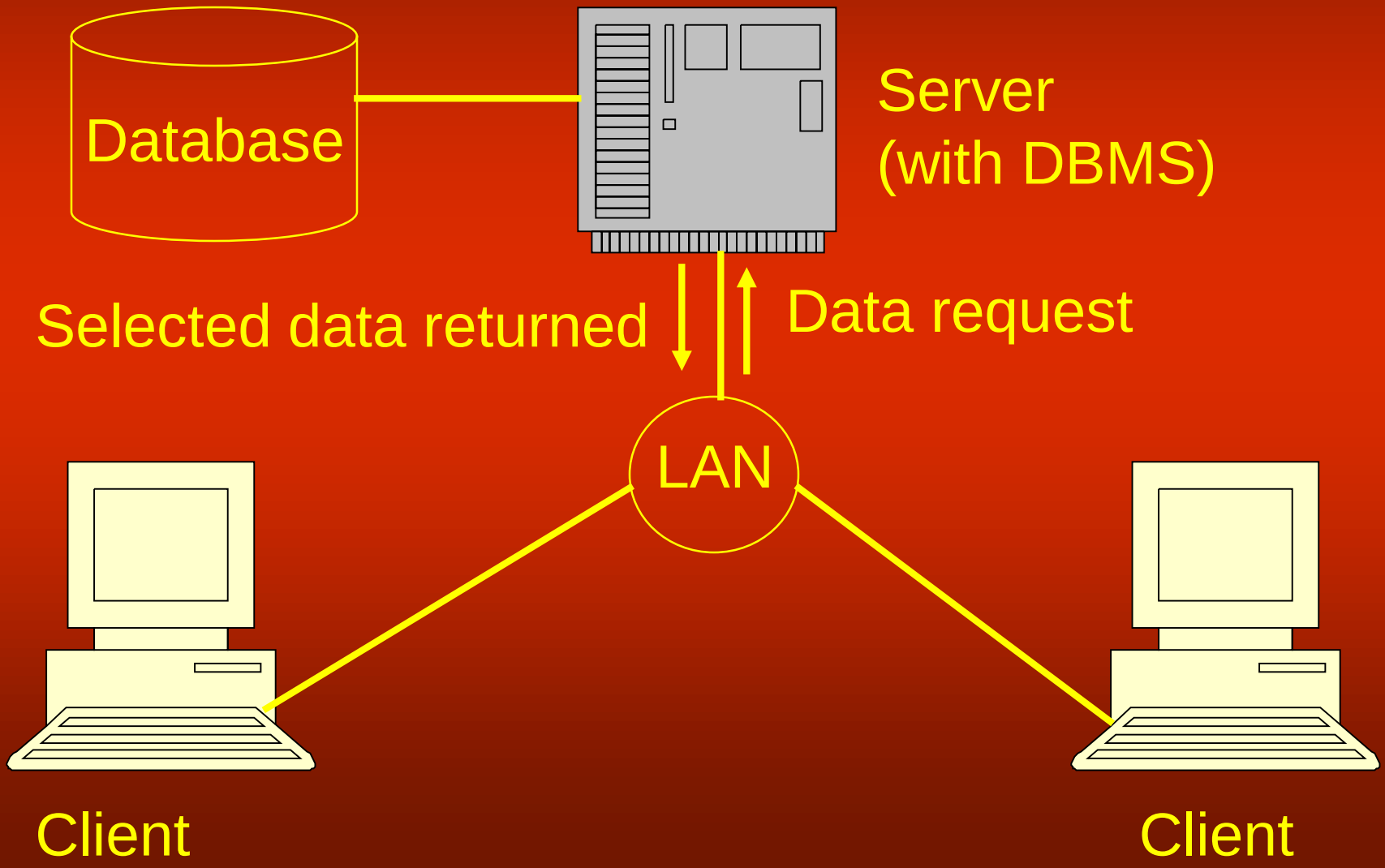
All processing at
a central computer

Dumb Terminals

File Servers



Client-Server



Today's Summary

- Completed 3-L architecture
- Discussed data independence
- Discussion on DBMS

Thanks and Allah Hafiz

Database Management Systems

Lecture - 4

Database Management Systems

Lecture - 5

Lecture's Overview

- Database Application Development Process
- Preliminary Study of System

Database Application Development Process

Involves

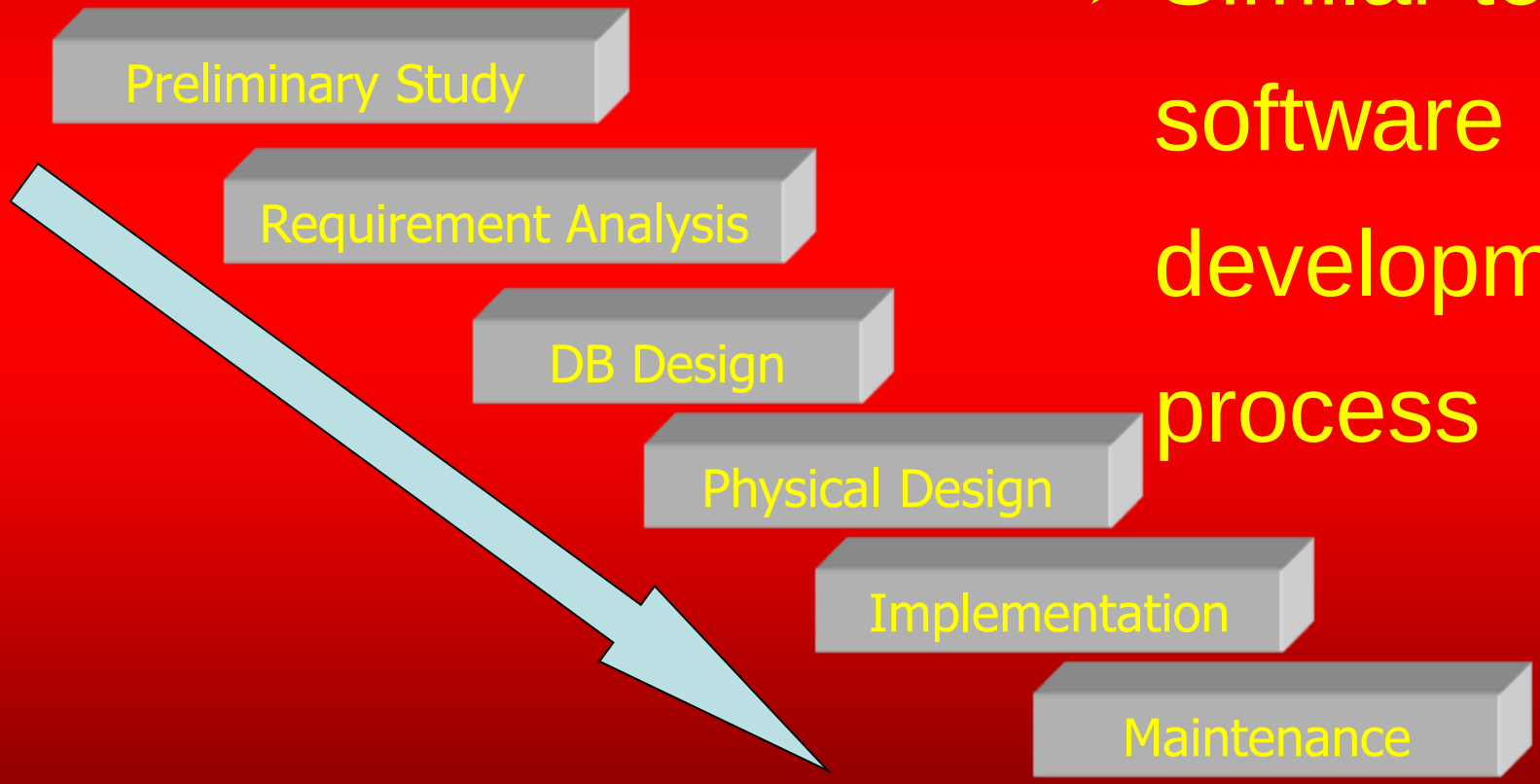
- Database Design
- Application Programs
- Implementation

Database Design

- A DB Design is a model of a particular real-world system
- It provides a picture of reality
- Should be simple and self-explanatory

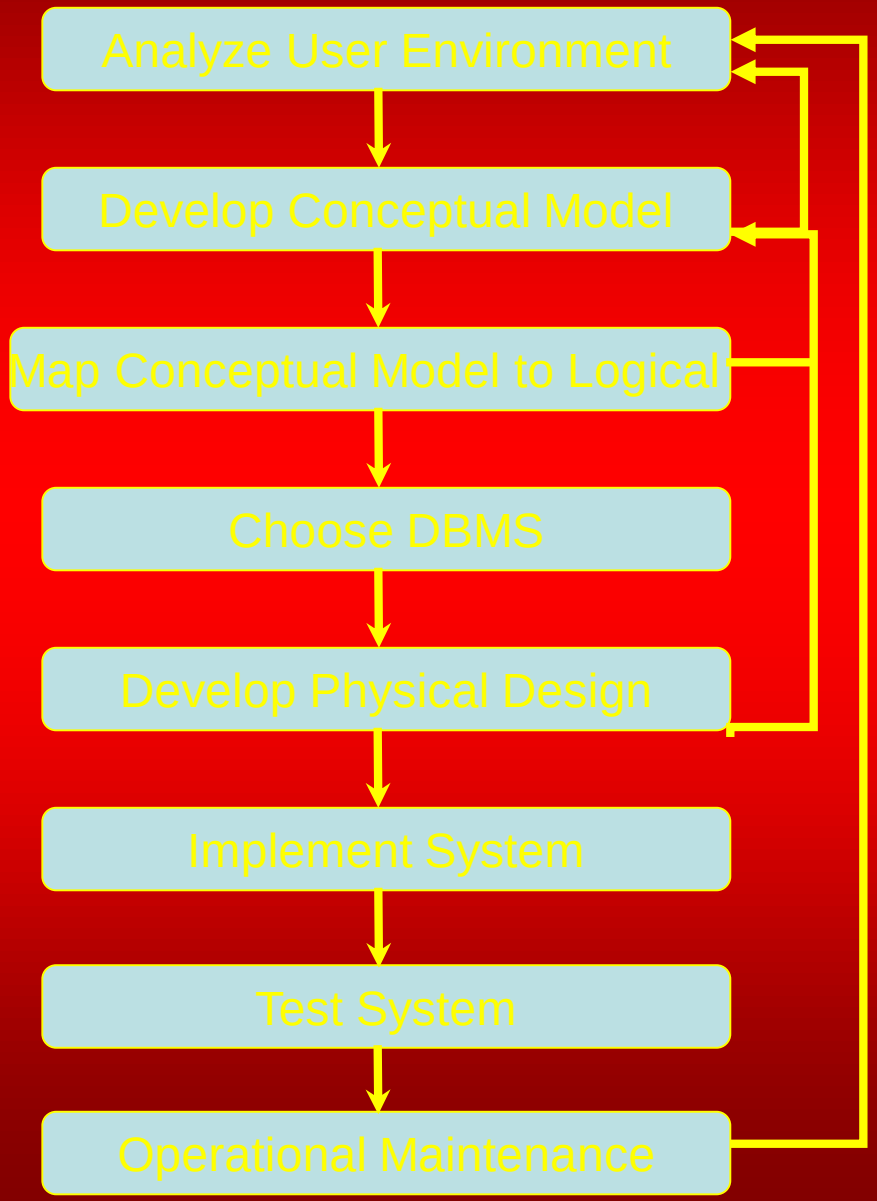
Database Development Process

Database Development Process



- Similar to software development process

Design Stages



Analyze Existing System

- Objective: To understand the working of existing system
- Analyze users' requirements

Tool Used

- Why to use at all?
- Data Flow Diagrams

Data Flow Diagrams

Data Flow Diagrams (DFDs)

- Represent the flow of data between different processes within a system

Data Flow Diagrams

- Simple & intuitive, not focusing on details
- To describe, what users do, rather than what computers do

Data Flow Diagrams

- Limitations
  - Focus only on flows of information
  - Decision points/basis not included

DFD-Symbols

Dataflows: pipelines through which packets of information flow. Arrows are labeled with name of the data that moves through



DFD-Symbols

Data Store

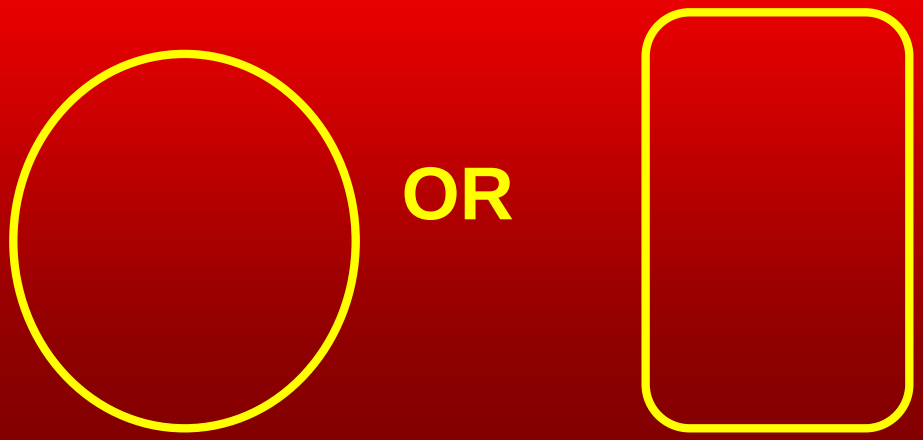
- Repositories of data in system
- Static data
- Data held for processing
- Name is a noun phrase



DFD-Symbols

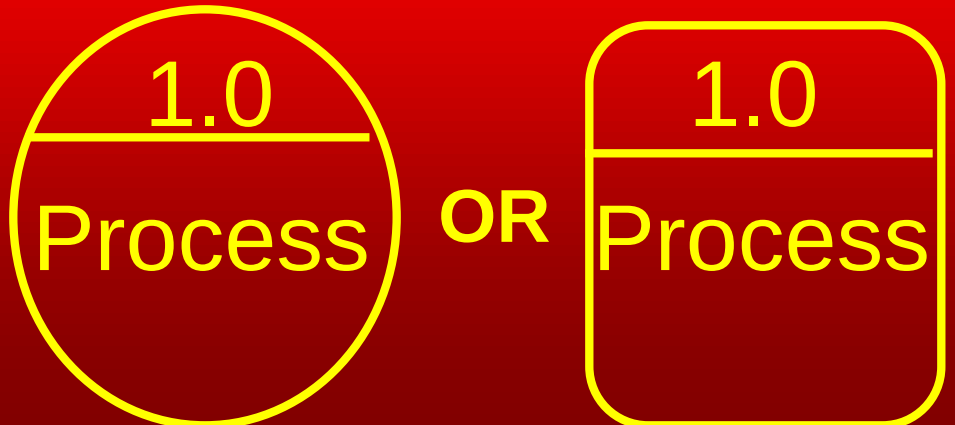
Process

transforms incoming data flow into outgoing data flow



DFD-Process

- Numbered
- Name is verb/object phrase;
- Noun for high-level systems



DFD-Symbols

External Entities

- Sources/destinations for data
- Outside the system
- Name is Noun Phrase



DFD-Symbols

Collector

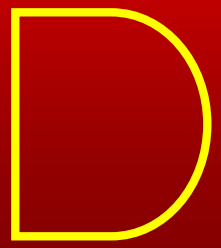
- Several data flows combine here
- No processing occurs here



DFD-Symbols

Separator

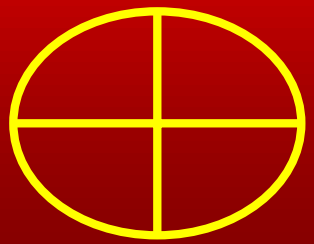
- Several data flows split from here
- No processing



DFD-Symbols

Ring-sum operator

- Shows two possible data flows
- Only one is followed



DFD-Symbols

AND operator

- Shows two data flows
- Both are followed



Types of DFD

- Context diagram
- Level 0 diagram
- Detailed diagram

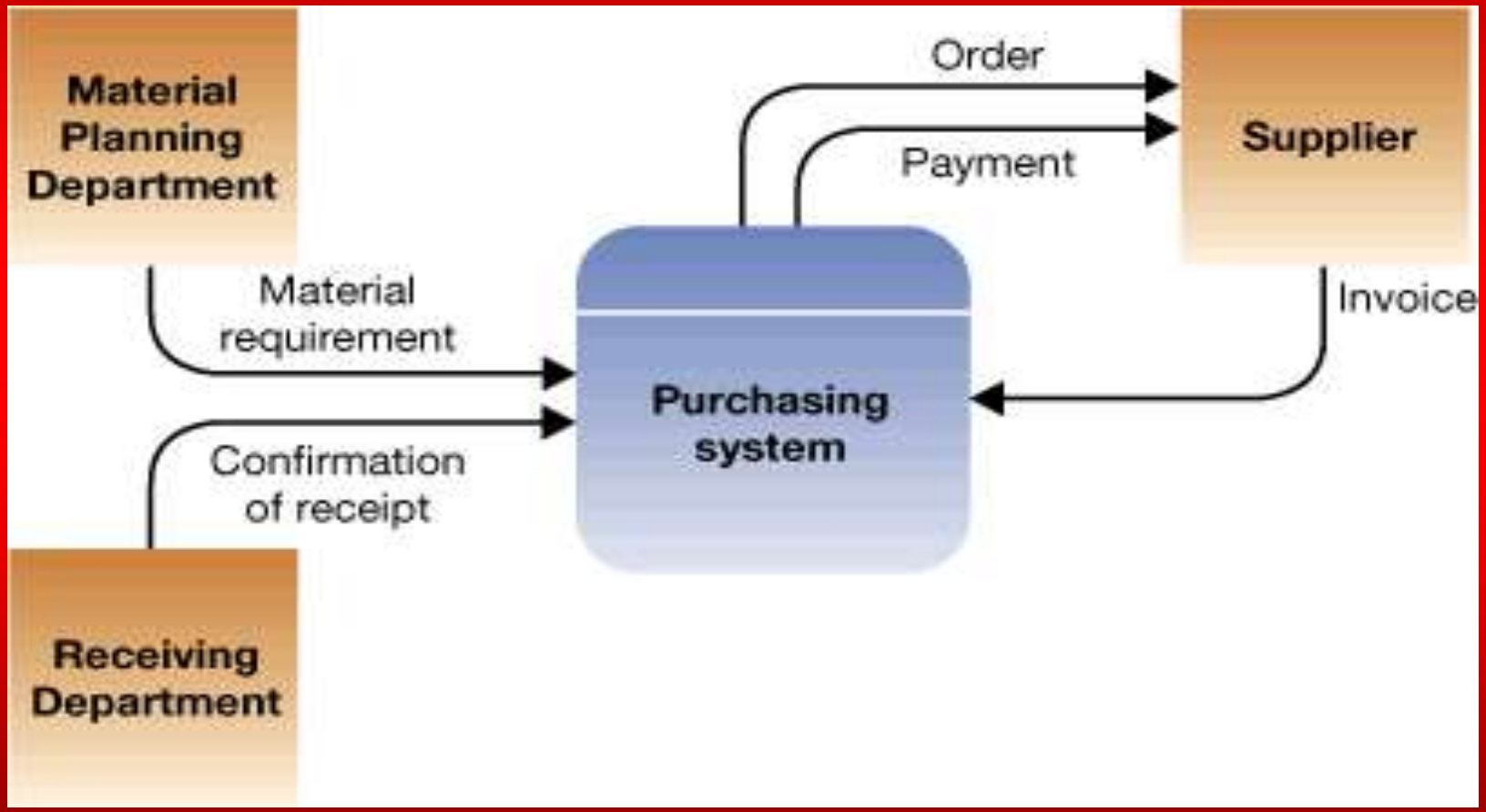
Context Diagram

- Consists of single Process/System
- Represents the system/process being analyzed
- Name: Usually a Noun phrase

Context Diagram

- Show Context Only
- Inputs/outputs
- External Entities
- No Data Stores
- No flows between external entities

Context Diagram



Context Diagram

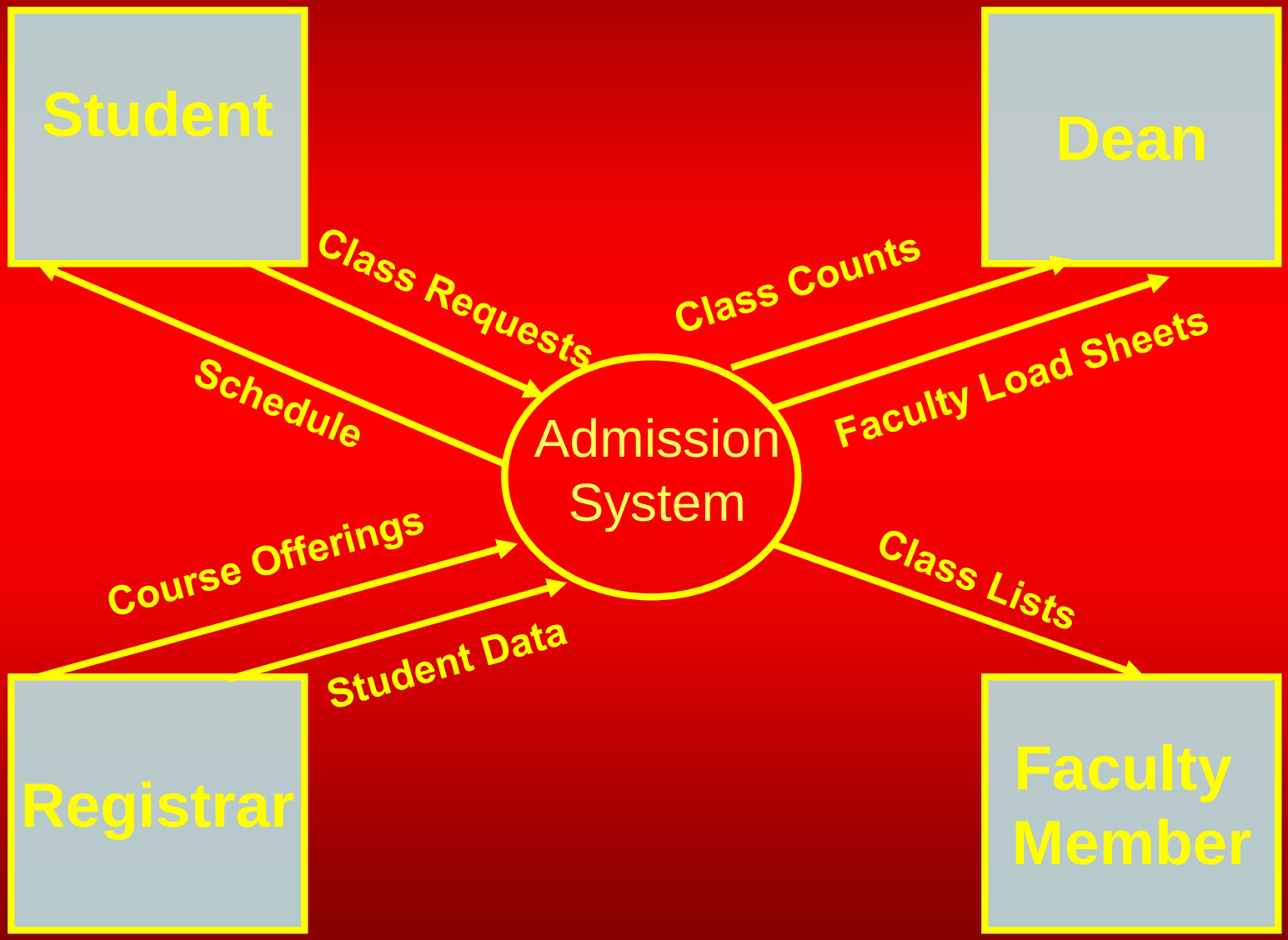
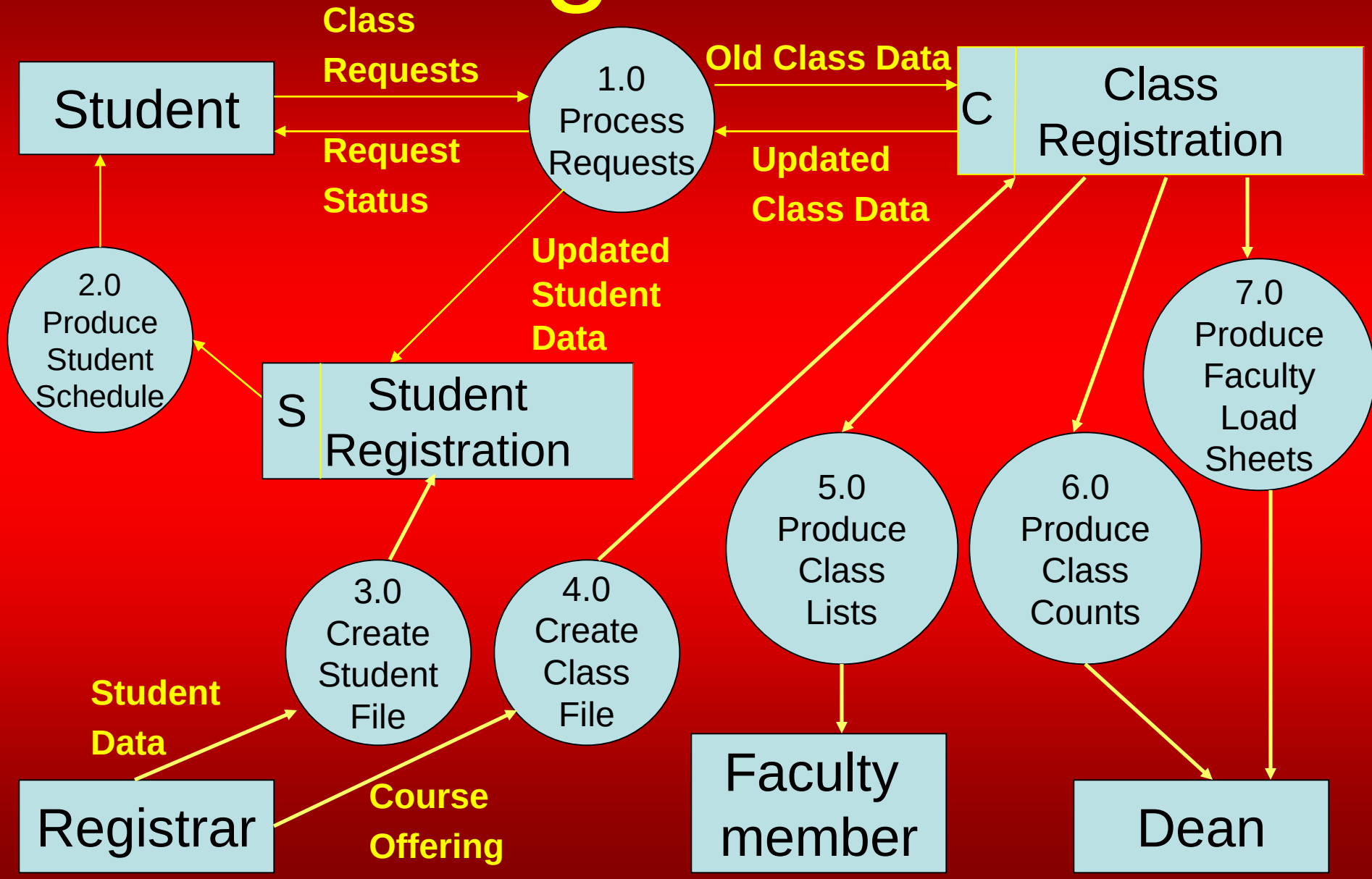
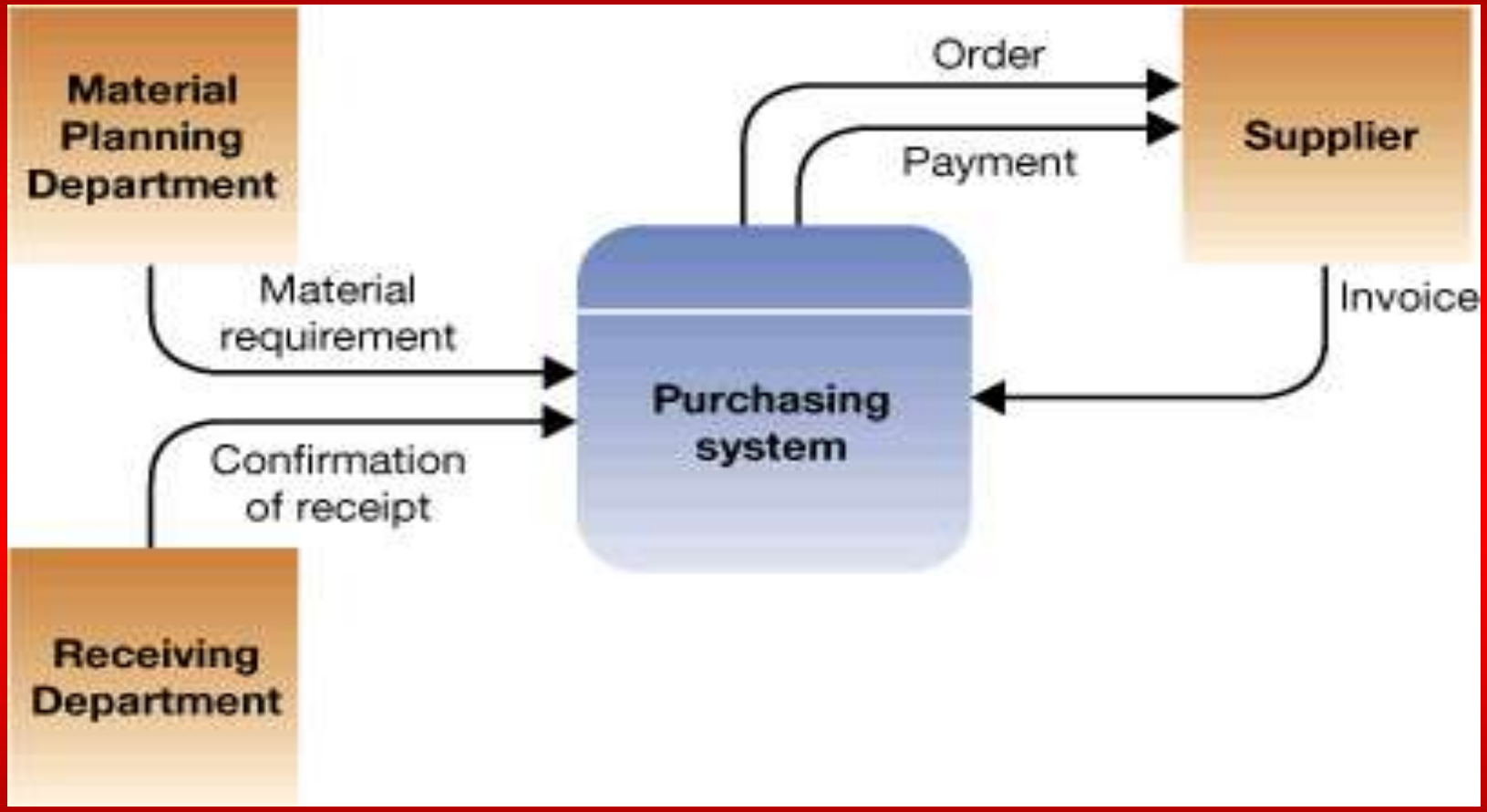


Diagram 0


Class Requests
Student
1.0 Process Requests
Old Class Data
C
Class Registration
Request Status
Updated Class Data
Updated Student Data
2.0 Produce Student Schedule
S
Student Registration
7.0 Produce Faculty Load Sheets
5.0 Produce Class Lists
6.0 Produce Class Counts
3.0 Create Student File
4.0 Create Class File
Student Data
Registrar
Course Offering
Faculty member
Dean

That is all for lecture 5, however,
Due to time available discussed
Level 0 diagram in air, to be included
Later inshAllah


Material Planning Department
Order
Payment
Supplier
Material requirement
Invoice
Purchasing system
Confirmation of receipt
Receiving Department

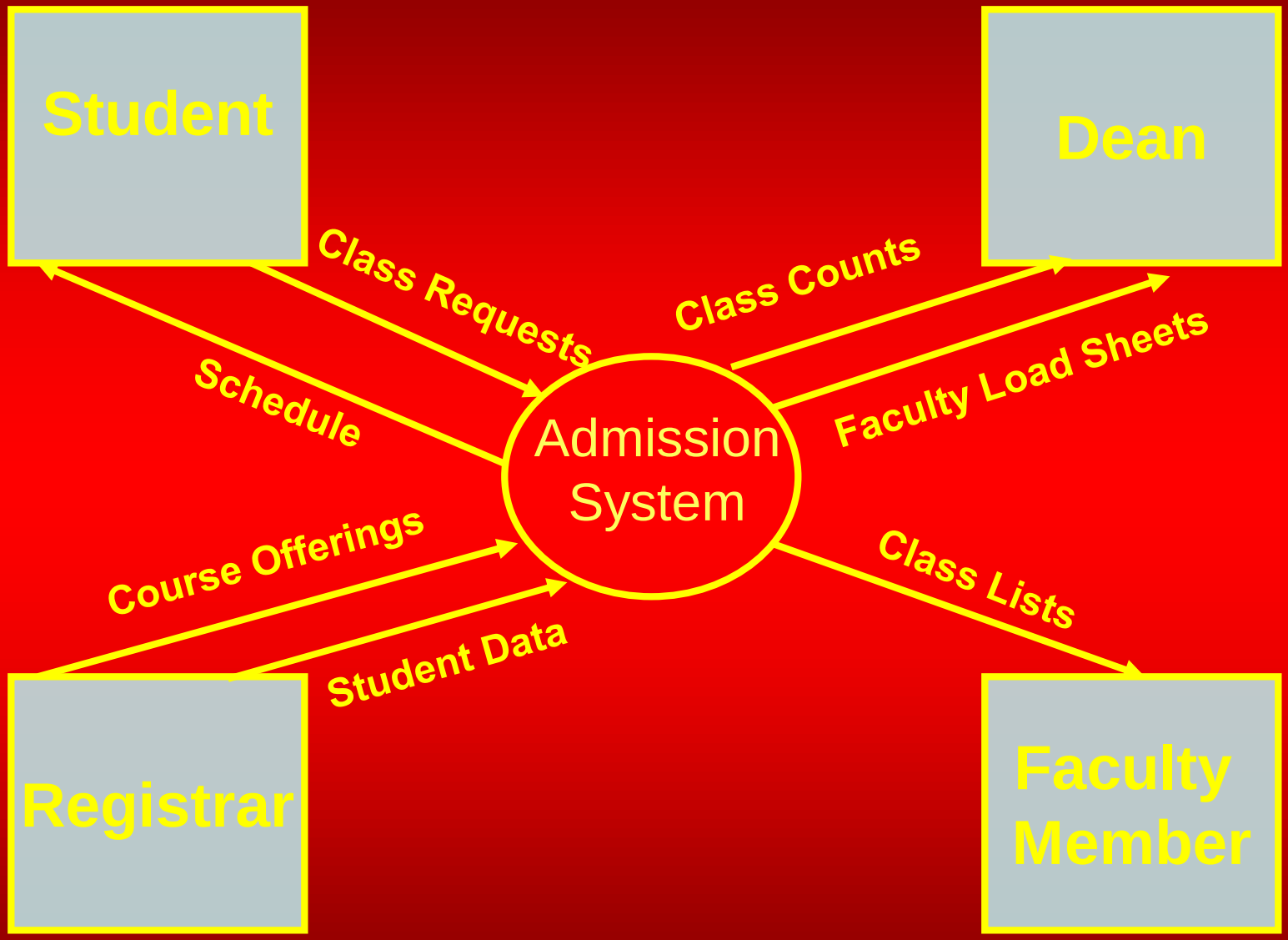
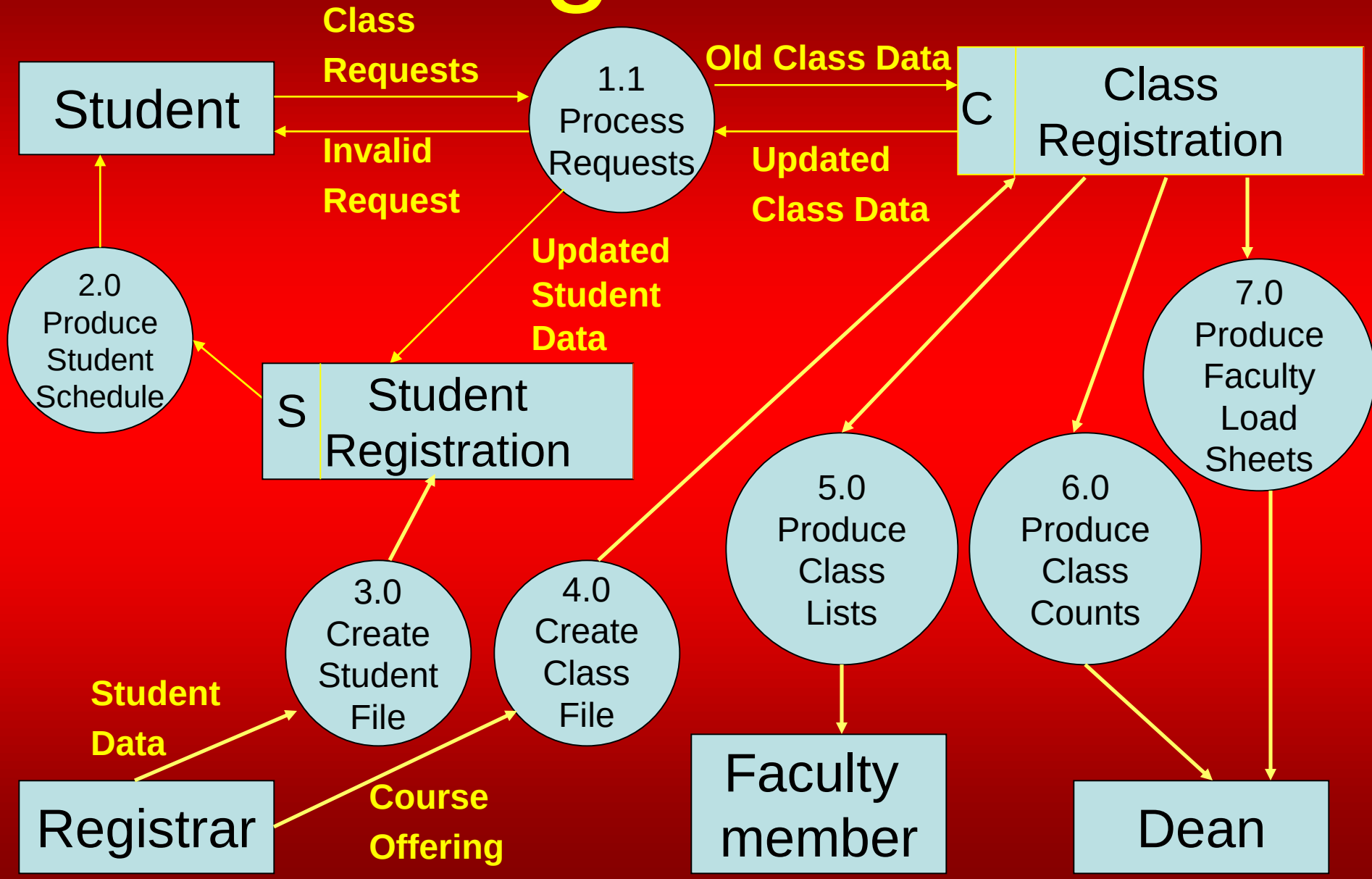
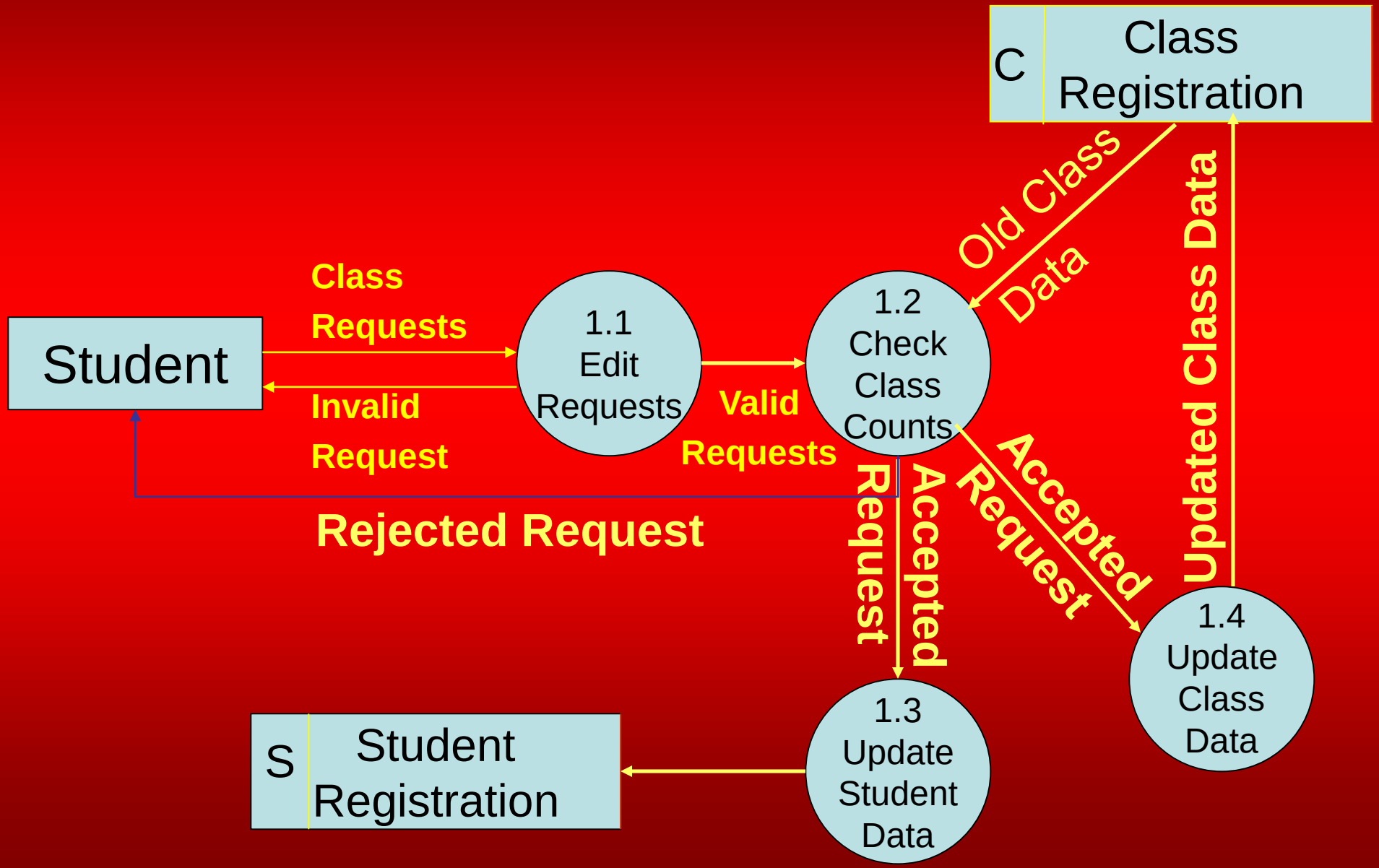
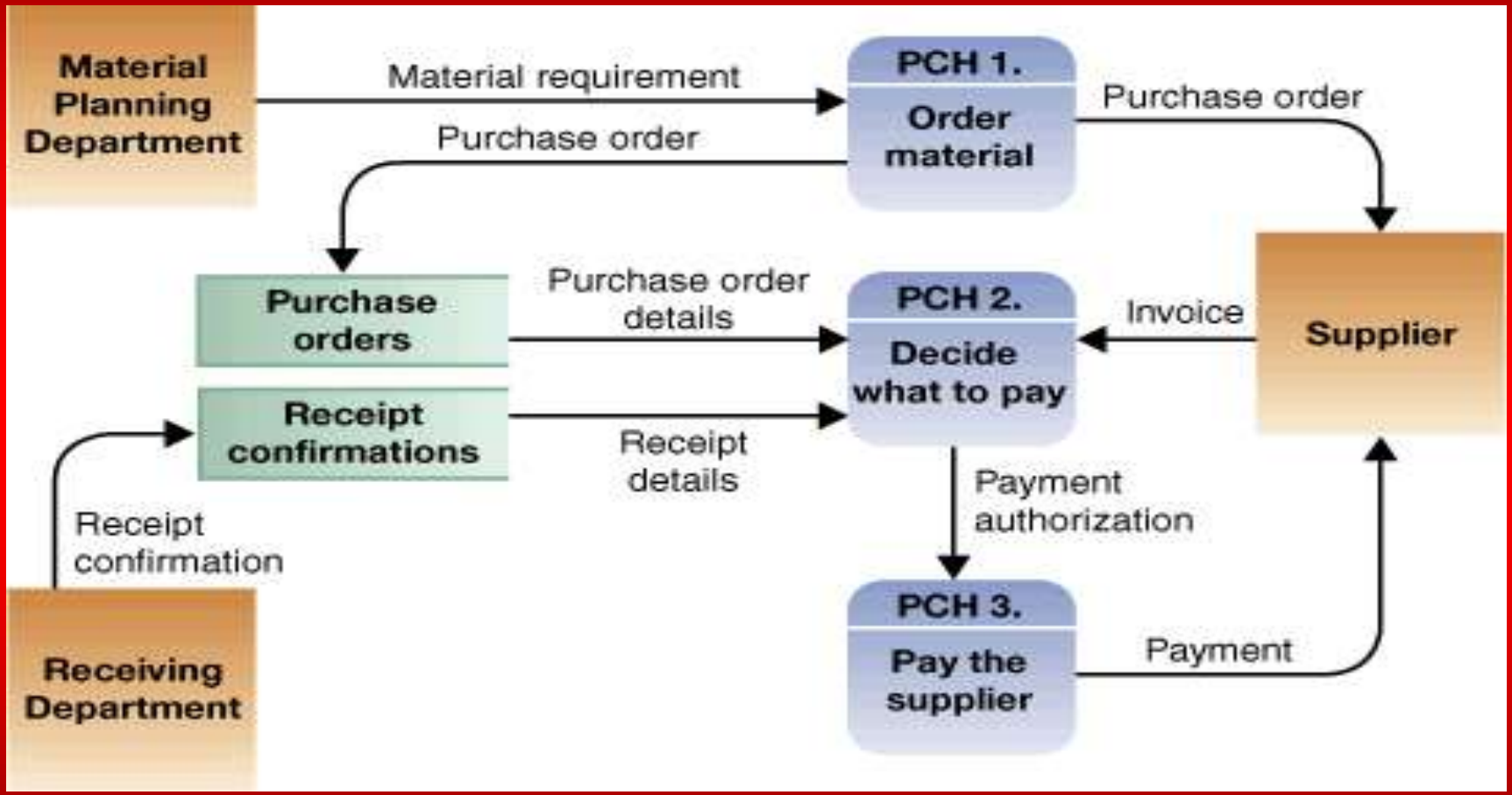

Student
Dean
Class Requests
Class Counts
Schedule
Faculty Load Sheets
Admission System
Course Offerings
Class Lists
Student Data
Registrar
Faculty Member

Diagram 0


Class Requests
Student
1.1 Process Requests
Old Class Data
C
Class Registration
Invalid Request
Updated Class Data
Updated Student Data
2.0 Produce Student Schedule
S
Student Registration
7.0 Produce Faculty Load Sheets
5.0 Produce Class Lists
6.0 Produce Class Counts
3.0 Create Student File
4.0 Create Class File
Student Data
Registrar
Course Offering
Faculty member
Dean

Detailed Diagram




Material Planning Department
Material requirement
PCH 1.
Order material
Purchase order
Purchase order
Purchase orders
Purchase order details
PCH 2.
Decide what to pay
Invoice
Supplier
Receipt confirmations
Receipt details
Payment authorization
Receipt confirmation
PCH 3.
Pay the supplier
Payment
Receiving Department

Overview

- Continue discussion on DFD
- Discussion on Design Phase
- Database Design and Data Models

Database Management Systems

Lecture - 5

Database Management Systems

Lecture - 6

In Previous Lecture

- We were discussing Context Diagram that….

In Previous Lecture

- We were discussing Context Diagram that….
- And also the level 0 Diagram which is ….

Detailed Diagrams

- Show processes in successive levels of detail
- shows major sub-processes

Detailed Diagrams

- Outline numbering
  - 1.1, 1.2, …, 1.n Level one, shows internal details for each major sub-process
  - 1.1.1, 1.1.2,…1.1.n Level two

Detailed Diagrams

- Limit number of process to 7 to 9 per page/diagram for clarity
- All elements must be named

Detailed Diagrams

- All process have both inputs and outputs
  - Outputs must differ from inputs
- Child diagrams and parent process must have same input/output

Detailed Diagram

Data Dictionary

A repository of information that describes logical structure of database, or it contains the metadata

Data Dictionary

- May be integrated or freestanding
- Integrated is referred by DBMS
- Freestanding as a CASE tool

Cross Reference Matrix

- A Data Dictionary tool
- Used to link different things, like functions to entities, or requirements to attributes
- Can be manual even

Cross Reference Table

	Book Issued	Class List	Courses Offered	All Faculty
BooksToFac	✓			
BooksToStu	✓			
FacName		✓	✓	✓
FacAddress				✓
NoOfFac				✓
NoOfStu		✓		
StuName		✓		

Summary

- Analyze User Environment or
- Preliminary Study + Requirements Analysis
- Use of CASE or manual tools

Database Design Phase

Database Design

- Design and Model mean the same
- Database Design represents logical structure of the database

Database Modeling

- Process of creating a logical representation of the structure of the database
- The most important task in database development

Points to Remember

- A database must mirror the real world. Only then can it answer questions about the real world!
- The emphasis of data modeling is on representing reality

Points to Remember

- Ideally it should be a represented graphically
- The goal is to identify the facts to be stored in the database
- Data modeling involves users and analysts

Data Model

- Created using a data model
- Data Model: is a set of tools or constructs that is used to construct a database design

Components of a DM

- Structures
- Manipulation language
- Integrity constraints
- Beep beep, caution

Significance of DM

- Facilitates and provides guidelines or rules in the DB design process
- Every DBMS is based on a DM

Types of Data Models

- Semantic: Entity-Relationship, Object-Oriented
- Record based: Hierarchical, Network, Relational

DB Design Types

- Conceptual: using SDM
- Logical: using DM of tool
- Physical: using the DBMS

Database Management System

Lecture - 6

Database Management System

Lecture - 7

Entity-Relationship Data Model

E-R Data Model

A semantic data model, used for the graphical representation of the conceptual database design

Major Components

- Entities
- Attributes
- Relationships

Entity

- Term used to mean three different meanings
  - Entity type
  - Entity instance
  - Entity set

Entity Type

- A name/label assigned to items/objects that exist in an environment and that have similar properties
- It could be person, place, event or even concept

Entity Type

- Distinguishable from other entity types on the basis of properties
- Identified through abstraction process
- Different from External Entity

Entity Instance & Set

- A particular object belonging to a particular entity type
- Entity Type: Employee
- Entity Instance: M. Sharif
- Entity Set: All employees

Types of Entity Types

Entity types can be classified into regular/strong/independent ETs or weak/dependent ETs

Weak Entity Types

An entity type whose instances cannot exist without being linked with instances of some other entity type, i.e., they cannot exist independently

Strong Entity Type

- A strong/regular entity type is the one whose instances can exist independently, i.e., without being linked to other instances
- Strong ETs have their own identity

Naming Entity Types

- Singular noun recommended
- Organization specific names
- Write in capitals
- Abbreviations can be used, be consistent

Symbols

Regular Entity Type

NAME

Weak Entity Type

NAME





Attribute

An **attribute** of an entity type is a defining property or quality of the instances of that entity type. Entity instances of same entity type have the same attributes. (e.g. Student Identification, Student Name)

Domain of an Attribute

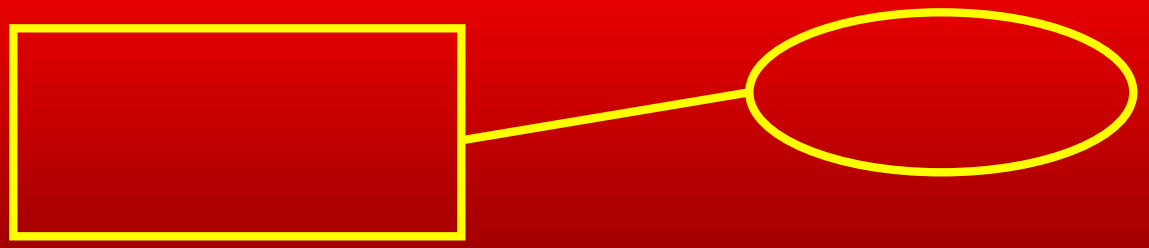
- Every attribute has a domain
- Set of possible values for an attribute
- The attributes in an entity set get the values from the same domain

Types of Attributes

- Single vs composite
- Single valued vs multi-valued
- Stored vs derived

Symbols for Attributes

- Each represented as an oval, linked with an ET symbol



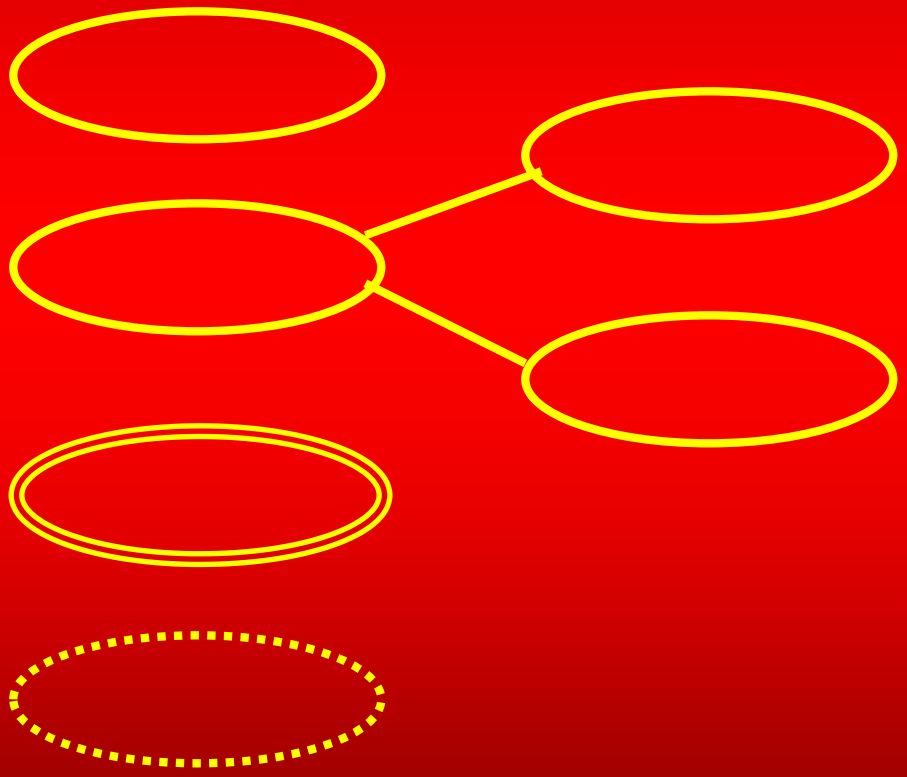
Symbols for Attributes

Simple

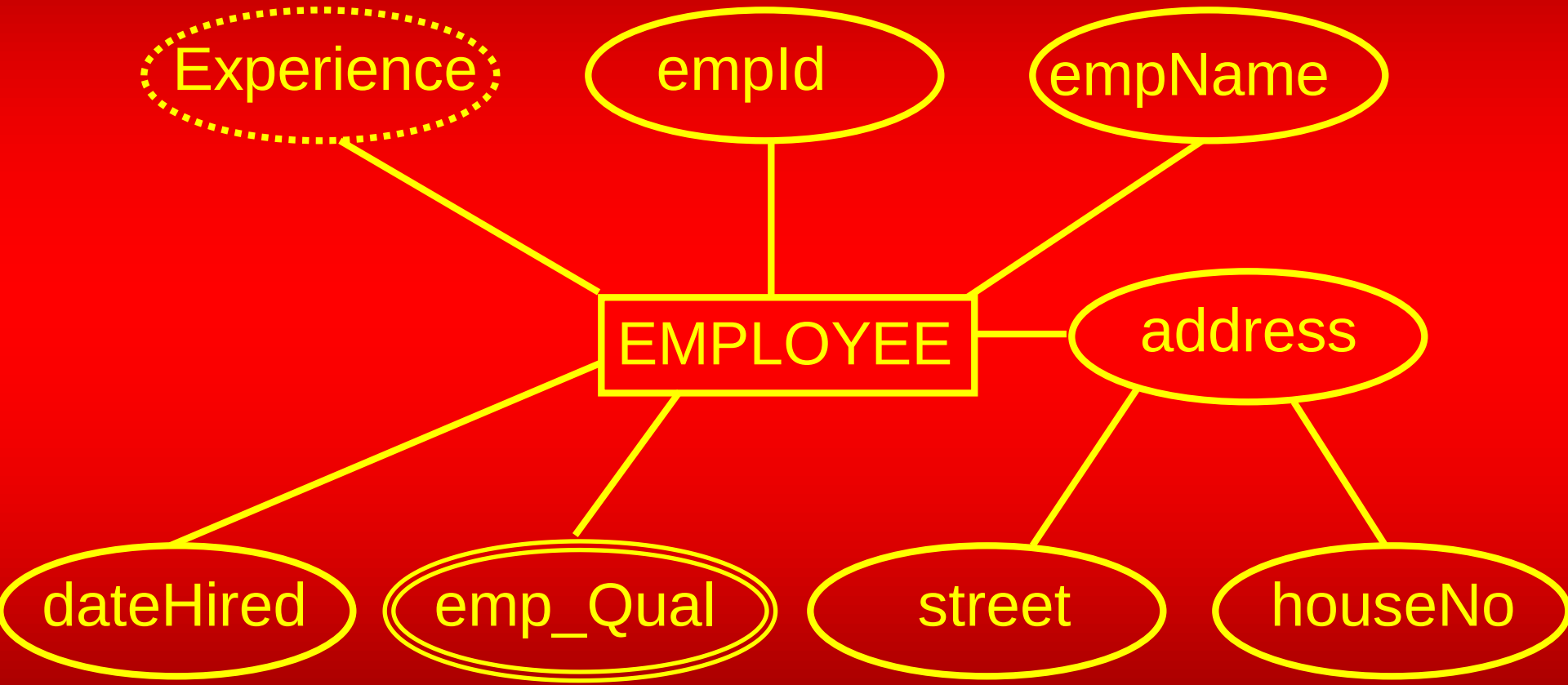
Composite

Multi-valued

Derived



Example


Experience
empId
empName
EMPLOYEE
address
dateHired
emp_Qual
street
houseNo

Thanks

Database Management System

Lecture - 7

Database Management System

Lecture - 8

Today's Overview

- Key and its different types
- Relationships in E-R Data Model
- Types of relationships

Key Attributes

- An attribute or set of attributes to identify an entity instance uniquely
- Types
  - Super key
  - Candidate key
  - Primary key
  - Secondary and Alternate keys

Example of Key

StdId	StdName	Address	ClName	CurSem
S1020	Suhail Dar	Mareer Hassan	MCS	4
S1038	Shoaib Baber	Model Town	BCS	3
S1015	Tahira Ejaz	Wah Cantt	MCS	2
S1018	Arif Mehmood	Satellite Town	BIT	4
S1025	Suhail Shah	Garhi Shahoo	BCS	6

Simple or Composite Key

- A key consisting of single attribute is called simple key, e.g., StudID, itemNo
- A key consisting of more than one attribute is known as composite key, like {Program_Code,Course_Code}

Composite Key Example

OFFERING

ProgCode	**CourseCode**	**MarksAlloc**	**CrHrs**
MCS	DS	100	3
MCS	DBS	100	3
MBA	DBS	100	3
BCS	NW	100	3

Super Key

- Definition same as of key
- For example, for EMPLOYEE and STUDENT entity types EmpID and StudID are the superkeys respectively.

Composite Key Example

StdId	StdName	Address	ClName	CurSem
S1020	Suhail Dar	Mareer Hassan	MCS	4
S1038	Shoaib Baber	Model Town	BCS	3
S1015	Tahira Ejaz	Wah Cantt	MCS	2
S1018	Arif Mehmood	Satellite Town	BIT	4
S1025	Suhail Shah	Garhi Shahoo	BCS	6

Super Key

Any set of attributes containing a super key is also a super key since it too uniquely identifies an entity e.g. {StudID, major}

Super Keys

StdId	StdName	Address	ClName	CurSem
S1020	Suhail Dar	Mareer Hassan	MCS	4
S1038	Shoaib Baber	Model Town	BCS	3
S1015	Tahira Ejaz	Wah Cantt	MCS	2
S1018	Arif Mehmood	Satellite Town	BIT	4
S1025	Suhail Shah	Garhi Shahoo	BCS	6

Candidate Key

A candidate key is the super key that does not contain extra attributes. It might have more than one attribute that uniquely identifies an entity. e.g {name, address}

Candidate Keys

A super key such that no proper subset of its attributes is itself a super key. e.g. {StudID, Major} is not a candidate key because it contains a subset, StudID, that is a super key

CK Example

StdId	StdName	Address	ClName	CurSem
S1020	Suhail Dar	Mareer Hassan	MCS	4
S1038	Shoaib Baber	Model Town	BCS	3
S1015	Tahira Ejaz	Wah Cantt	MCS	2
S1018	Arif Mehmood	Satellite Town	BIT	4
S1025	Suhail Shah	Garhi Shahoo	BCS	6

Primary Key

A primary key is the main/chosen candidate key from the possible set of candidate keys that is most suitable for entity identification.

Primary Key

- It may be a single attribute or a composite key.
- None of its attributes can have ***NULL*** values.

Primary Key

The other candidate keys called Alternate keys provide another method of accessing records.

Need for Key

- Need for unique identification and access
- Secondary Keys: We access on something not necessarily unique

Database Management System

Lecture - 8

Database Management System

Lecture - 9

Relationships

Relationships

- Relationships are the *connections and interactions* between the entities instances, e.g., Program and Student ETs are linked
- How to identify relationships

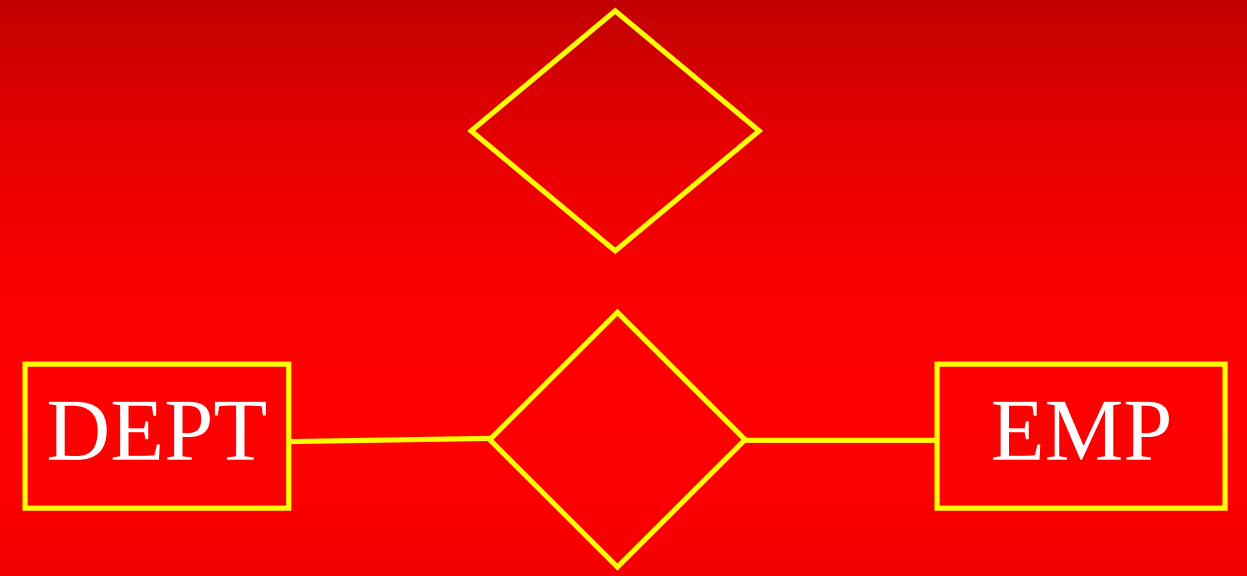
Naming Relationships

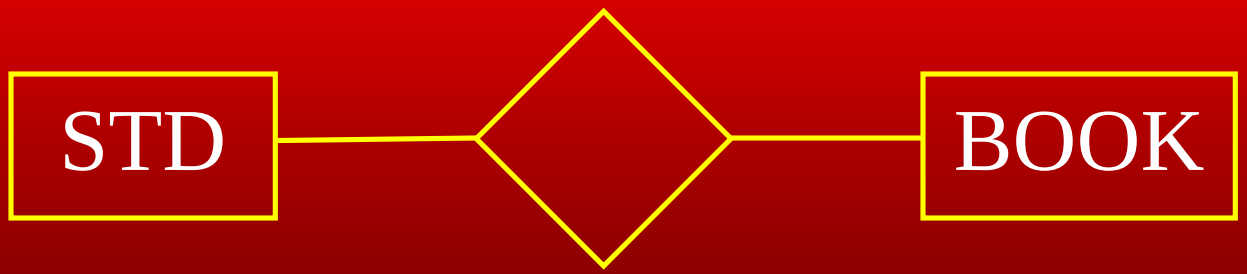
- Up to you
- If there is no proper name of the association in the system then participants' names of abbreviations are used

Naming

- STUDENT and CLASS have ENROLL relationship
- However, it can also be named as STD_CLS

Symbol for Relationships





Relationships

- Relationship type can be identified like an entity type
- A relationship type is an abstraction of a relationship

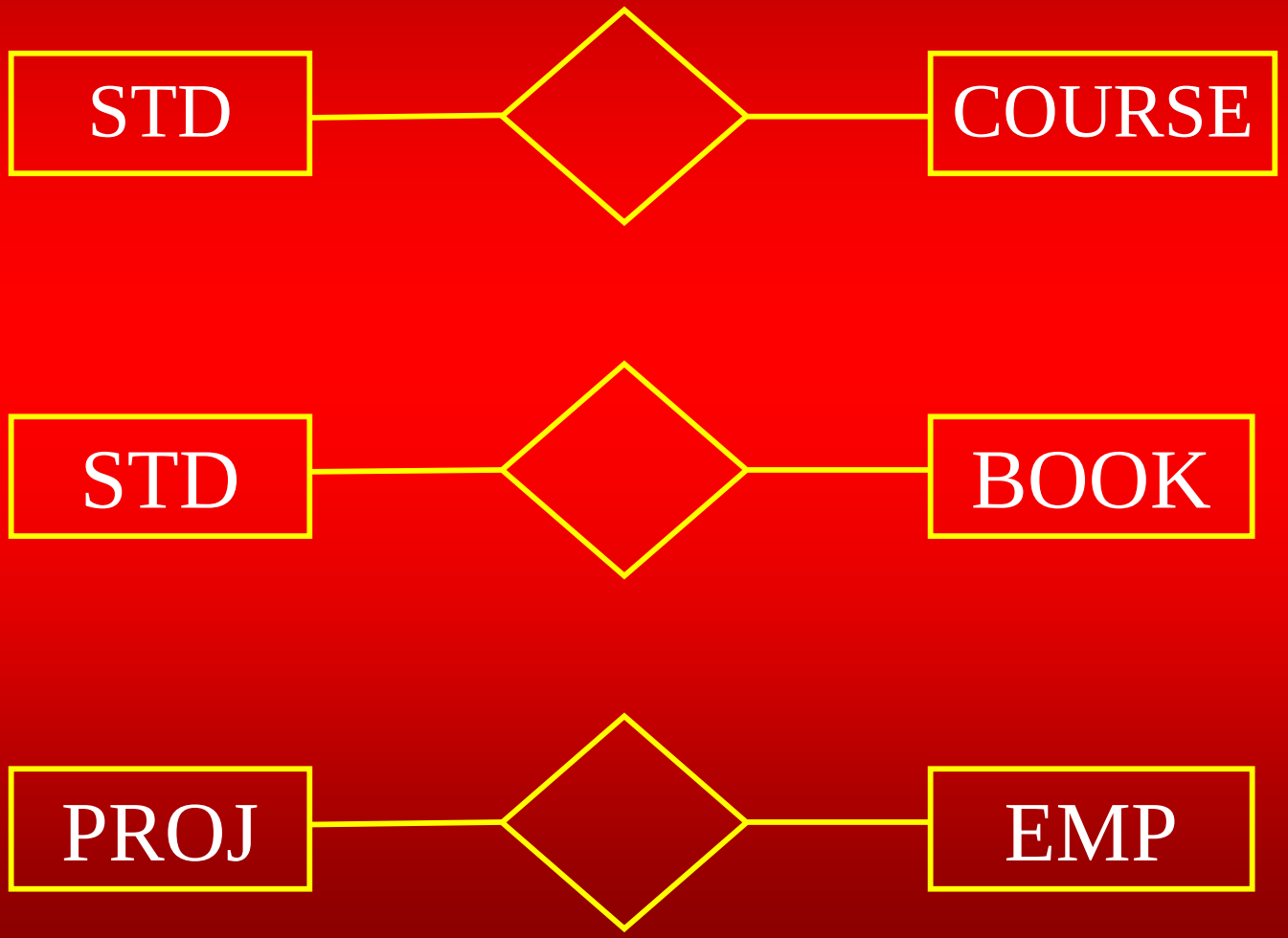
Relationships

- Entities involved in a relationship are called its participants
- Types of the relationships can be established on the basis of participant ETs

Relationships Types

A Binary relationship is the one that links two entity types e.g. STUDENT-CLASS

Binary Relationship Example



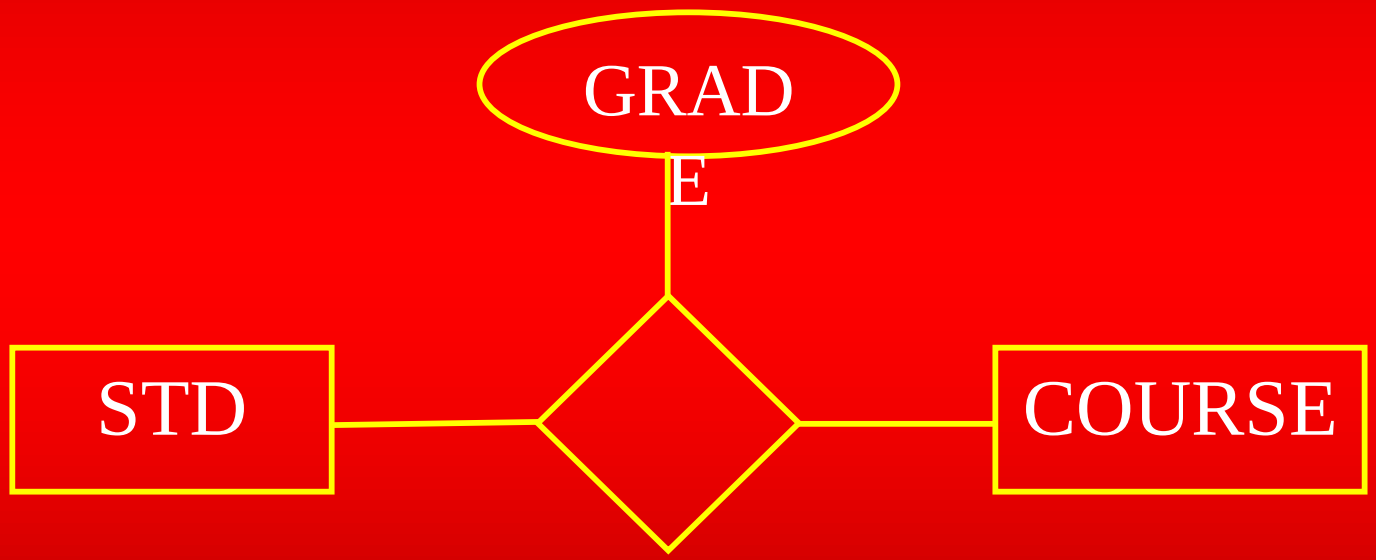
Binary Relationships

- May also have instances, that can be formally described in an ordered pair form
- {(S1001, OS), (S1020, DS), (S1002, DS), (S1058, NW)}

Attributes of the Rships

- The key
- The relationships can have their descriptive attributes
- Where to place

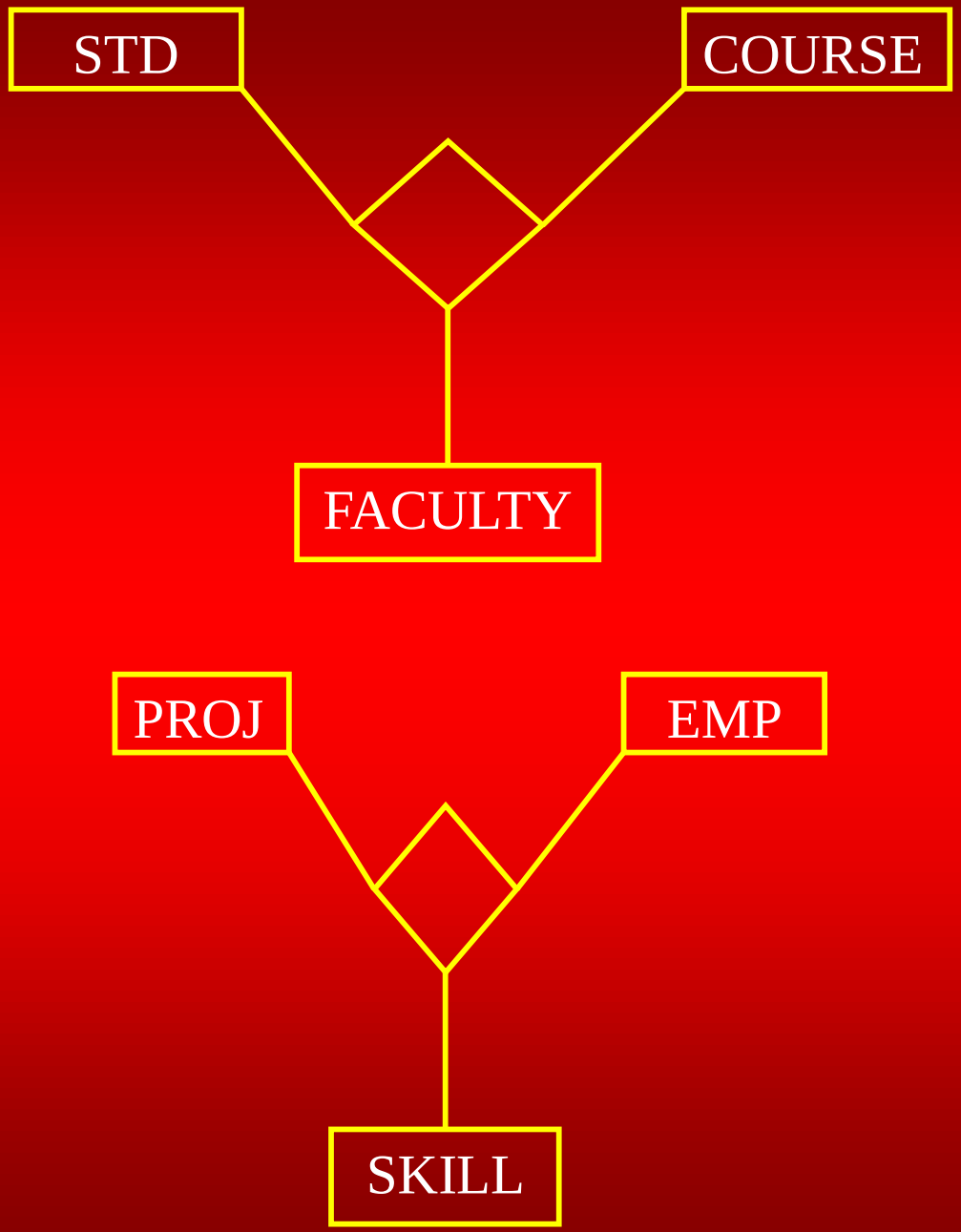
Attributes of Rships



Ternary Relationships

- One that involves three entity types
- STUDENT-CLASS-FACULTY

Ternary Relationship Examples


STD
COURSE
FACULTY
PROJ
EMP
SKILL

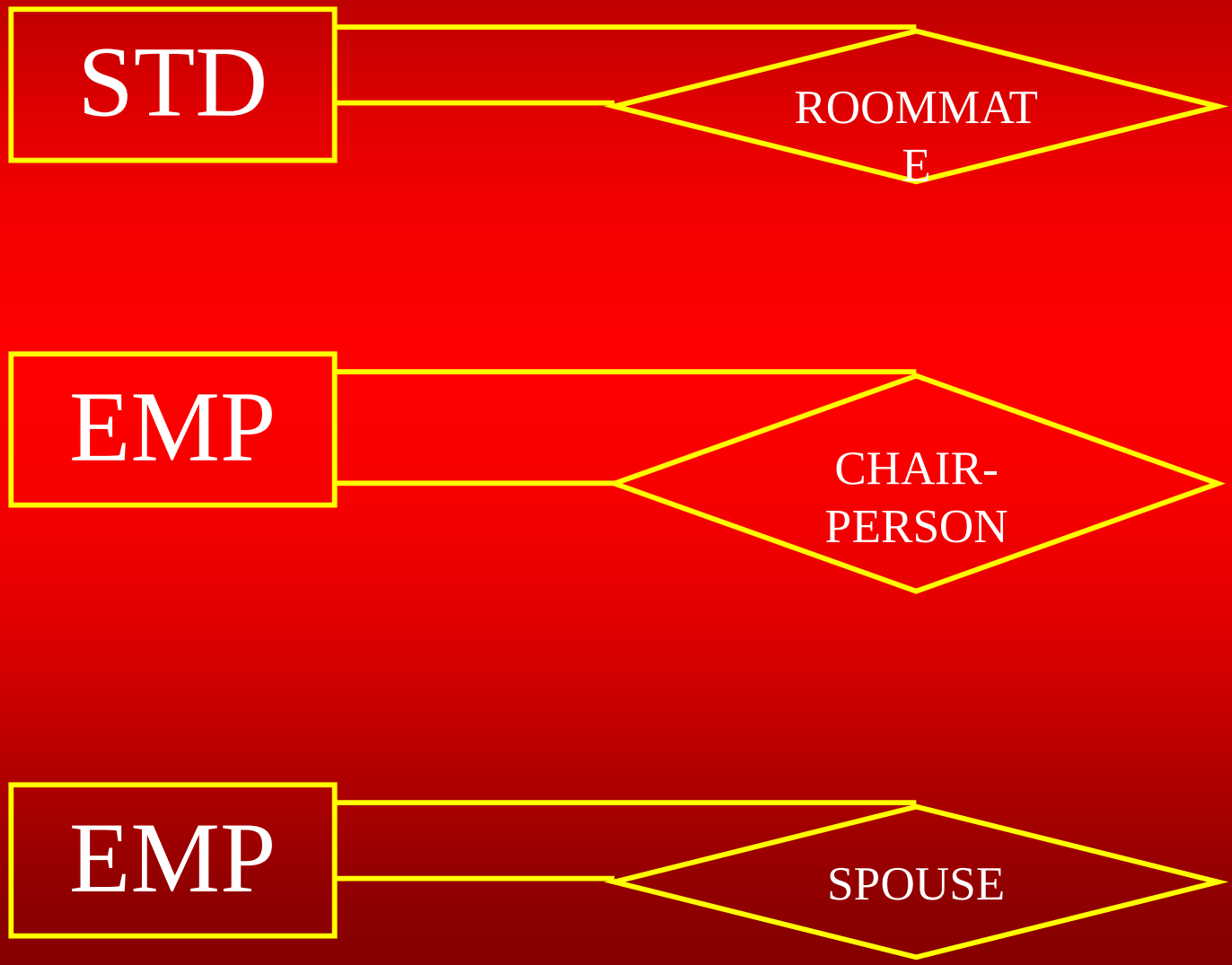
Ternary Relationships

- Instances in ordered triples
- Example {(S1013, MCS4, Adnan), (S1023, MCS3, Fasih)}

Unary Relationship

- An ET linked with itself, also called recursive relationship
- Example Roommate, where STUDENT is linked with STUDENT

Unary Relationship Examples



Cardinality of Rships

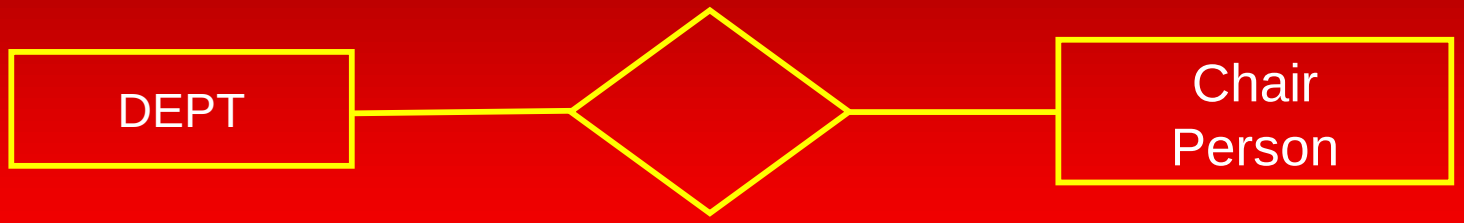
Number of instances of one entity type that can possibly be related to instances of other entity type

Types of Cardinalities

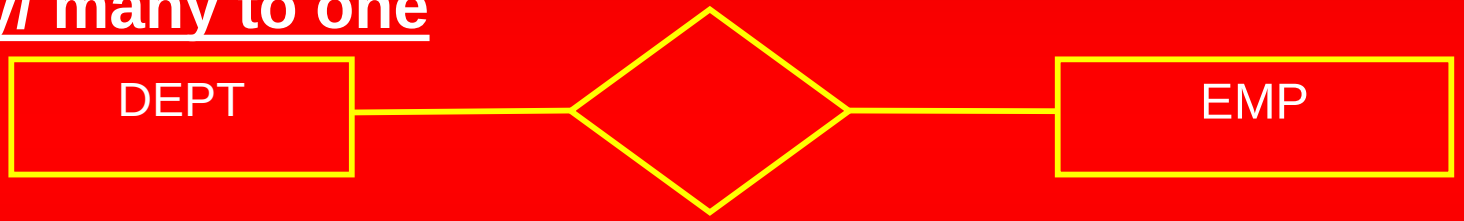
- One to one
- One to many
- Many to one
- Many to many

Types of Cardinalities

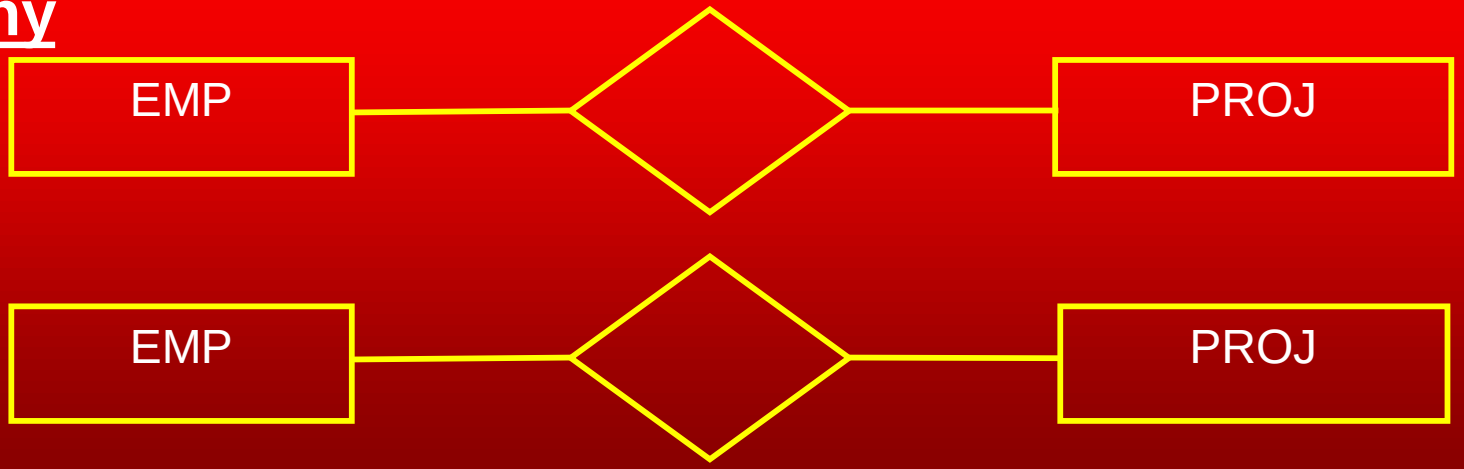
One to one



One to many/ many to one



Many to many



Database Management System

Lecture - 9

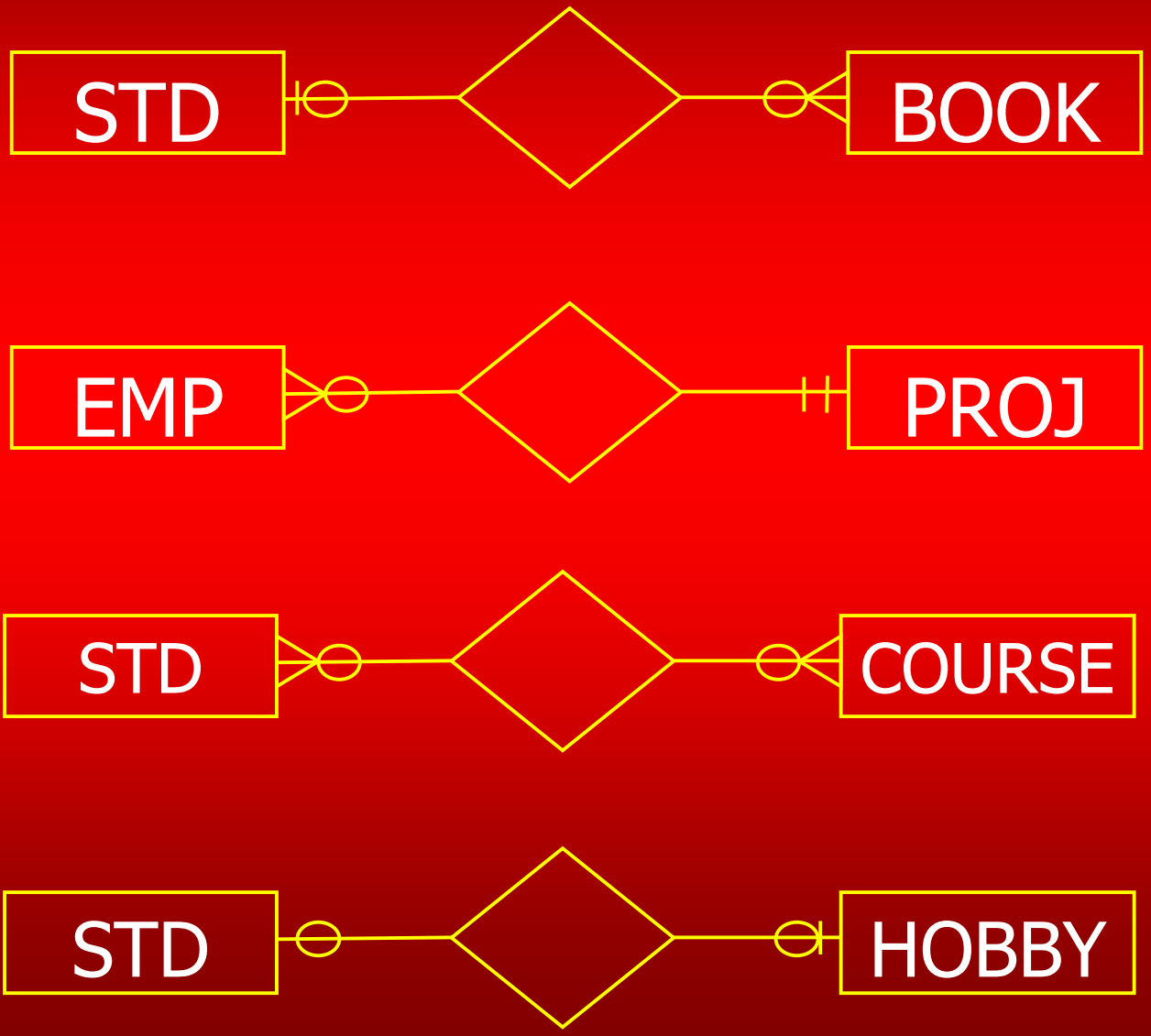
Database Management System

Lecture - 10

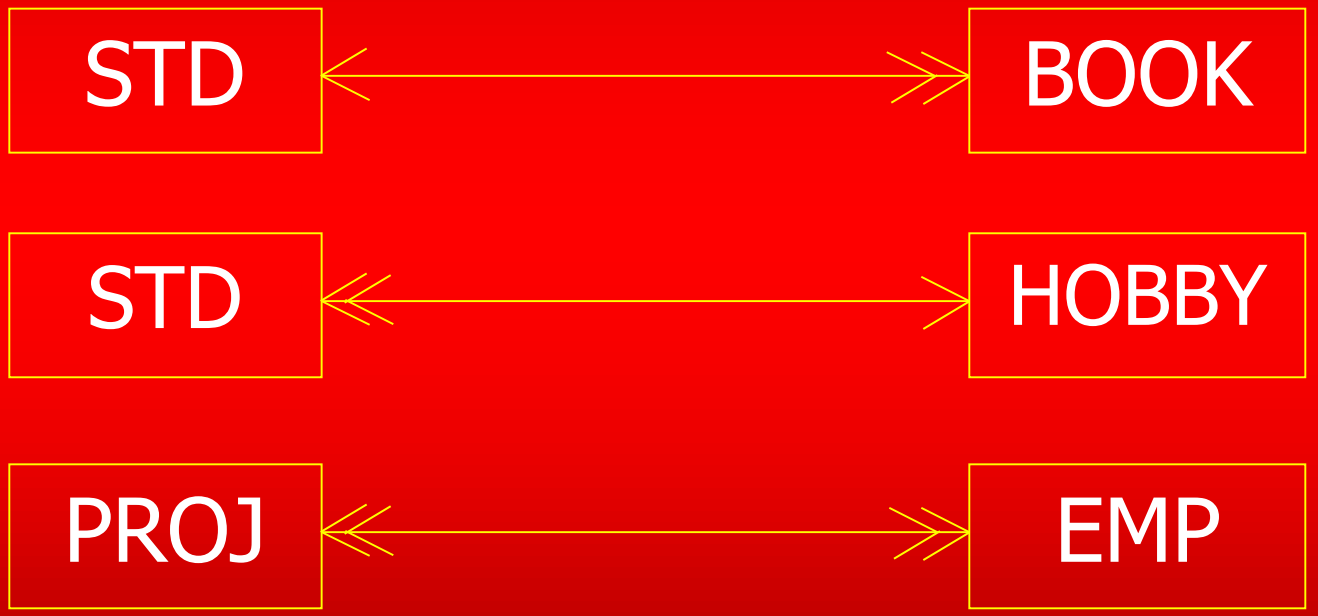
Minimum Cardinality

- Determines whether the link is compulsory or optional
- Important, since it effects the implementation

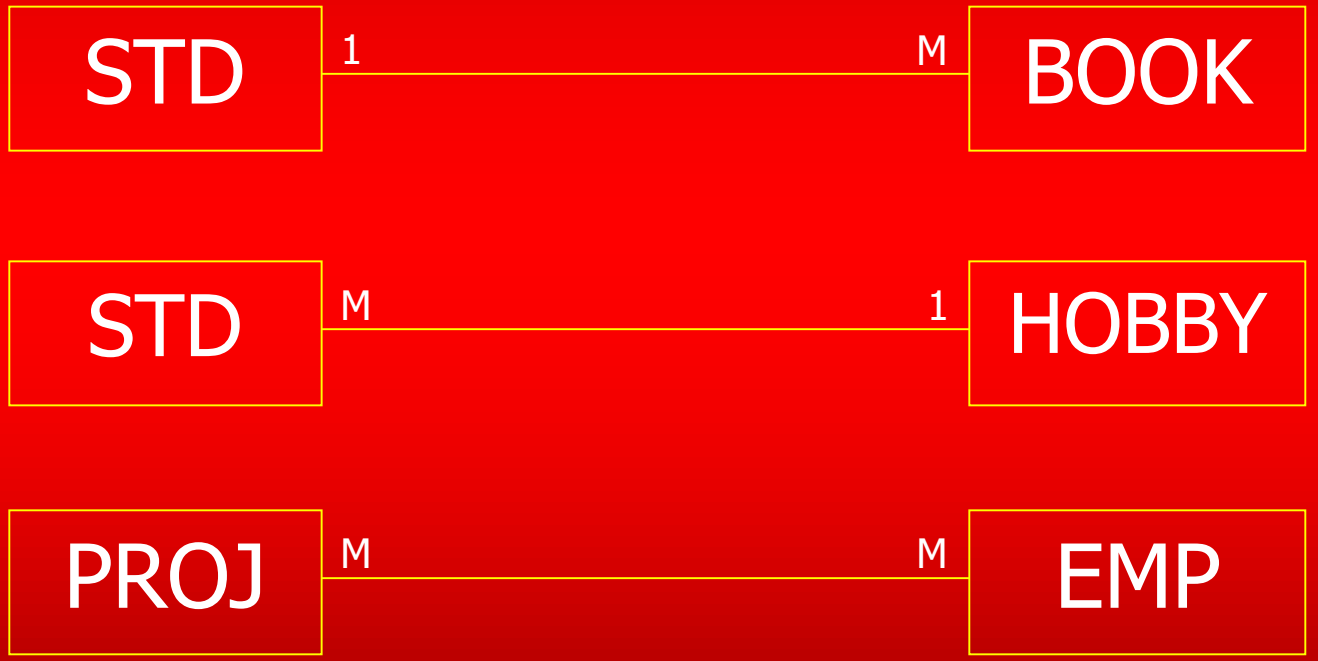
Cardinality Example



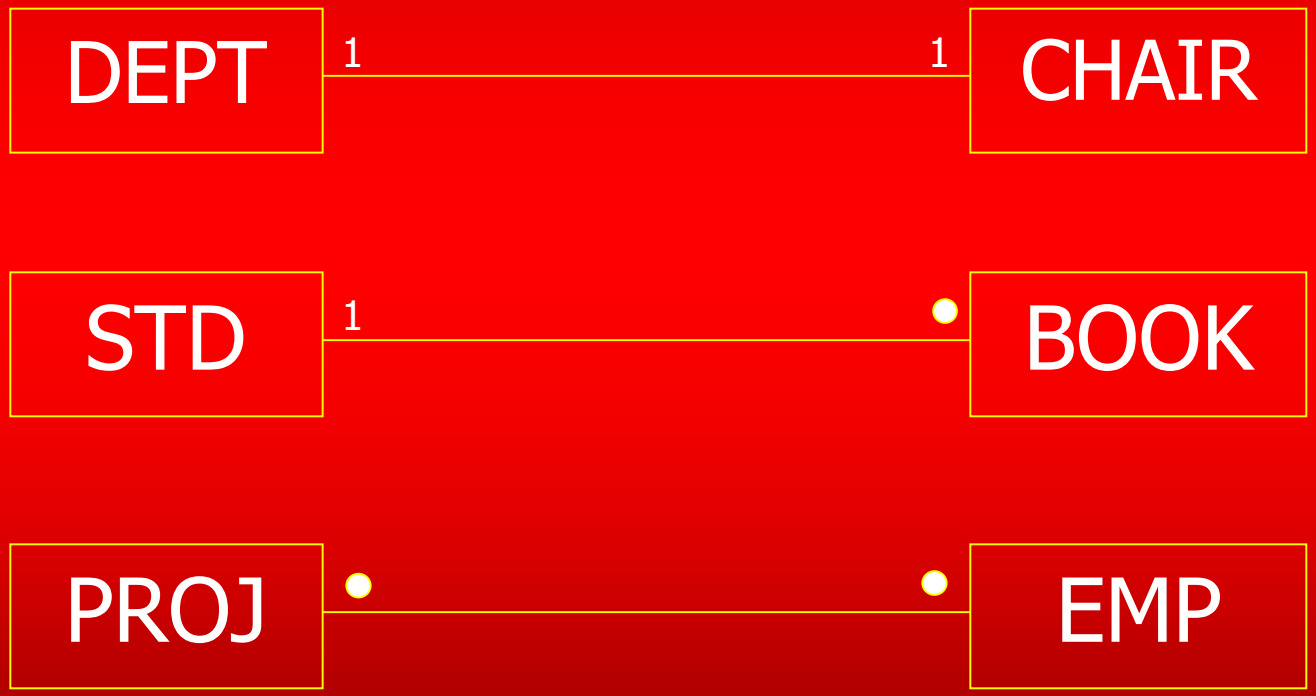
Other Notations



Other Notations



Other Notations



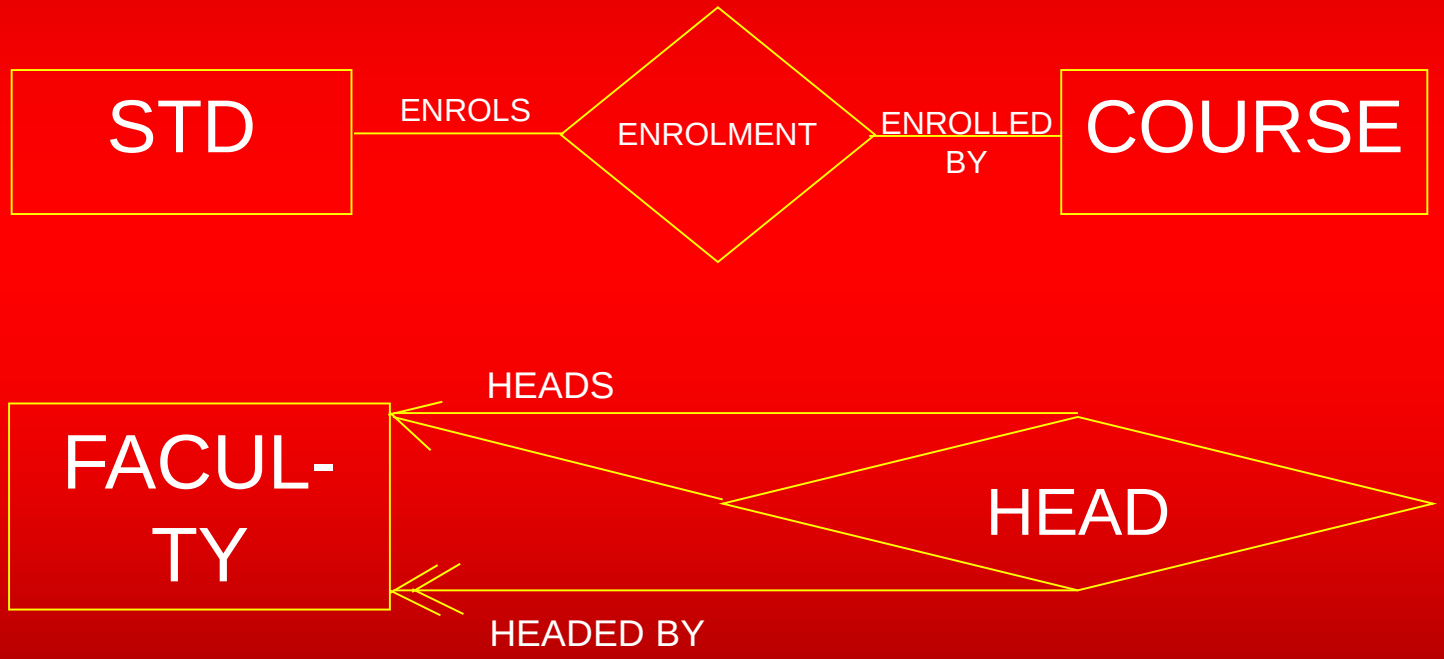
Roles in Relationships

- Determine the role ETs play in a relationship
- Most of the time is clear from the context, like in STD and COURSE relationship

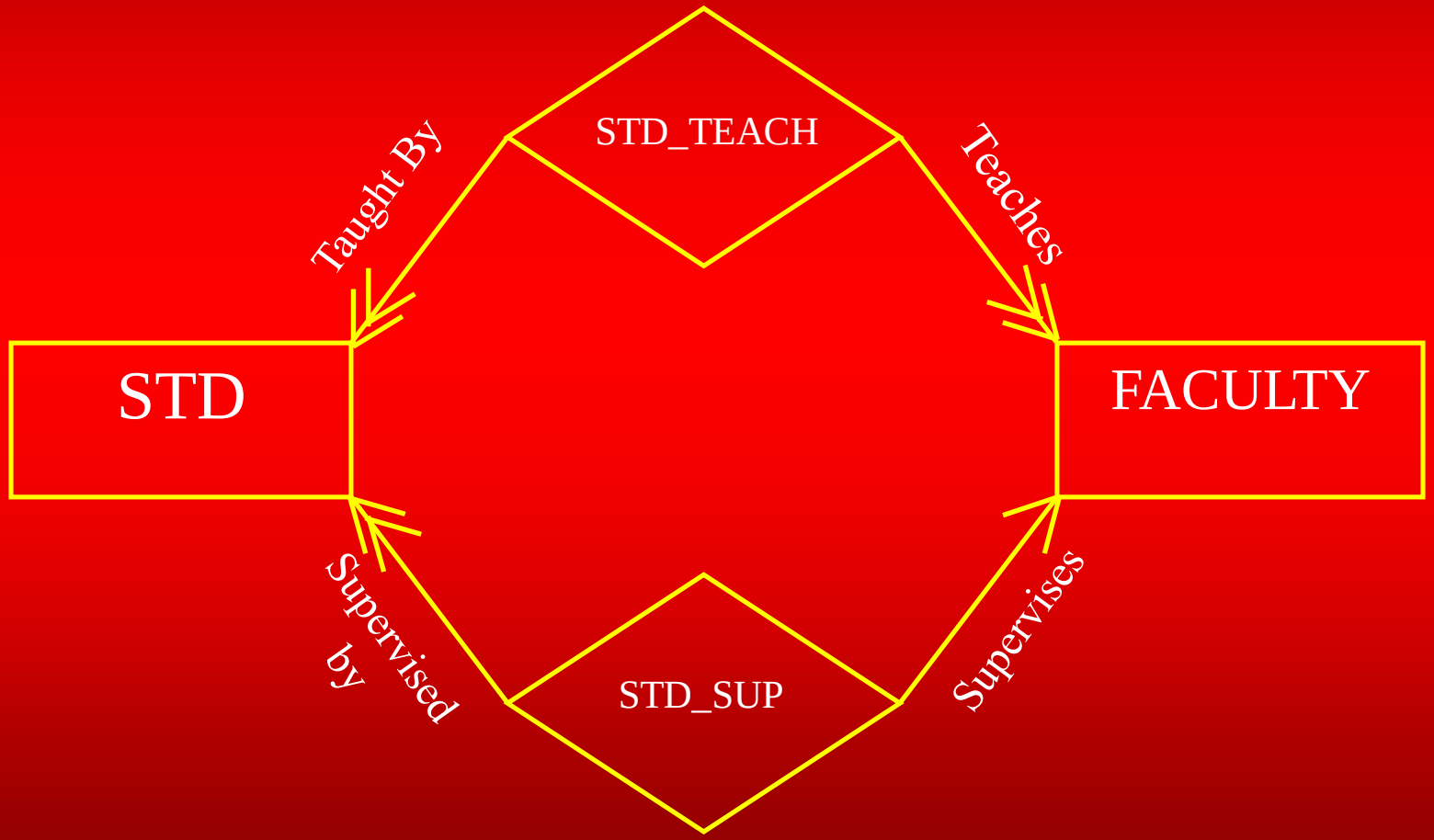
Roles in Relationships

- Two situation when they should be expressed explicitly
- A one to one relationship
- Two ETs having more than one relationship

Roles Examples



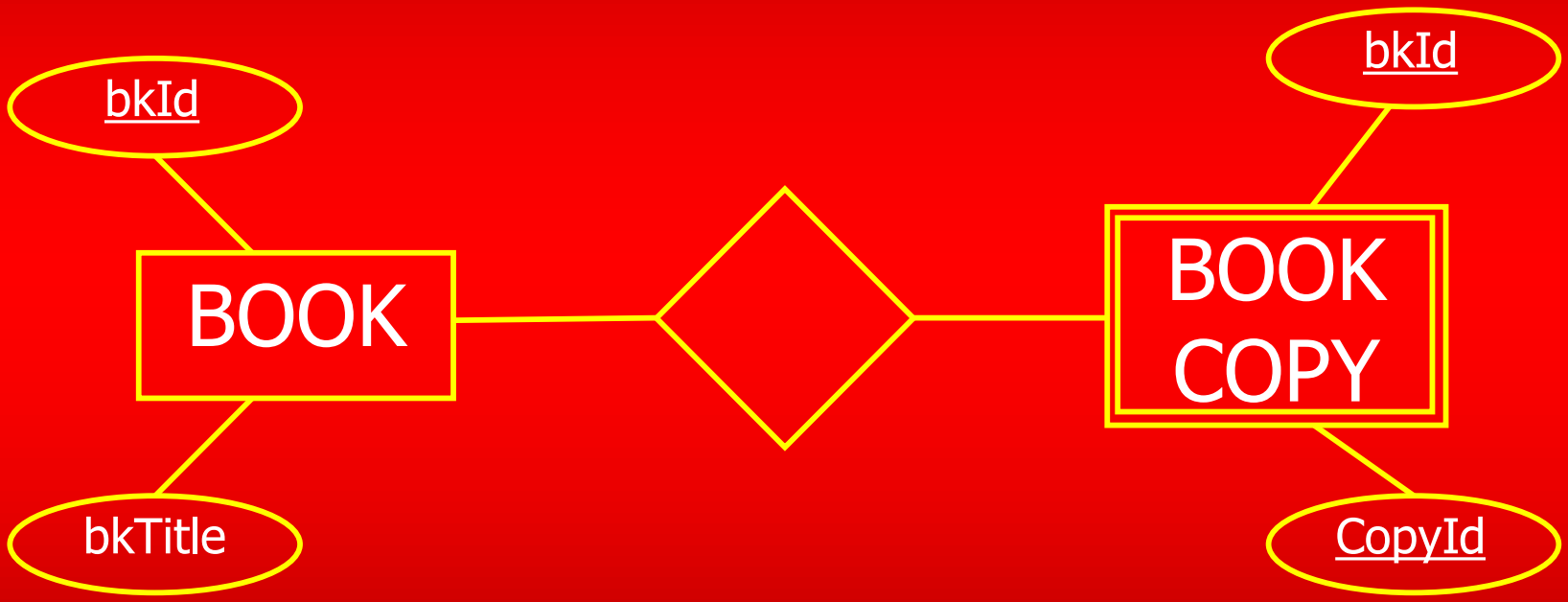
Roles Examples



Dependencies

- A type of constraint
- Existence dependency
- Identifier dependency
- Referential Dependency

Existence Dependency



Enhancements in E-R Data Model

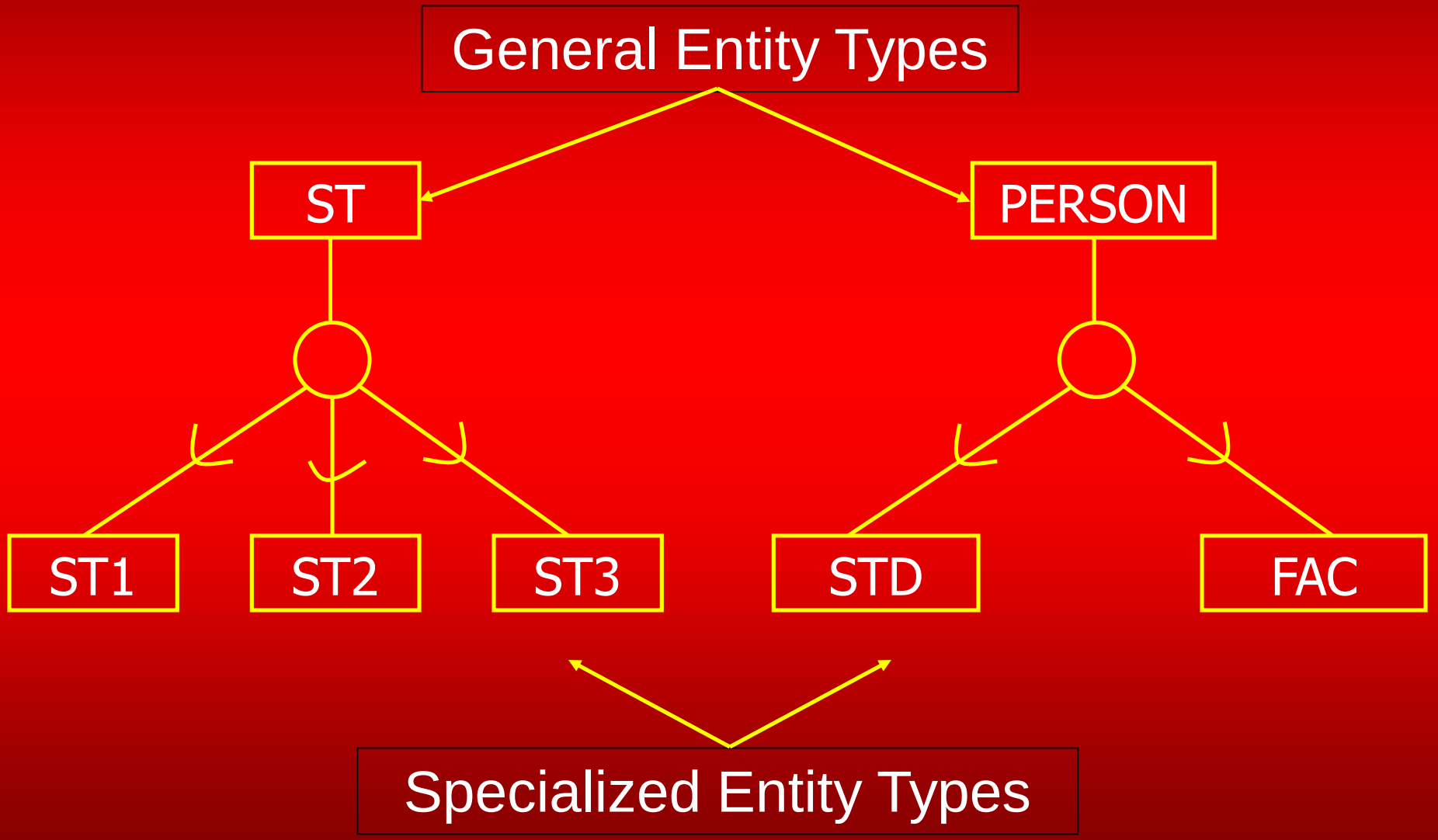
EE-R Data Model

- Different proposals
- Most common feature is representation of supertypes and subtypes
- A popular feature of Object Oriented paradigm

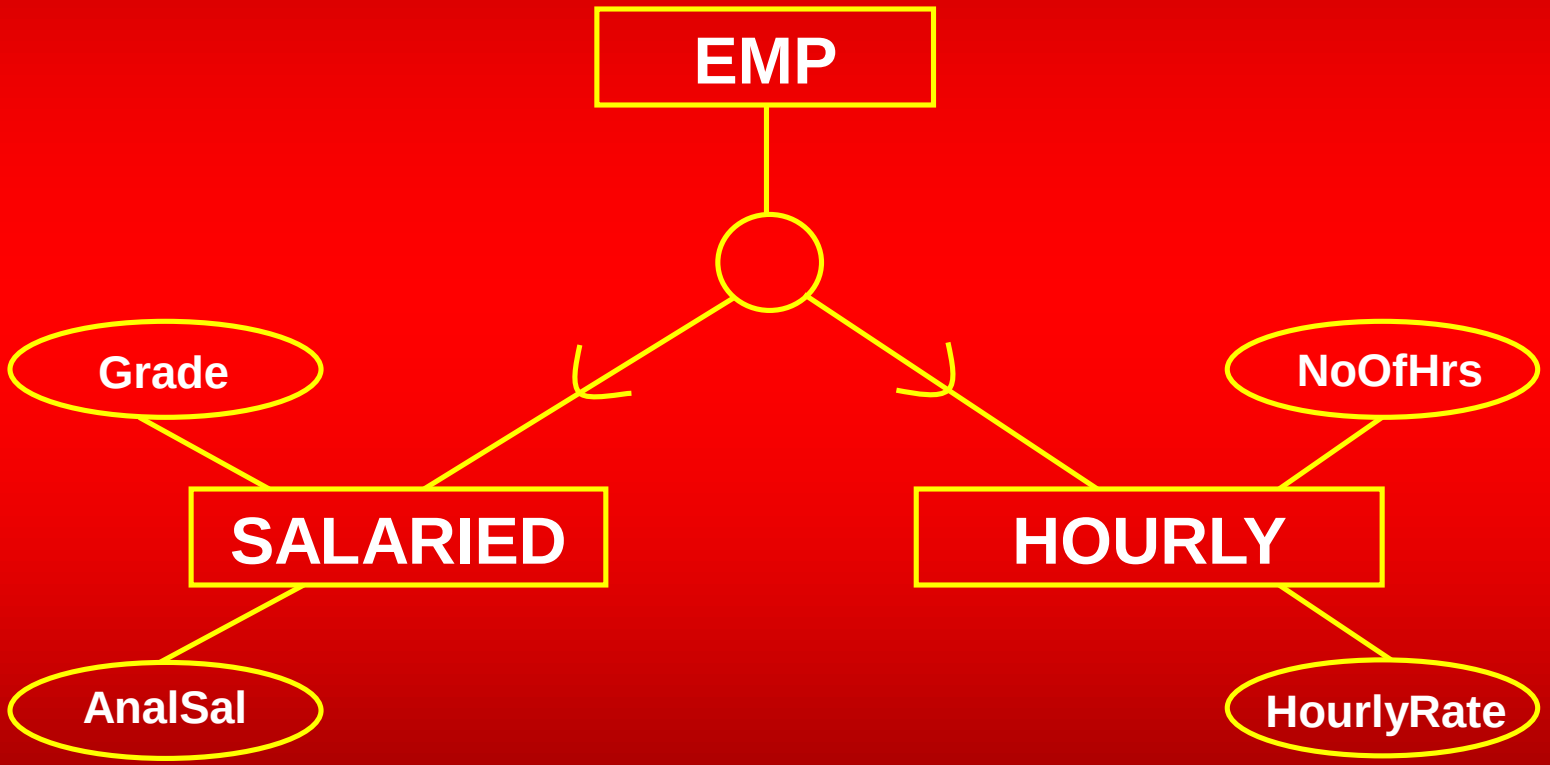
Super/Subtypes

- Also called generalization/ specialization
- Supertype is called a General Entity type whereas subtypes are the specializations

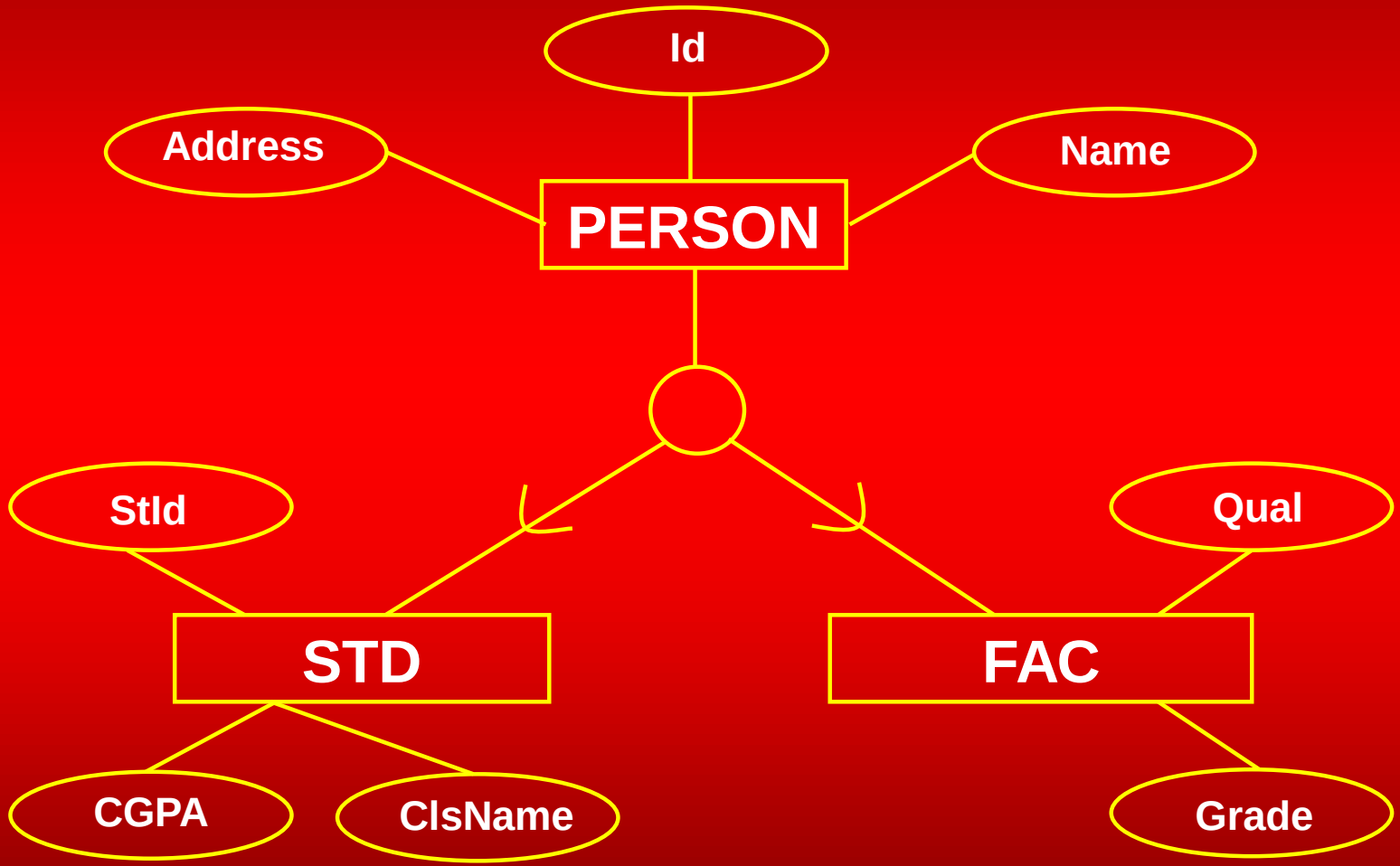
Super/Subtypes



Super/Subtypes



Super/Subtypes



Database Management System

Lecture - 10

Database Management System

Lecture - 11

Inheritance

- Gen/Spec relationship results inheritance between supertype and subtypes
- Subtypes inherit or get all the attributes of supertype

Super-Sub type Example

Super/Subtype Relationship

- Use/Advantage
- How to identify
  - General knowledge
  - Based on the attributes

Specifying Constraints

- Completeness constraint
  - Total specialization rule
  - Partial specialization
- Disjointness constraint
  - Disjoint rule
  - Overlap rule

Completeness Example 1

Completeness Example 2

Disjointness Example 1

Disjointness Example 2

Subtype Discriminator

- To determine the subtype of a supertype instance
- Place an attribute in the supertype whose value determines the subtype type

Disjoint Discriminator

Overlap Discriminator

Summary of the E-R Data Model

- A semantic data model
- Used for the conceptual database design
- Provides three main structures
- Entities, Relationships and attributes

- Entity reflects entity type, entity instance and entity set
- Entity types are classified into weak and regular entity types
- Regular ETs can exist independently and weak ETs can’t

Symbols

Regular Entity Type



Weak Entity Type



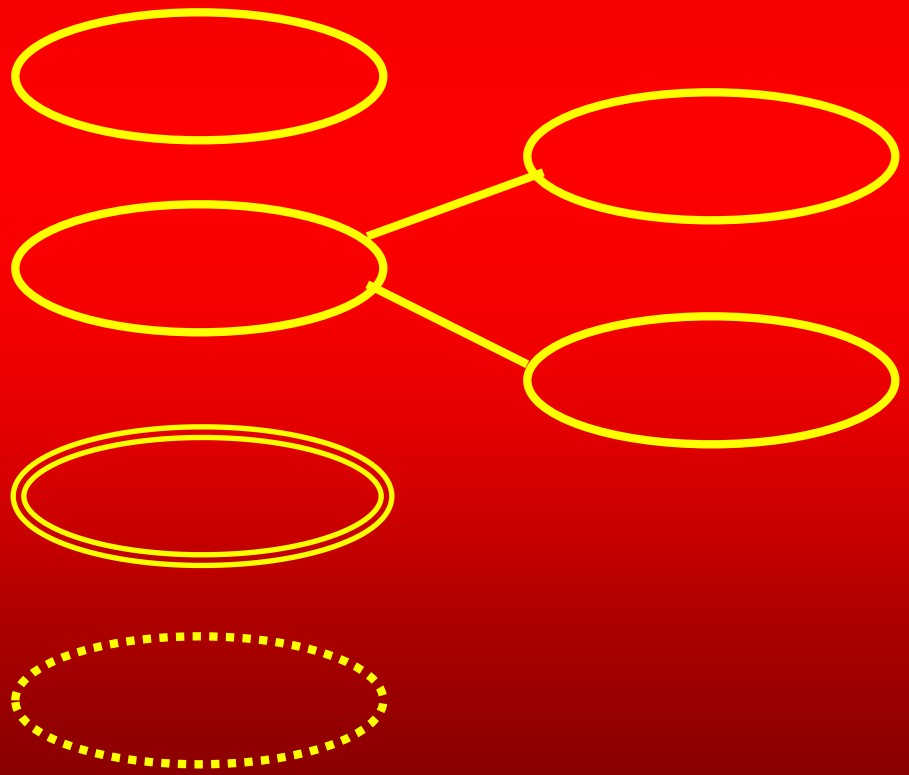
- Attribute represents a property or characteristic of an ET
- Attribute can be

  Simple

  Composite

  Multi-valued

  Derived



- Keys are used for data access
- Keys types are
  - Super key
  - Candidate key
  - Primary key
  - Alternate key
  - Secondary key

- Relationships are link or association between ETs
- Relationships can be unary, binary, ternary or n-ary
- Relationships are represented using diamonds linked with participant ETs

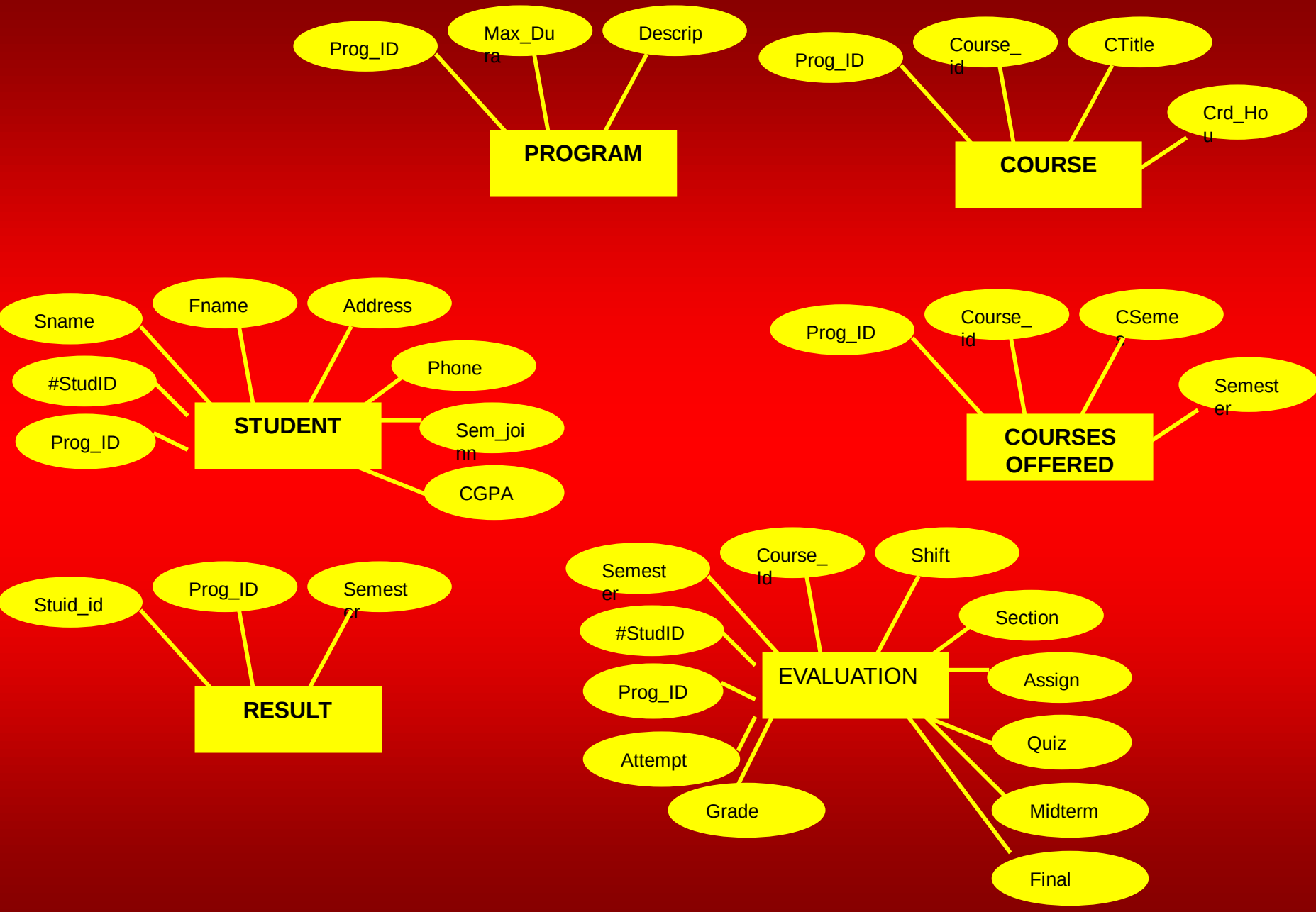
Unary, Binary and Ternary Rships

- Relationships have cardinalities
- Cardinalities can be
  - One to one
  - One to many
  - Many to one
  - Many to many

- Minimum and maximum cardinalities are also specified in the E-R data model
- Different notations are used in E-R data model to represent cardinalities

Cardinalities slide

- Enhanced E-R data model adds the additional features in the E-R data model
- Supertype/subtype relationship also called gen/spec includes inheritance


Prog_ID
Max_Du ra
Descrip
PROGRAM
Prog_ID
Course_ id
CTitle
Crd_Ho u
COURSE
Sname
Fname
Address
#StudID
Phone
Prog_ID
STUDENT
Sem_joi nn
CGPA
Prog_ID
Course_ id
CSeme r
Semest er
COURSES OFFERED
Stuid_id
Prog_ID
Semest er
RESULT
Semest er
Course_ Id
Shift
#StudID
Section
Prog_ID
EVALUATION
Assign
Attempt
Quiz
Grade
Midterm
Final

Database Management System

Lecture - 11

Database Management System

Lecture - 12

Practice Session

- Application of E-R data model
- We study an example scenario and apply the design phases we have studied so far
- Data flow diagram and E-R database design

Practice Session

- Lets study Examination System of an Educational Institution
- During initial study phase, we identified following major points

Preliminary Study

1. Students are enrolled in programmes/degrees
2. Programmes comprise of different courses
3. Different programmes are offered at the start of semester

Preliminary Study

4. Students register courses of different programmes at the start of every semester
5. After valid registration students attend the classes
6. Teachers submit results during and at the end of semester

Preliminary Study

7. Subjects' grade points are calculated on the basis of results submitted
8. Semester GPA is calculated on the basis of subjects' GPs

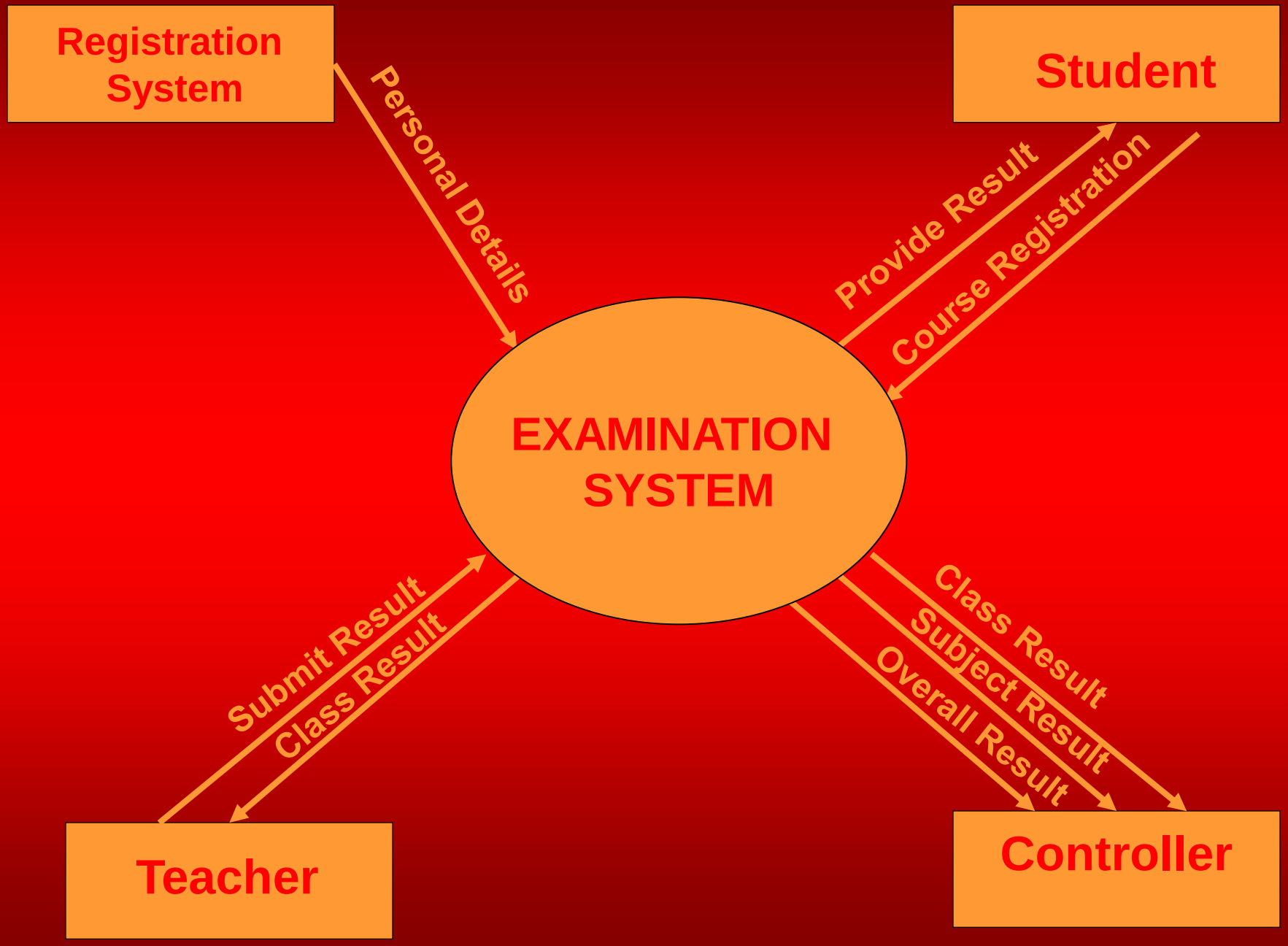
Preliminary Study

8. Cumulative GPA is calculated on the basis of semesters' GPAs
9. Courses, results, students data is stored in files

Outputs Required

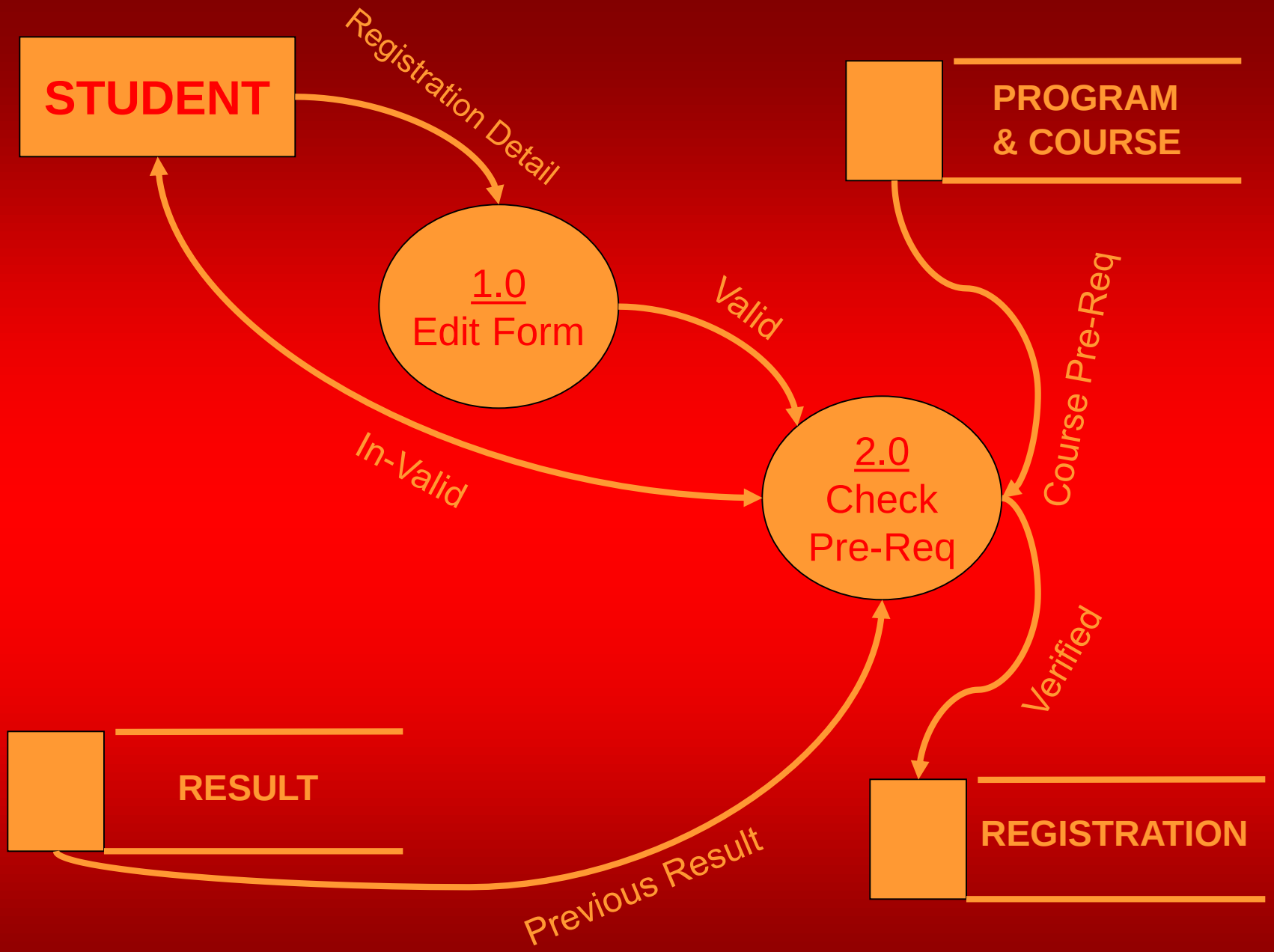
- Teachers and controller need class list or attendance sheet, class result; subject and overall
- Students need transcripts, semester result card, subject result

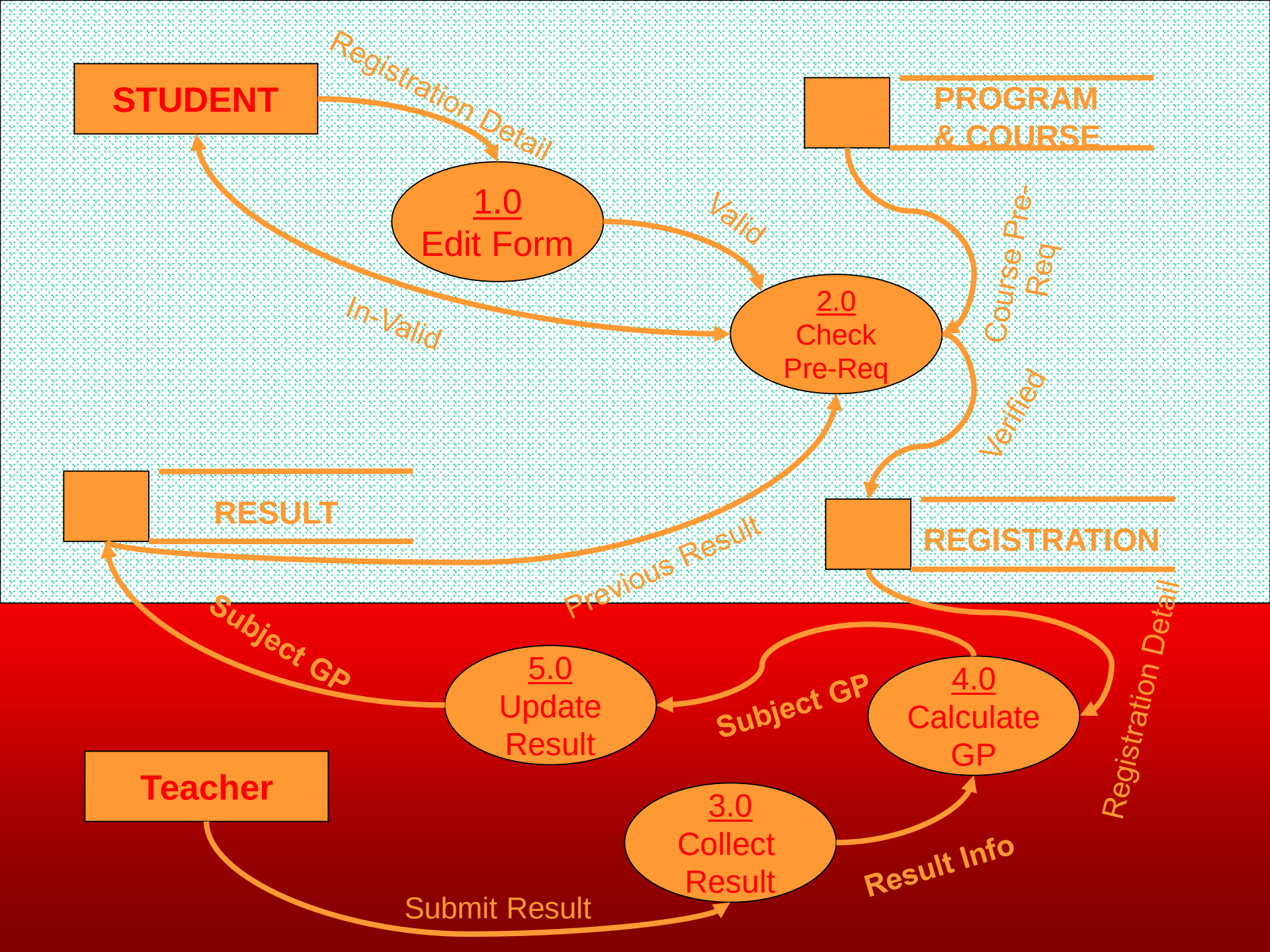
Data Flow Diagram

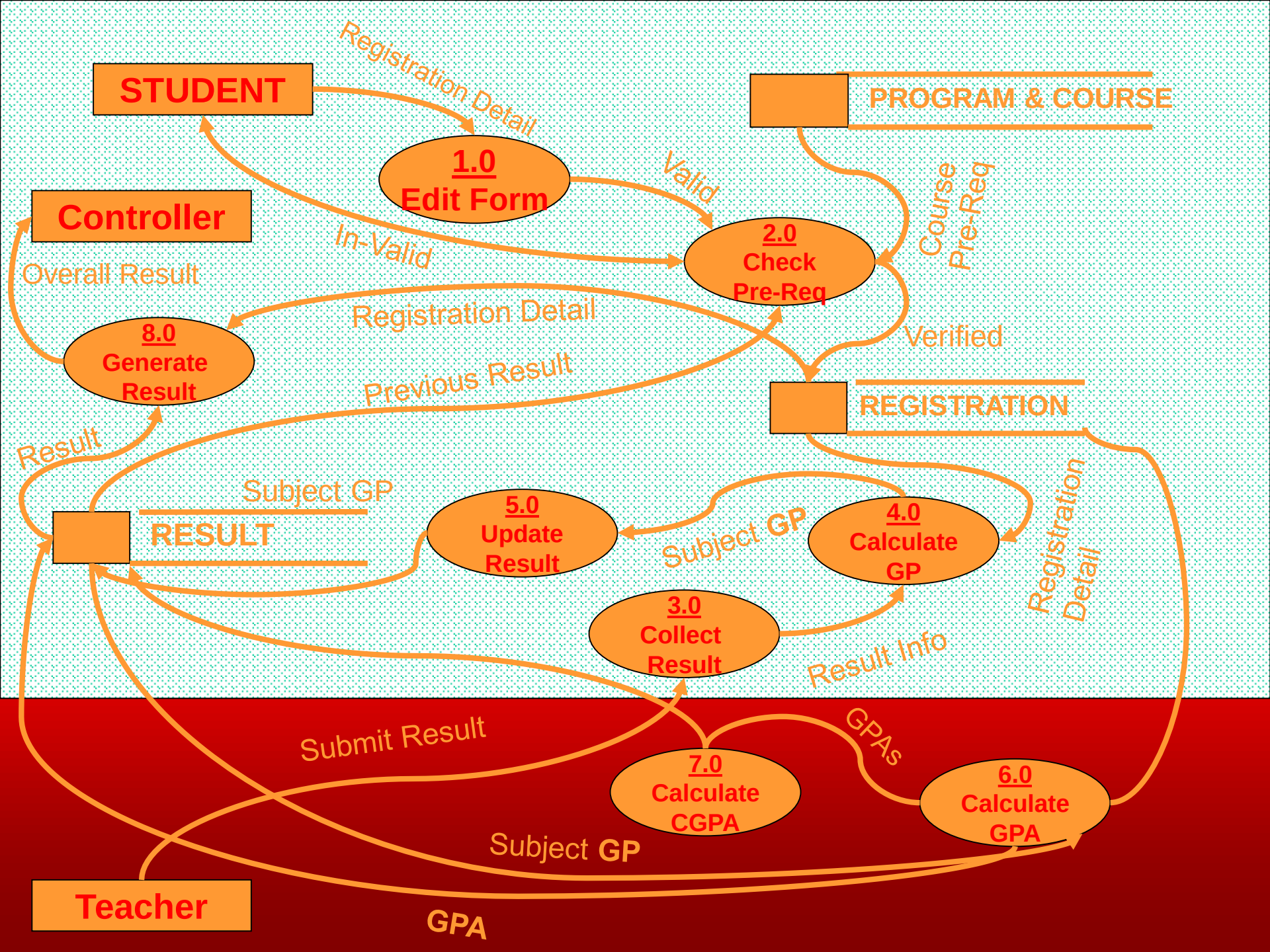

Registration System
Personal Details
Student
Provide Result
Course Registration
EXAMINATION SYSTEM
Submit Result
Class Result
Class Result
Subject Result
Overall Result
Teacher
Controller

MAIN MODULES

1. Subject registration
2. Result submission
3. Result calculation


STUDENT
Registration Detail
PROGRAM
& COURSE
1.0
Edit Form
Valid
In-Valid
2.0
Check
Pre-Req
Course Pre-Req
Verified
RESULT
REGISTRATION
Previous Result


STUDENT
Registration Detail
1.0
Edit Form
Valid
In-Valid
PROGRAM
& COURSE
Course Pre-
Req
2.0
Check
Pre-Req
Verified
RESULT
REGISTRATION
Previous Result
Subject GP
5.0
Update
Result
Subject GP
4.0
Calculate
GP
Registration Detail
Teacher
3.0
Collect
Result
Result Info
Submit Result


STUDENT
Registration Detail
1.0
Edit Form
Valid
PROGRAM & COURSE
Controller
In-Valid
2.0
Check
Pre-Req
Course
Pre-Req
Overall Result
Registration Detail
8.0
Generate
Result
Verified
Previous Result
REGISTRATION
Result
Subject GP
RESULT
5.0
Update
Result
Subject GP
4.0
Calculate
GP
Registration
Detail
3.0
Collect
Result
Result Info
Submit Result
GPAs
7.0
Calculate
CGPA
6.0
Calculate
GPA
Subject GP
Teacher
GPA

Cross Reference Matrix

- Reports vs Items

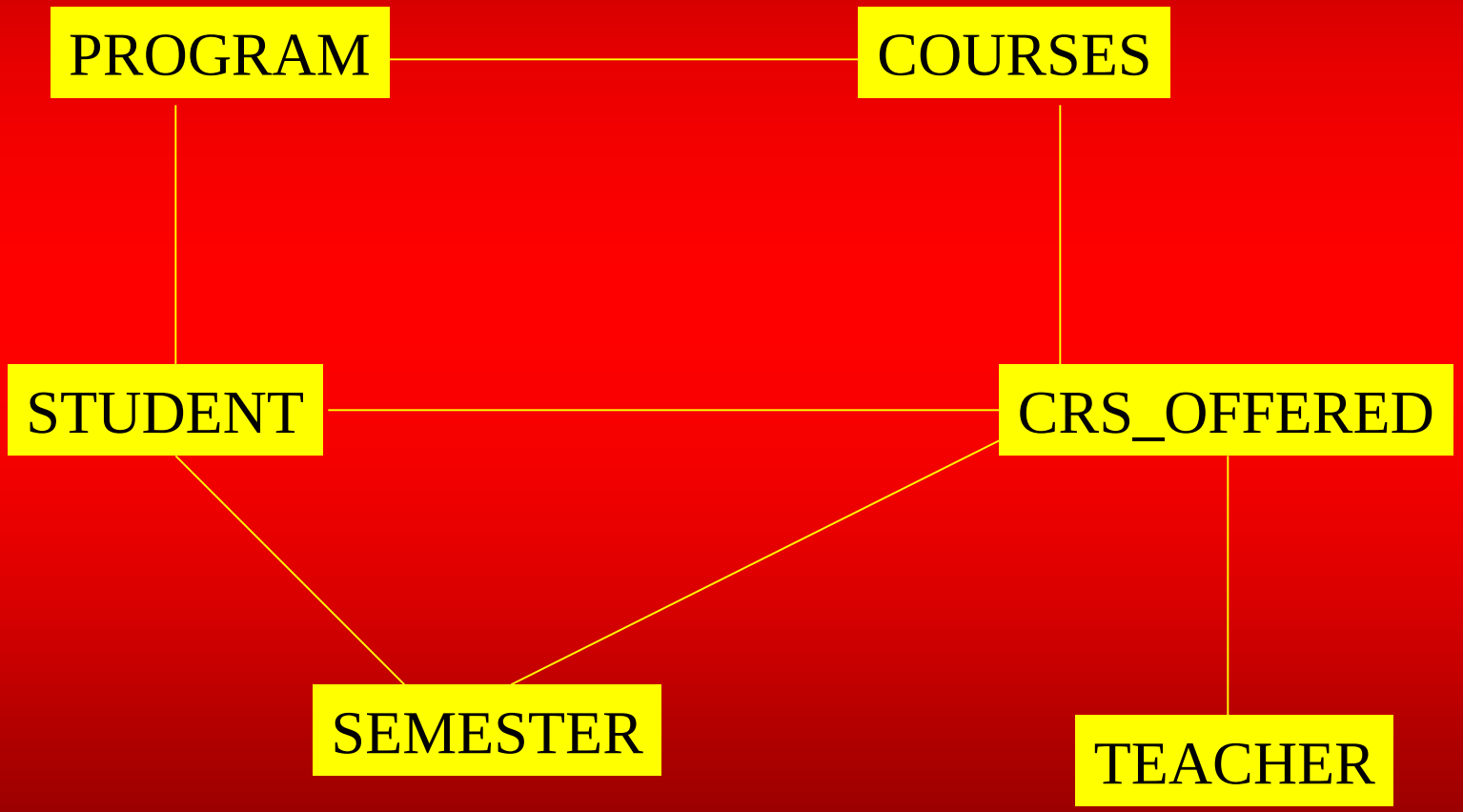
Analysis Phase

Candidate Entity Types

- From DFD
  - Student
  - Controller
  - Courses
  - Teachers
  - Courses Offered
  - Programmes
  - Registration
  - Results
  - Semester

Support your ETs by cross reference matrix

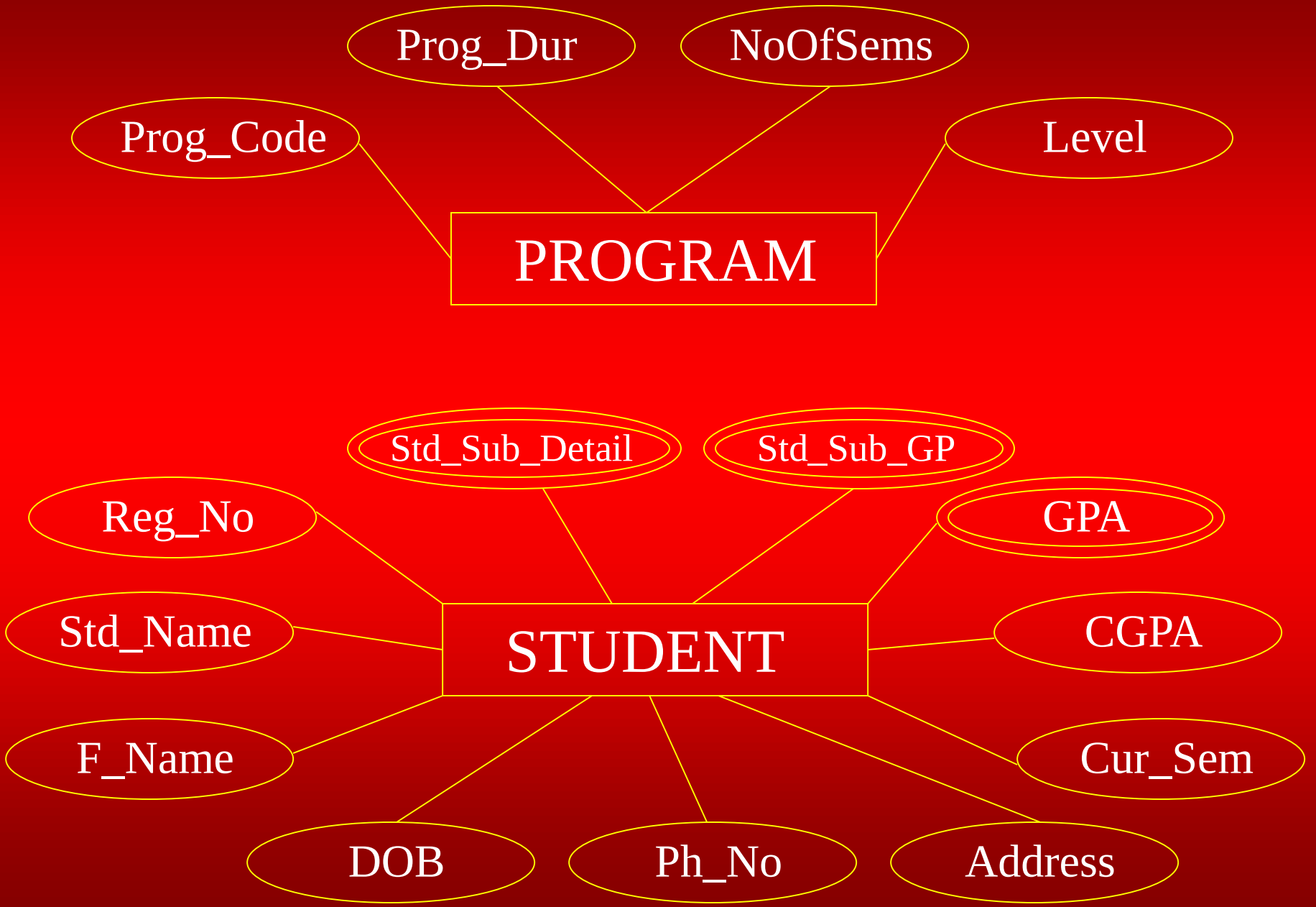
Candidate Entity Types

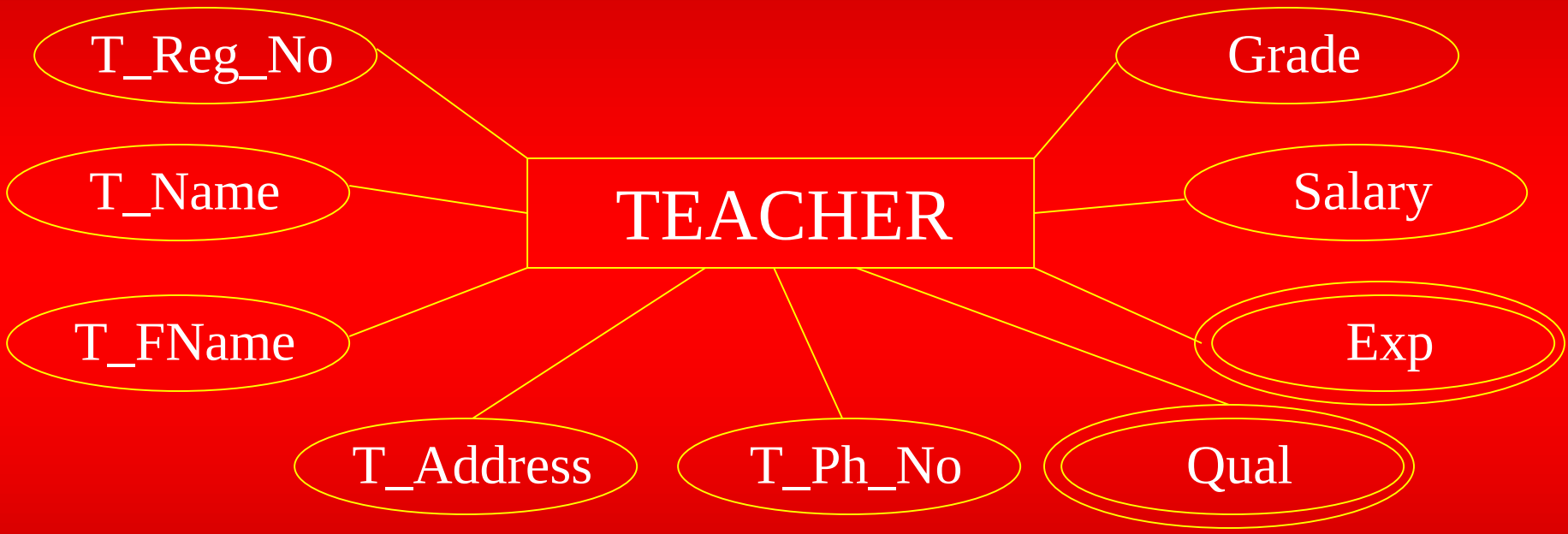


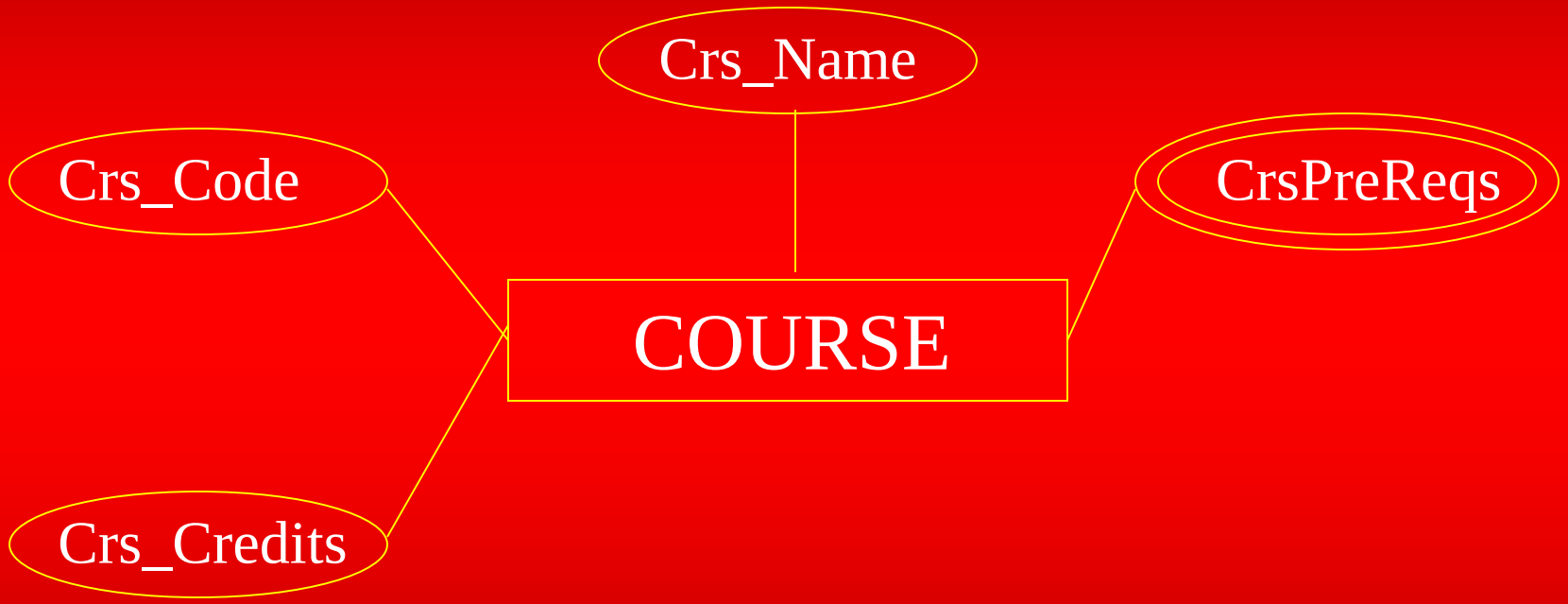
Database Management System

Lecture - 13

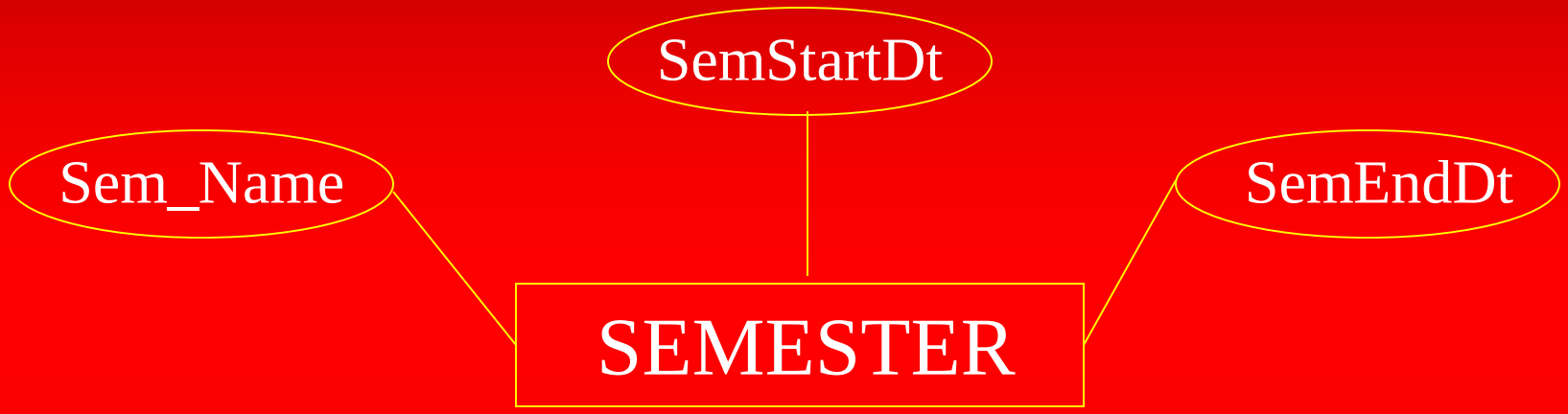
E-R Diagram

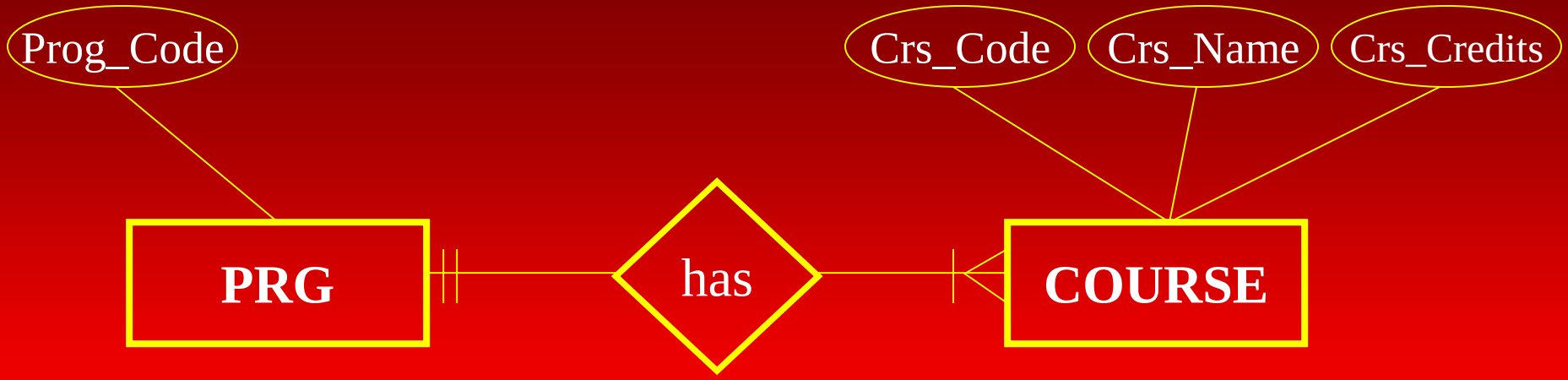

Prog_Dur
NoOfSems
Prog_Code
Level
PROGRAM
Std_Sub_Detail
Std_Sub_GP
Reg_No
GPA
Std_Name
STUDENT
CGPA
F_Name
Cur_Sem
DOB
Ph_No
Address

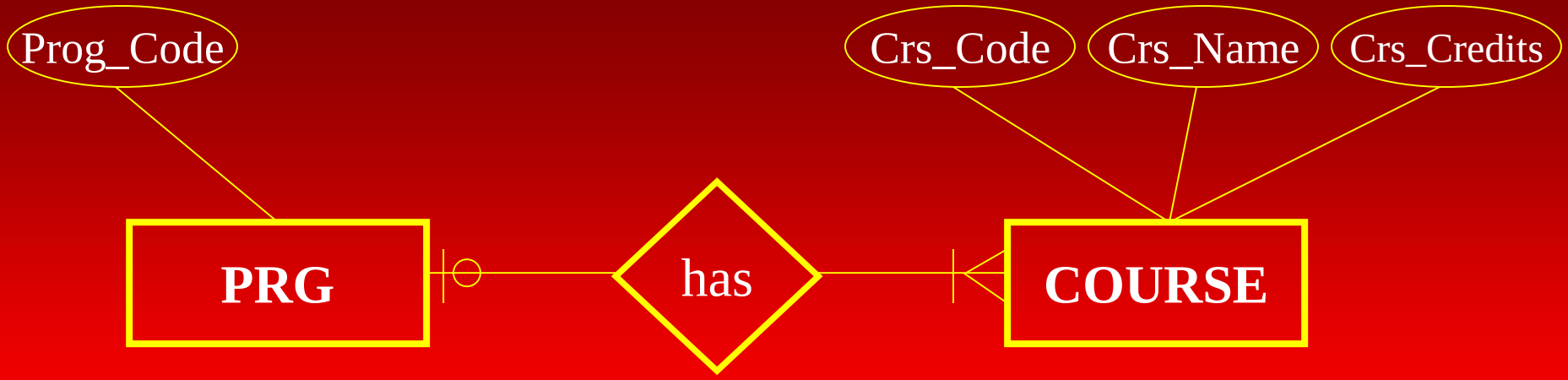

T_Reg_No
T_Name
T_FName
T_Address
T_Ph_No
Qual
Exp
Salary
Grade
TEACHER

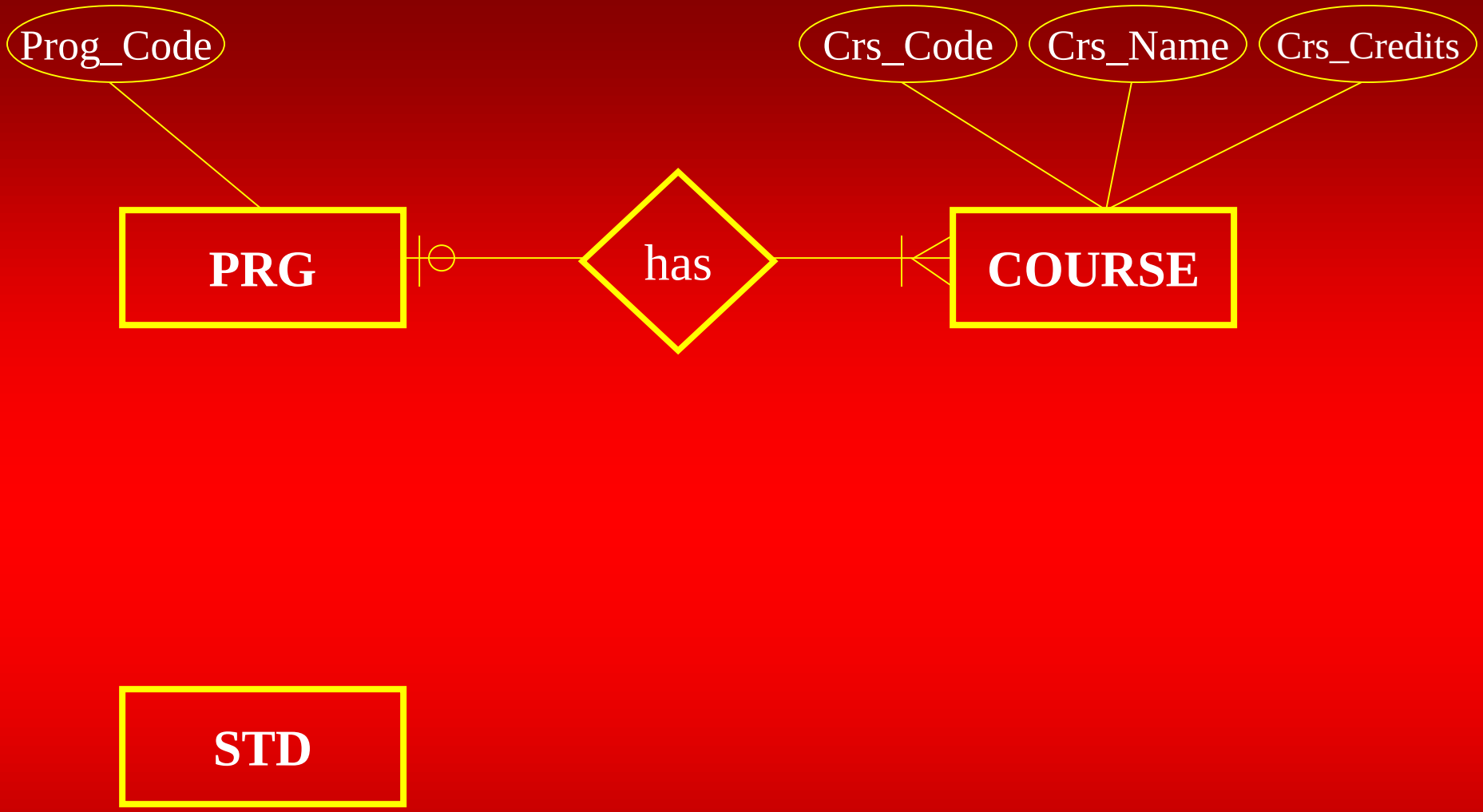

Crs_Name
Crs_Code
CrsPreReqs
COURSE
Crs_Credits

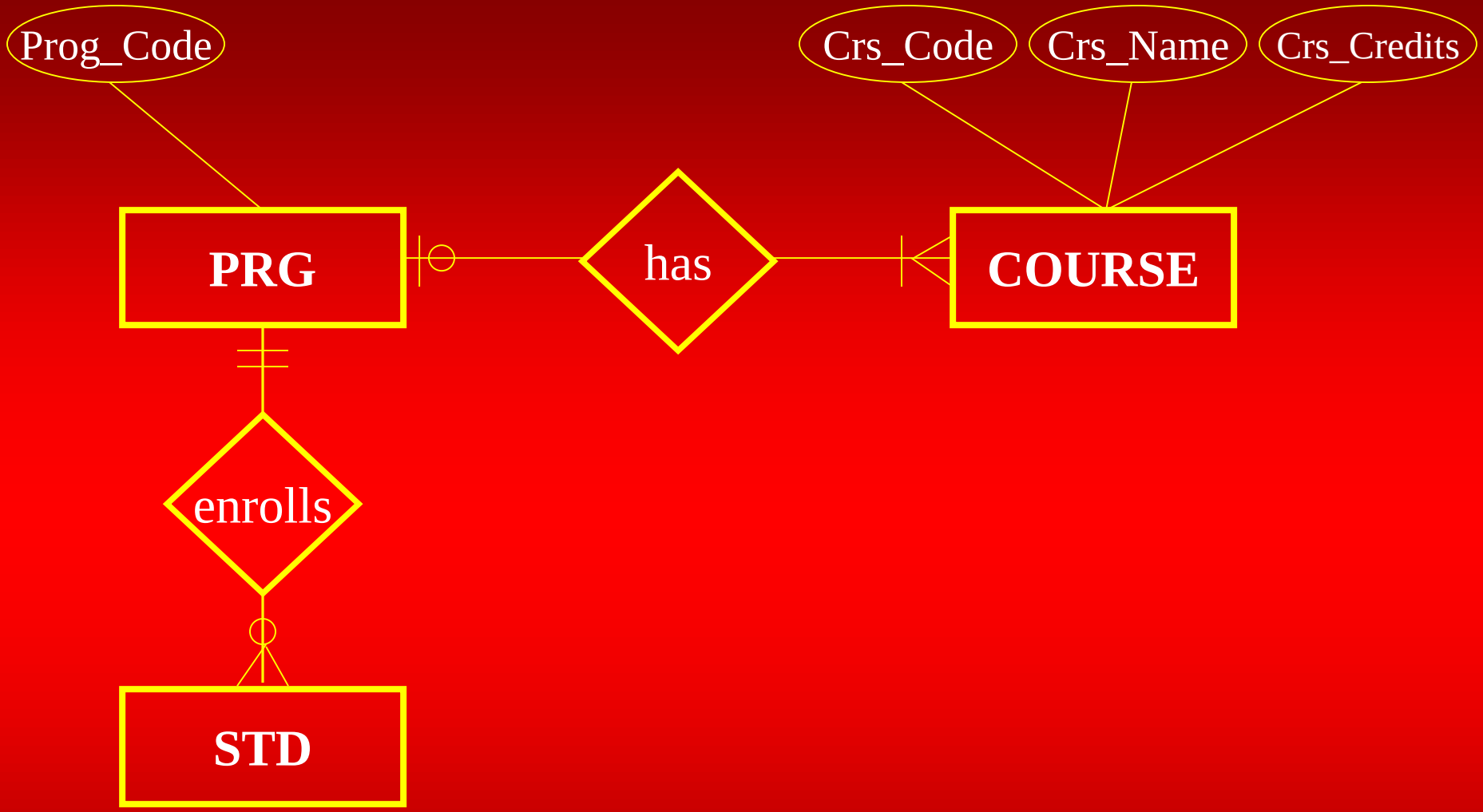
COURSE
OFFERED IN

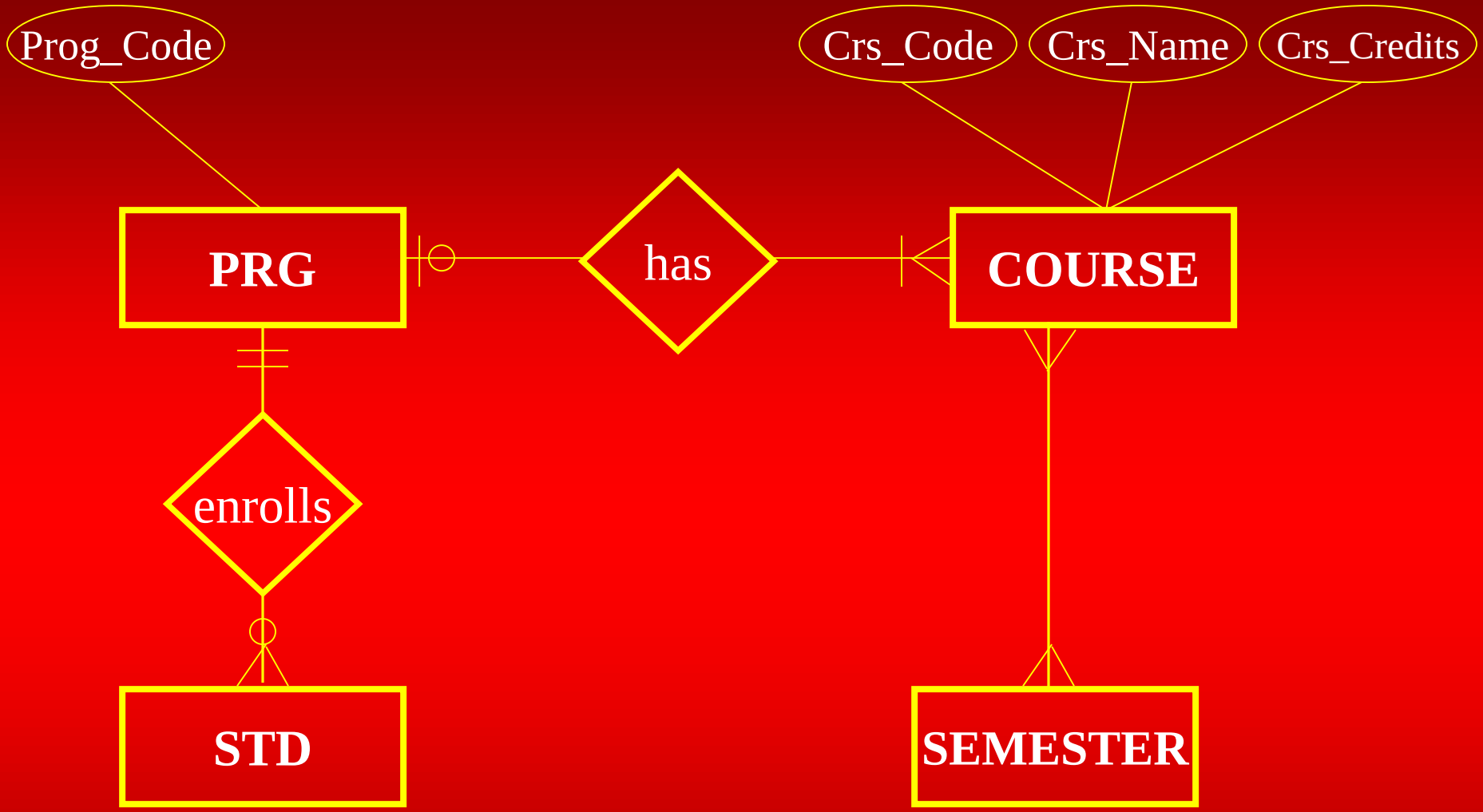

SemStartDt
Sem_Name
SemEndDt
SEMESTER

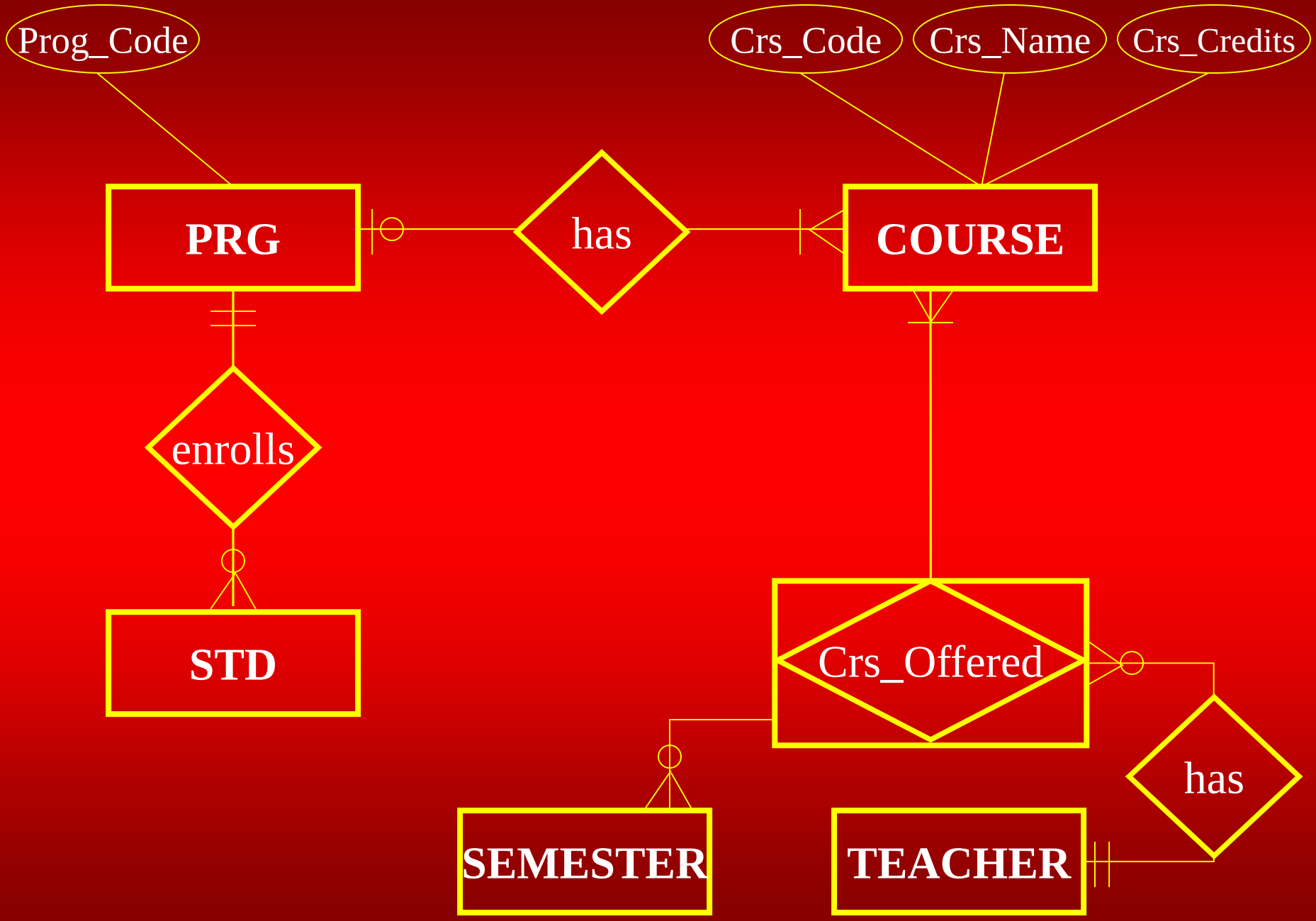

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE

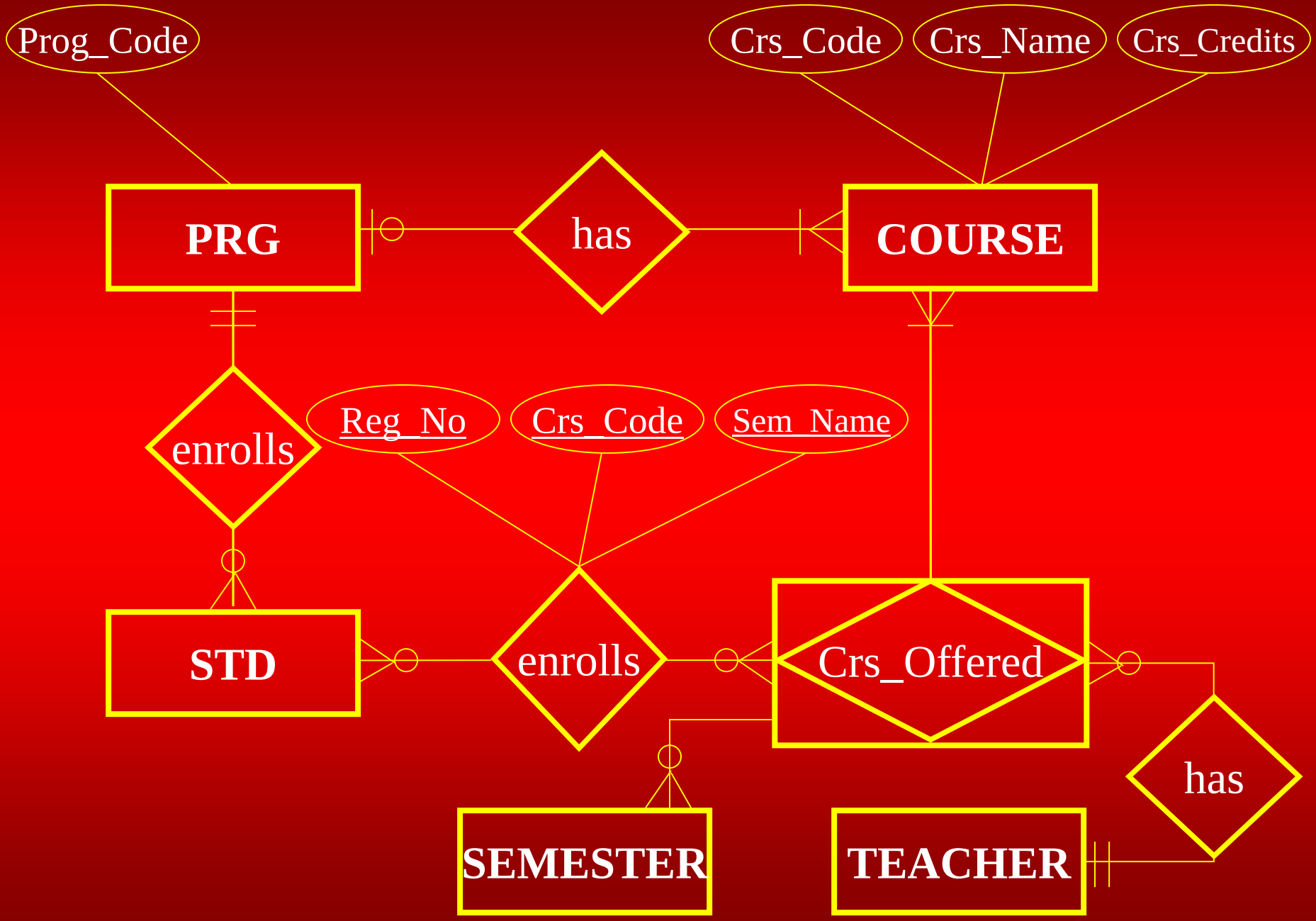

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE

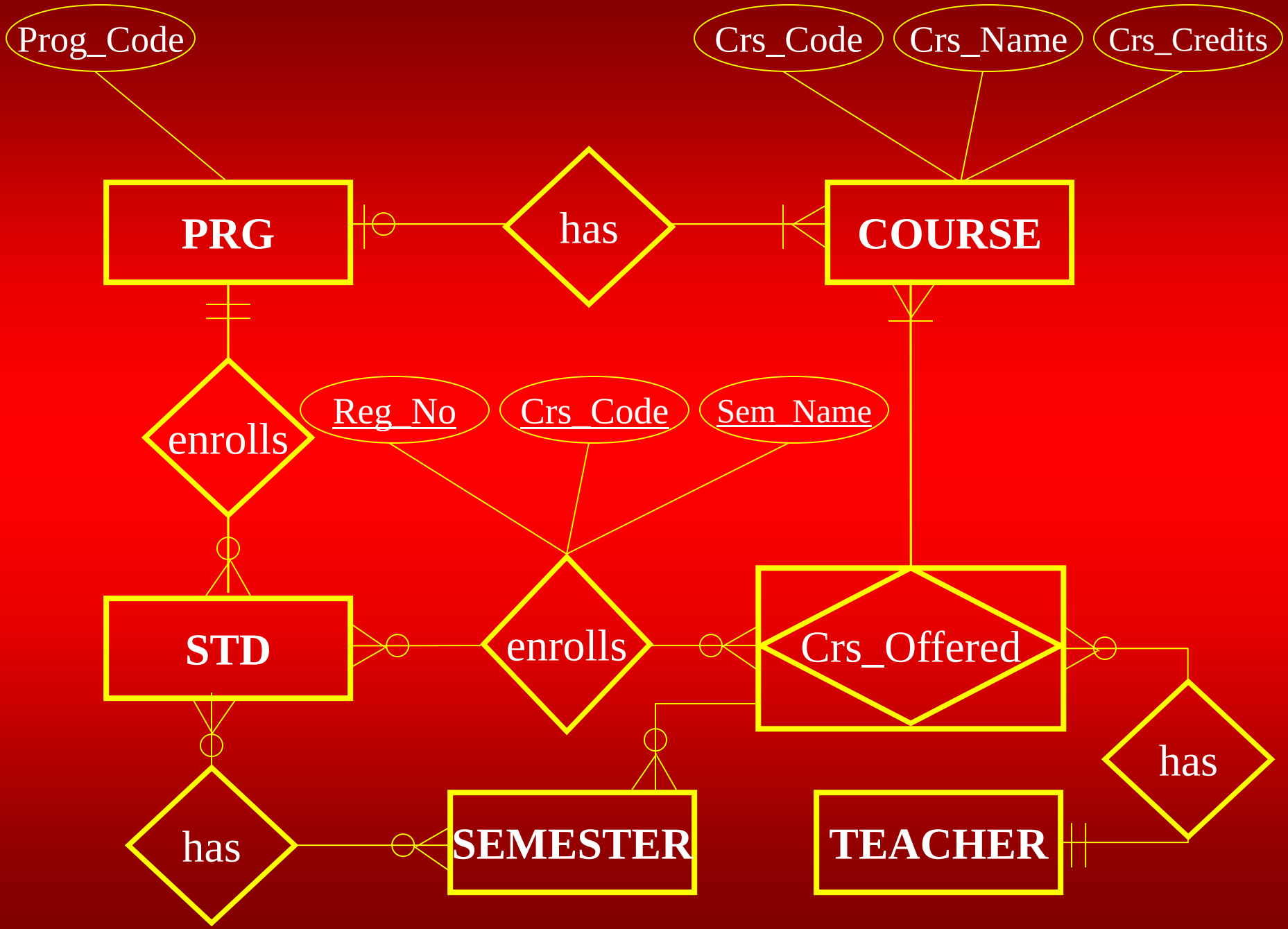

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
STD

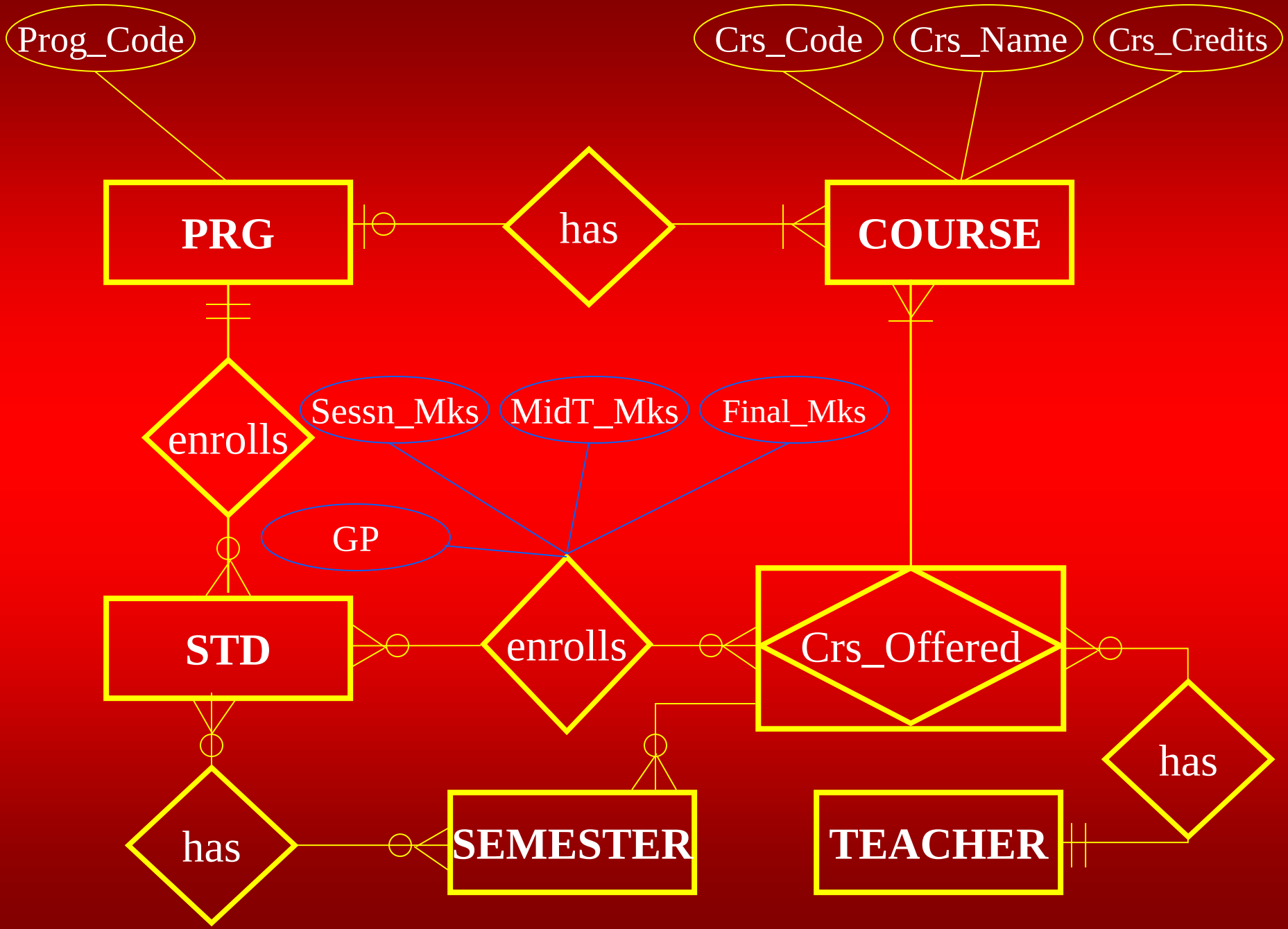

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
STD

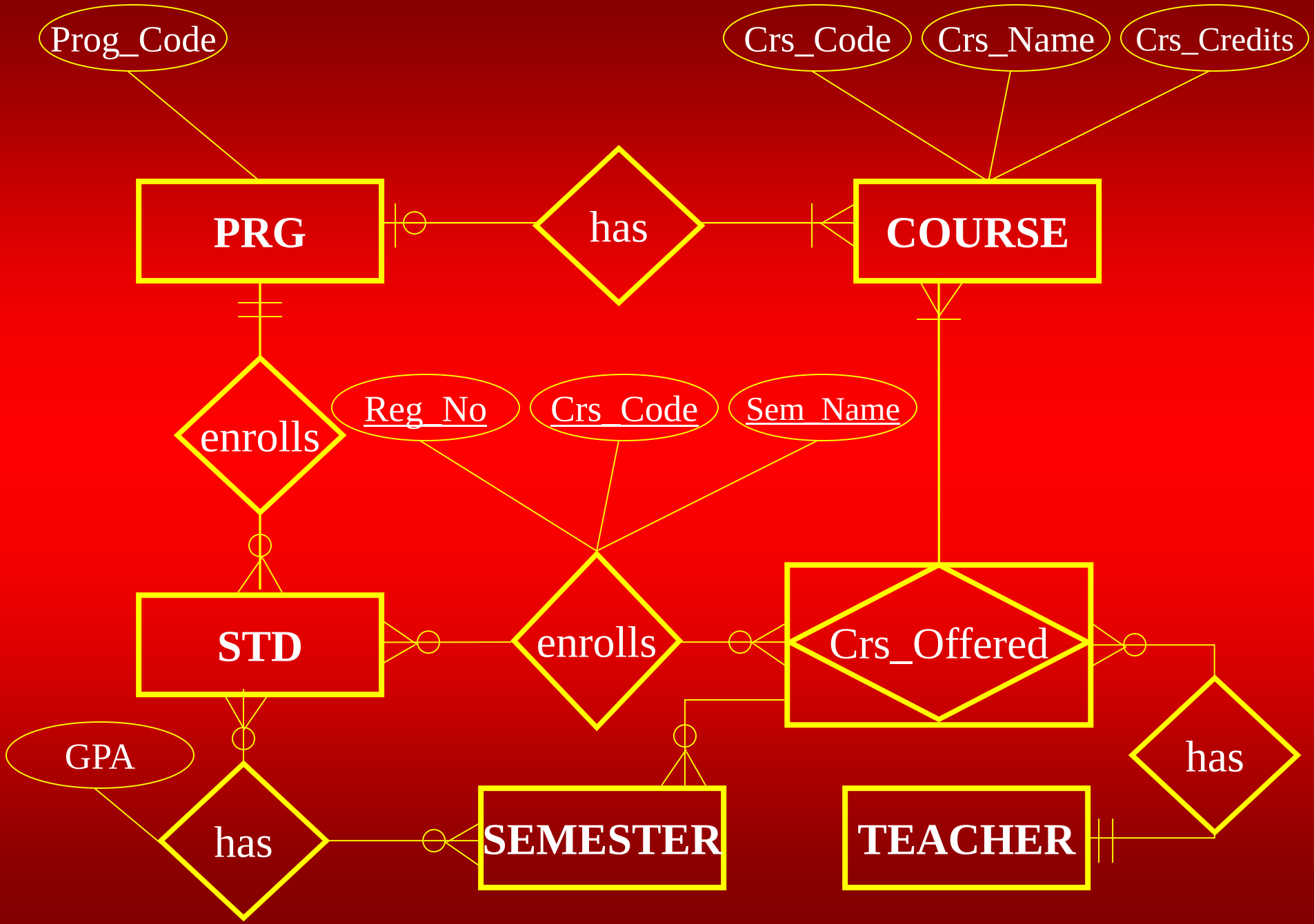

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
STD
SEMESTER

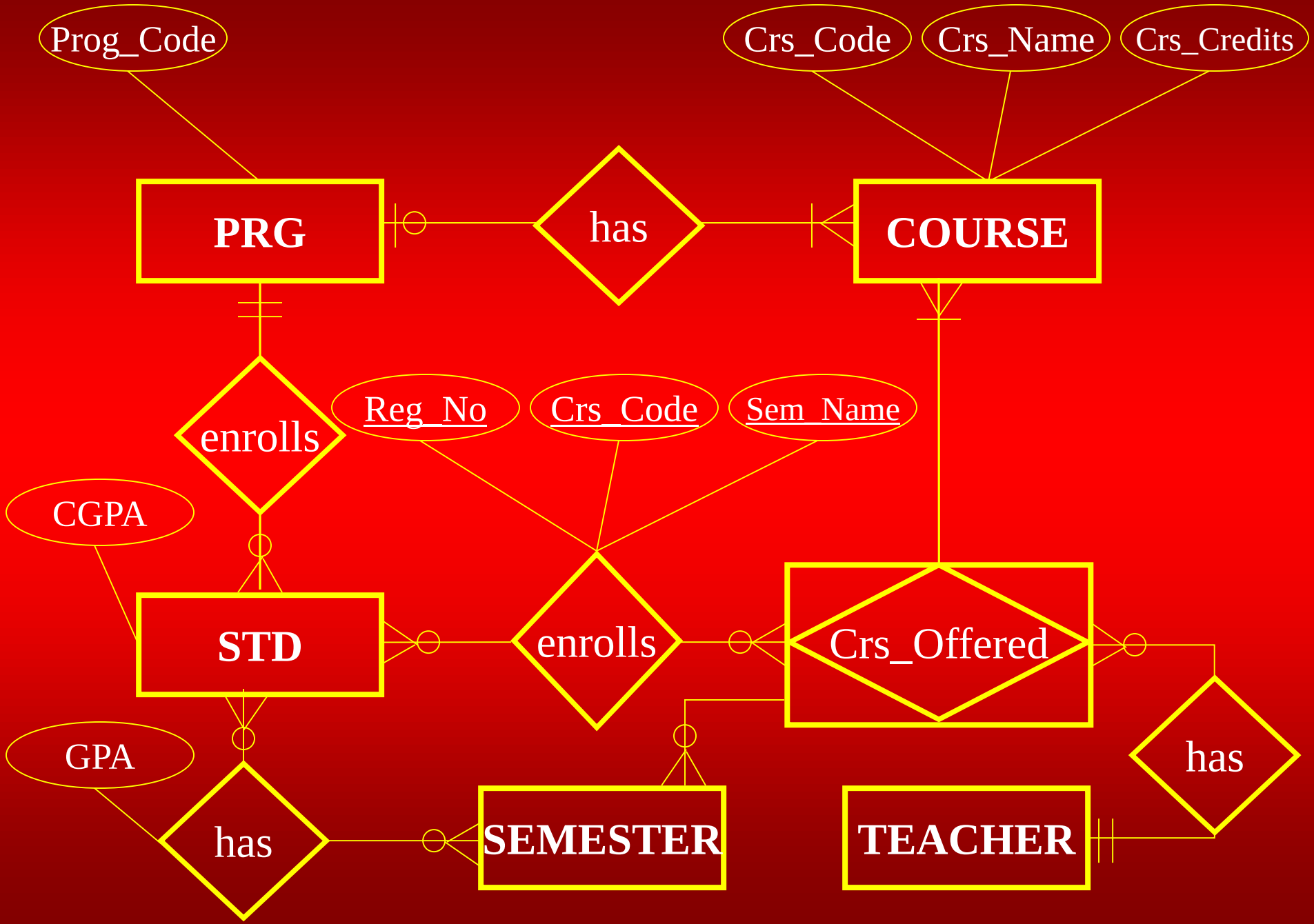

Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
STD
Crs_Offered
has
SEMESTER
TEACHER


Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
Reg_No
Crs_Code
Sem_Name
STD
enrolls
Crs_Offered
has
SEMESTER
TEACHER


Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
Reg_No
Crs_Code
Sem_Name
STD
enrolls
Crs_Offered
has
has
SEMESTER
TEACHER


Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
Sessn_Mks
MidT_Mks
Final_Mks
GP
STD
enrolls
Crs_Offered
has
has
SEMESTER
TEACHER


Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
Reg_No
Crs_Code
Sem_Name
STD
enrolls
Crs_Offered
GPA
has
has
SEMESTER
TEACHER


Prog_Code
Crs_Code
Crs_Name
Crs_Credits
PRG
has
COURSE
enrolls
Reg_No
Crs_Code
Sem_Name
CGPA
STD
enrolls
Crs_Offered
GPA
has
has
SEMESTER
TEACHER

Database Management System

Lecture - 14

Logical Database Design

Introduction to Relational Data Model

Two Major Strengths

- Simplicity
- Strong Mathematical Foundation

Relational Data Model

- Presented by E. F. Codd in 1970, then of IBM
- Before Relational Data Model, two older data models were in use; Hierarchical, Network

Relational Data Model

- First DBMS built on Relational Data Model (RDM) was system R
- Another Relational DBMS built during those days was INGRES

Basic of RDM

- RDM used mainly for external, conceptual, and to some extent physical schema
- Separation of conceptual and physical levels makes manipulation much easier, contrary to previous data models

Basic of RDM

- RDM used for external, conceptual, and to some extent physical schema
- The basic structure is relation
- Both entities and relationships are modeled using tables/relations

Basics of RDM

- Relations physically represented as tables
- Table is a two dimensional representation of a relation
- Consists of rows and columns

Basics of RDM

- Columns represent attributes and rows represent records
- Rows, records and tuples all these terms are used interchangeably

Basic Properties of a Table

1. Each cell of a table contains atomic/single value
2. Each column has a distinct name; the name of the attribute it represents

Basic Properties of a Table

3. The values of the attributes come from the same domain
4. The order of the columns is immaterial

Basic Properties of a Table

5. The order of the rows is immaterial
6. Each row/tuple/record is distinct, no two rows can be same

A table

stID	stName	clName	doB	sex
S001	M. Suhail	MCS	12/6/84	M
S002	M. Shahid	BCS	3/9/86	M
S003	Naila S.	MCS	7/8/85	F
S004	Rubab A.	MBA	23/4/86	F
S005	Ehsan M.	BBA	22/7/88	M

Components of RDM

- As discussed in data model definition

1. Structure (relation/table)
2. Manipulation language (SQL)
3. Integrity constraints (Two)

Mathematical Relations

- Consider two sets
- A = {x, y} B = {2, 4, 6}
- Cartesian product of these sets
- A X B= {(x,2), (x,4), (x,6), (y,2), (y,4), (y,6)}

Mathematical Relations

- A relation is some subset of this Cartesian product, For example,
- R1= {(x,2), (y,2),(x,6),(x,4)}
- R2 = {(x,4), (y,6), (y,4)}

Mathematical Relations

The same notion of Cartesian product and relations can be applied to more than two sets, e.g. in case of three sets, we will have a relation of ordered triplets

Database Relations

- Thinking in some real world scenario
  - Name = {Ali, Sana, Ahmed, Sara}
  - Age = {15,16,17,18,……..,25}

Database Relations

Cartesian product of Name & Age

Name X Age= {(Ali,15), (Sana,15), (Ahmed,15), (Sara,15), …., (Ahmed,25), (Sara,25)}

CLASS = {(Ali, 18), (Sana, 17), (Ali, 20), (Ahmed, 19)}

Database Relations

- Let A1, A2, A3, …, A_n be some attributes and D1, D2, D3,…, D_n be their domains
- A relation scheme relates certain attributes with their domain in context of a relation

Relation Scheme

Can be represented as

R = (A1:D1, A2:D2, ……, An:Dn)

STD = (stId:Text, stName: text, stAdres:Text, doB:Date) OR

STD(stId, stName, stAdres, doB)

Database Relations

According to this scheme we can have a relation (instance of this scheme), like

STD={(stId:S001, stName:Ali, stAdres: Lahore, doB:12/12/76), (stId:S003, stName:A. Rehman, stAdres: RWP, doB:2/12/77)}

Database Relations

STD={(S001, Ali, Lahore, 12/12/76),
(S003, A. Rehman, RWP, 2/12/77)}

stId	stName	stAdres	doB
S001	Ali	Lahore	12/12/76
S002	A. Rehman	RWP	2/12/77

Database Management System

Lecture - 15

DB and Math Relations

- Properties of DB relations are similar to those of Mathematical relations, except
- The order of columns in Mathematical relation does matter

Degree and Cardinality

The number of rows in a relation is its cardinality and the number of columns is its degree

Relations Keys

- The concept of key and all different types of keys are applicable to the Relations
- Foreign Key: An attribute of a table B that is primary key in another table A

Foreign Key

Consider table EMP and DEPT

EMP (empId, empName, qual, depId)

DEPT (depId, depName, numEmp)

Integrity Constraints

- Two main types
- Entity integrity constraint
  - Primary key cannot have null value
- Referential integrity constraint
  - Value of Foreign key is either null or matches with a value in its home relation

Foreign Key Example

Significance of Constraints

- Constraints help to maintain the correctness, validity or integrity of the database
- Like null constrains, default value, domain constraint

RDM Components

- So far we have studied structure and integrity constraint component of the RDM
- Remaining; manipulation language will be discussed later

Designing Logical DB

- Logical DB design is obtained from conceptual DB design
- Generally involves transforming E-R data model to relational data model
- We have studied both DMs, now how to perform transformation

Transforming Rules

- Straightforward rules exist
- Can be performed manually as well as automatically
- Evaluate even if you use some tool, since multiple options exist

Mapping Entity Types

- Each regular entity type is transformed straightaway into a relation
- PK of the ET is declared as PK of relation
- Simple attributes of ET are included into the relation

Mapping Regular ET

Composite Attributes

- Since tables can contain only atomic values composite attributes need to be represented as a separate relation
- Quality becomes a limitation

Composite Attrib Example

Multi-valued Attributes

- An ET with a multi-valued attribute is transformed into two relations
- One contains the entity type and second the muli-valued attribute

Multi-valued Attribute

- The PK of the second relation is the PK of first relation and the attribute value itself
- Values are accessed through reference of the PK, that also serves as FK

MV Attrib Example

Mapping Weak ETs

- Identifier dependency: relation for weak entity type is created and the PK of the strong ET is used as a whole or part of the relation against weak ET

Identifier Dependency Example

Mapping Weak ETs

Referential Dependency: relation against weak ET has got its own PK, the link is established through FK, however, FK declared as not null

Ref. Dep. Example

Thanks

Database Management System

Lecture - 16

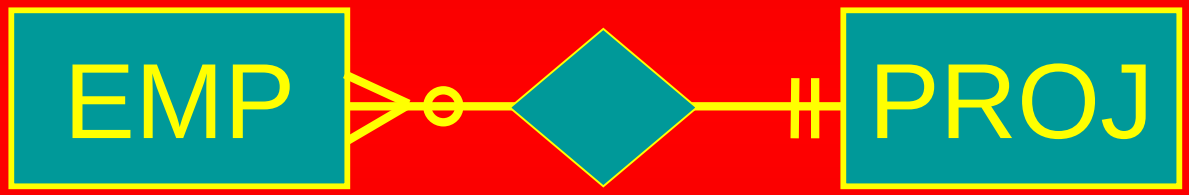
Mapping Relationships

- Relationships are mapped according to their degree and cardinalities
- Starting from binary relationships

Binary Relationships

- Cardinality: one to many
- The PK of the relation against many side ET contains the PK of the relation for ET on one side as FK

One to Many Example



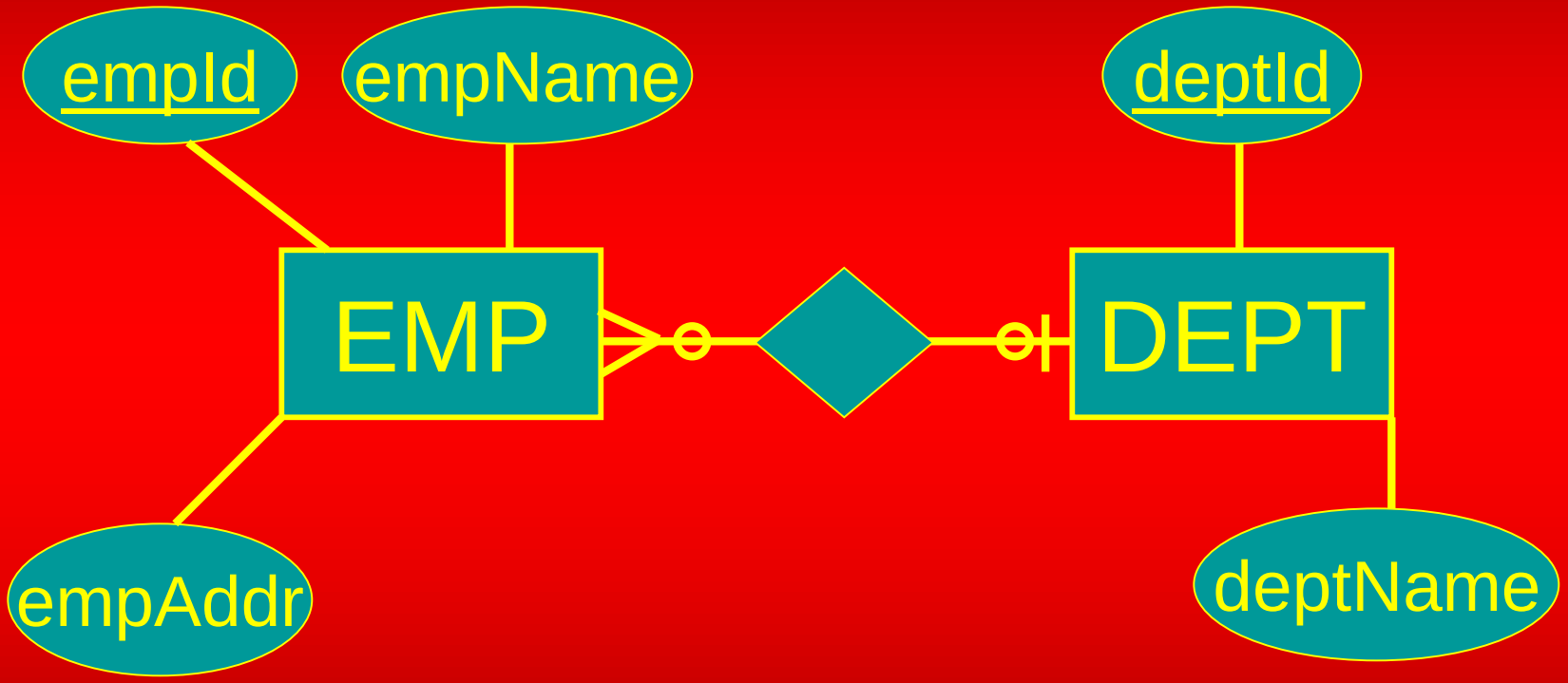
EMP(empId, empName, projId)

PROJ(projId, projName, projDur, projCost)

Minimum Cardinality

- Minimum cardinality on one side needs special attention
- If optional, define FK as normal
- If compulsory, make FK not null

Minimum Cardinality Example

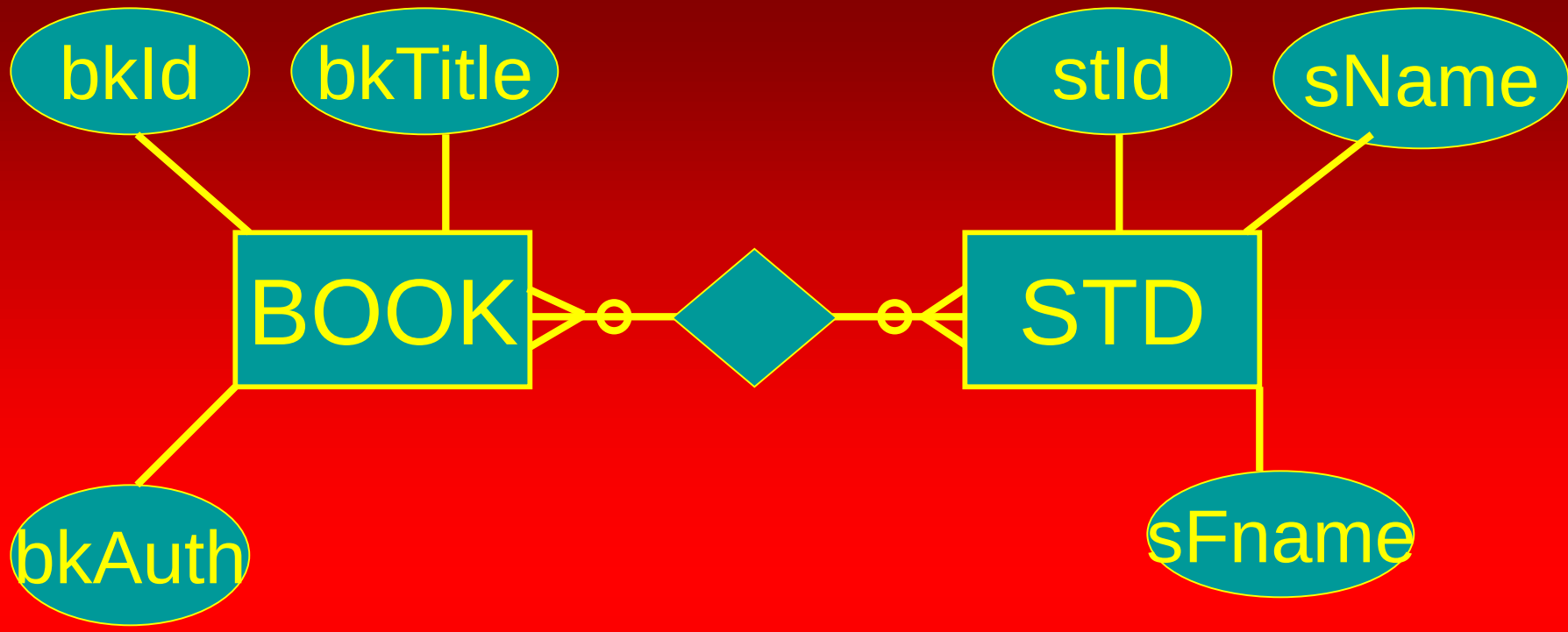


EMP(<u>empId</u>, empName, empAddr)
DEPT(<u>deptId</u>, deptName)

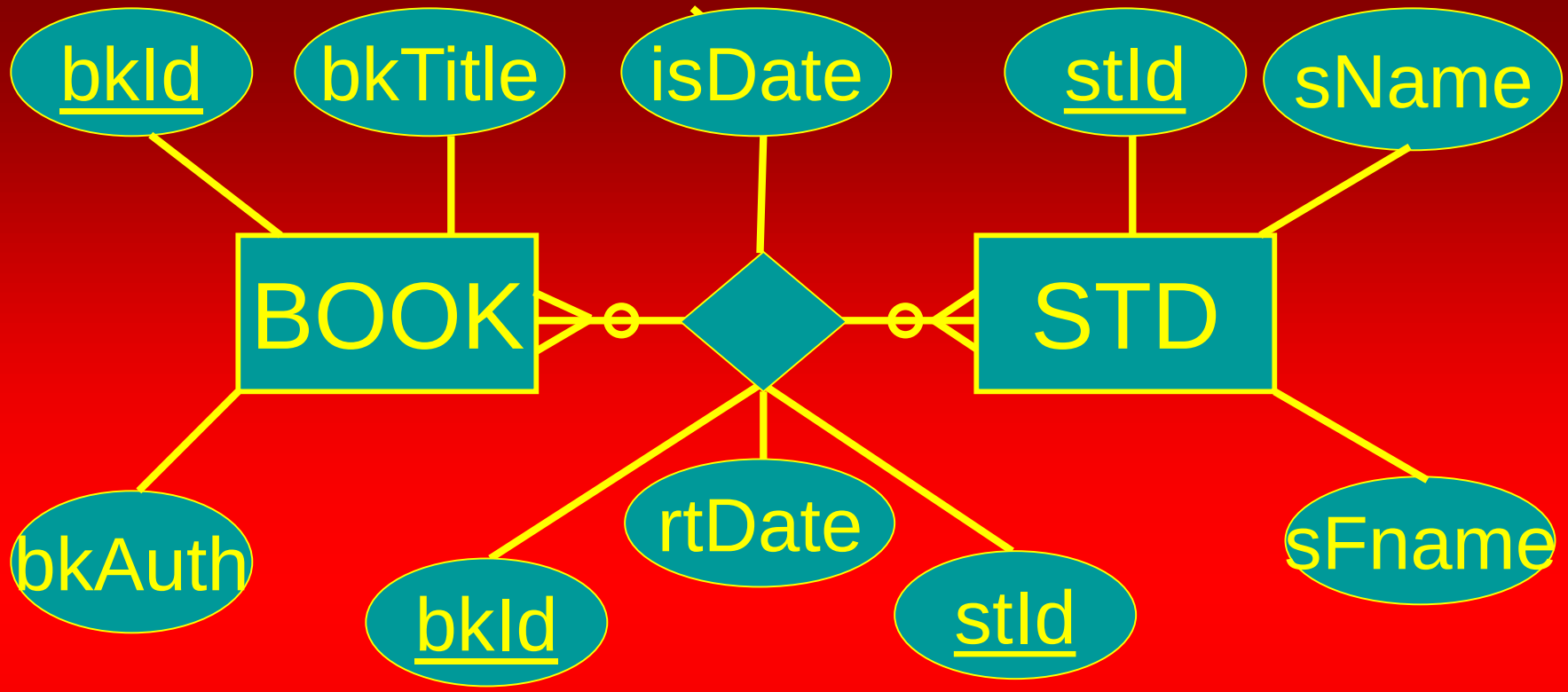
Many to Many Relationship

- A third table is created for the relationship, sometimes also called associative entity type
- Generally, the PKs of the participating ETs are used as PK of the third table

Many to Many Relationship Example



STD(<u>stId</u>, sName, sFname)
BOOK(<u>bkId</u>, bkTitle, bkAuth)
TARNS(<u>bkId</u>, <u>stId</u>)

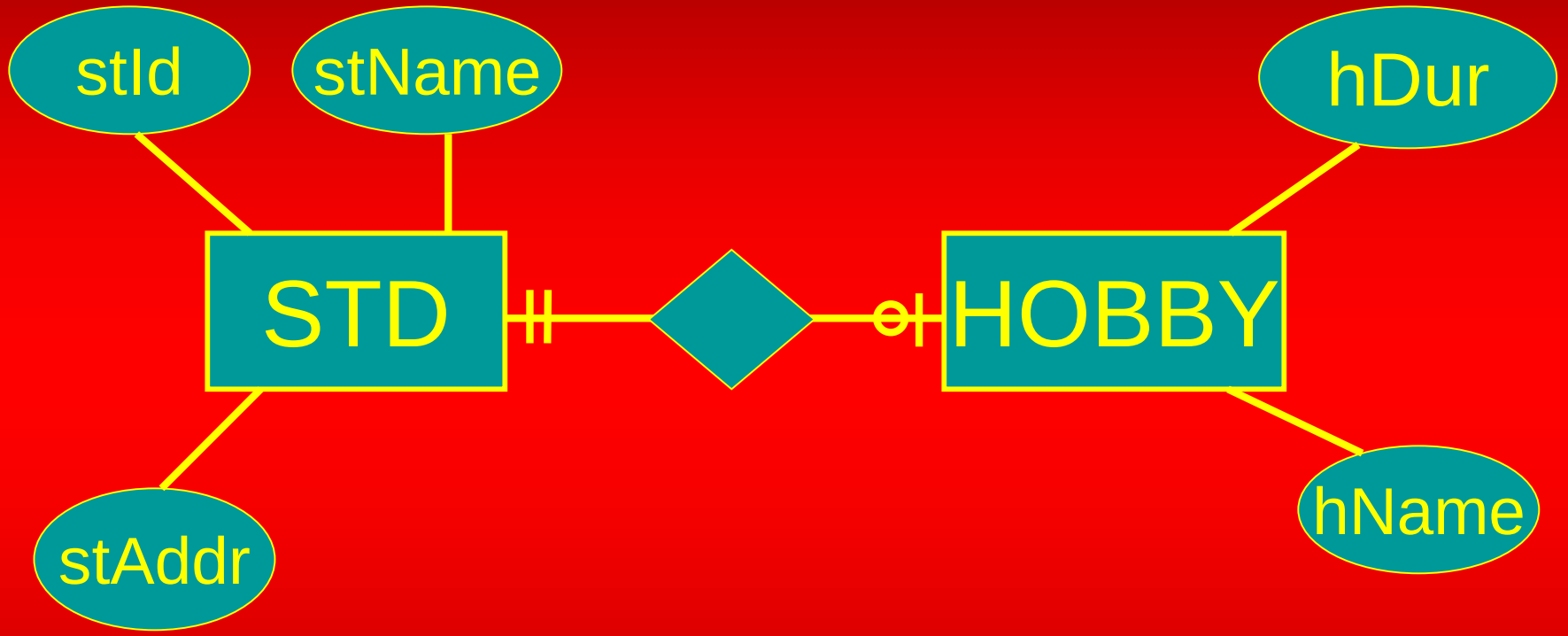


STD(stId, sName, sFname)
BOOK(bkId, bkTitle, bkAuth)
TARNS(bkId, stId, isDate, rtDate)

Binary one to one

- A special form of one to many relationship
- PK of one ET has to be included in the other as FK.
- Normally PK of compulsory side is included in the optional side

One to One Example

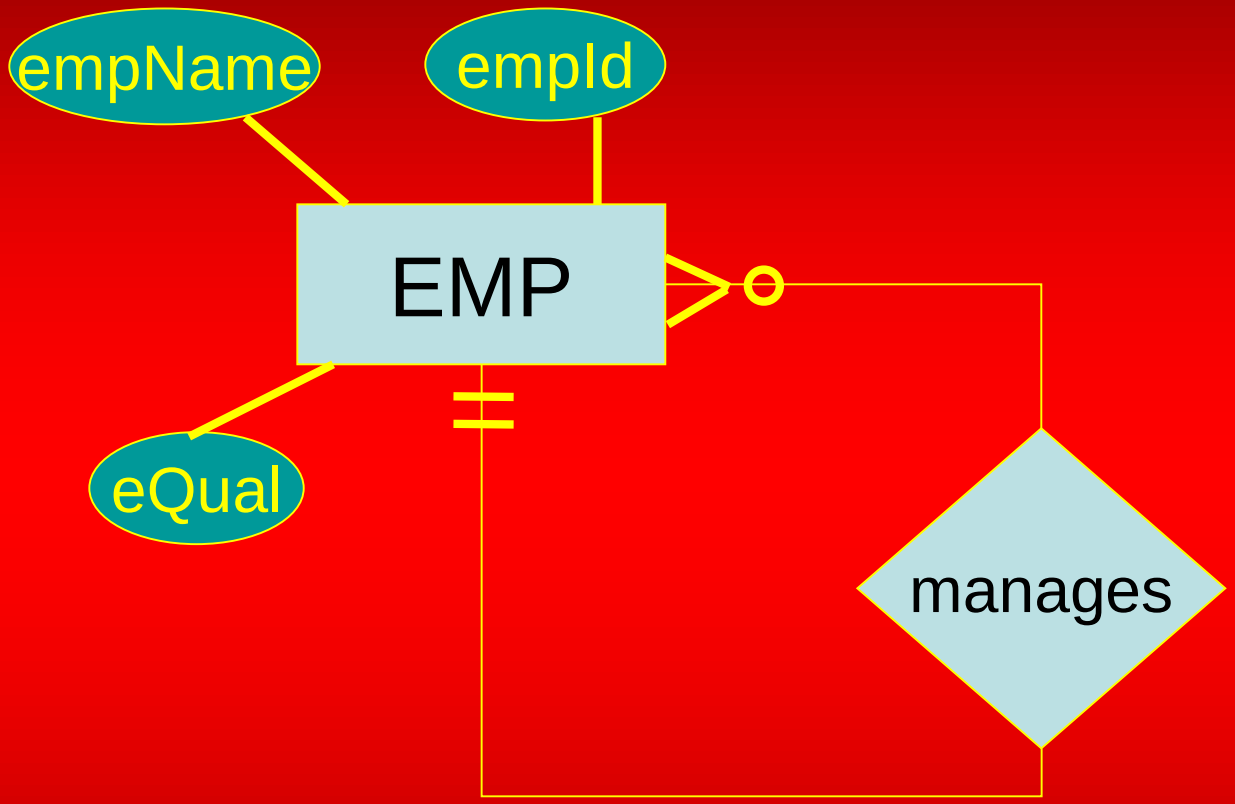


STD(stId, stName, stAddr)
HOBBY(hName, hDur, stId)

Unary Relationships

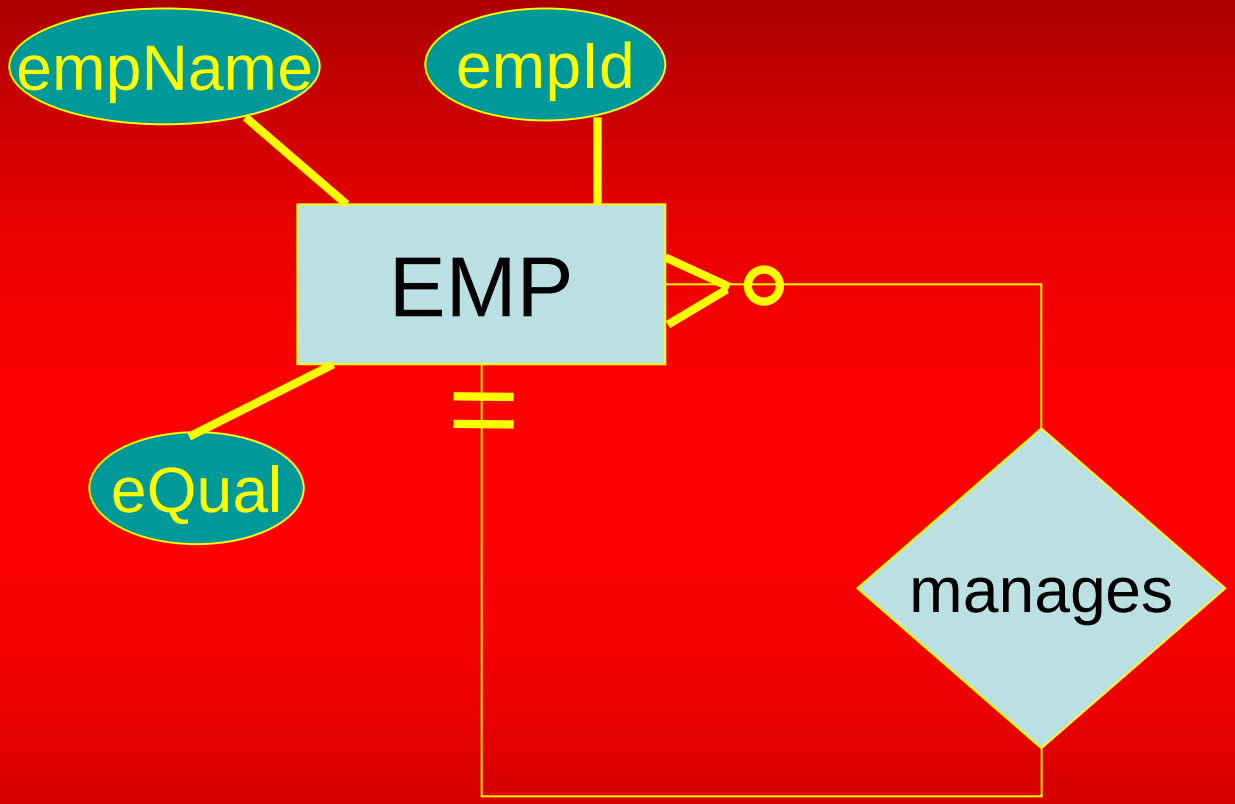
- Unary one to many introduces the PK of same ET as FK
- Many to many unary requires creation of another relation with a composite PK

Unary Example



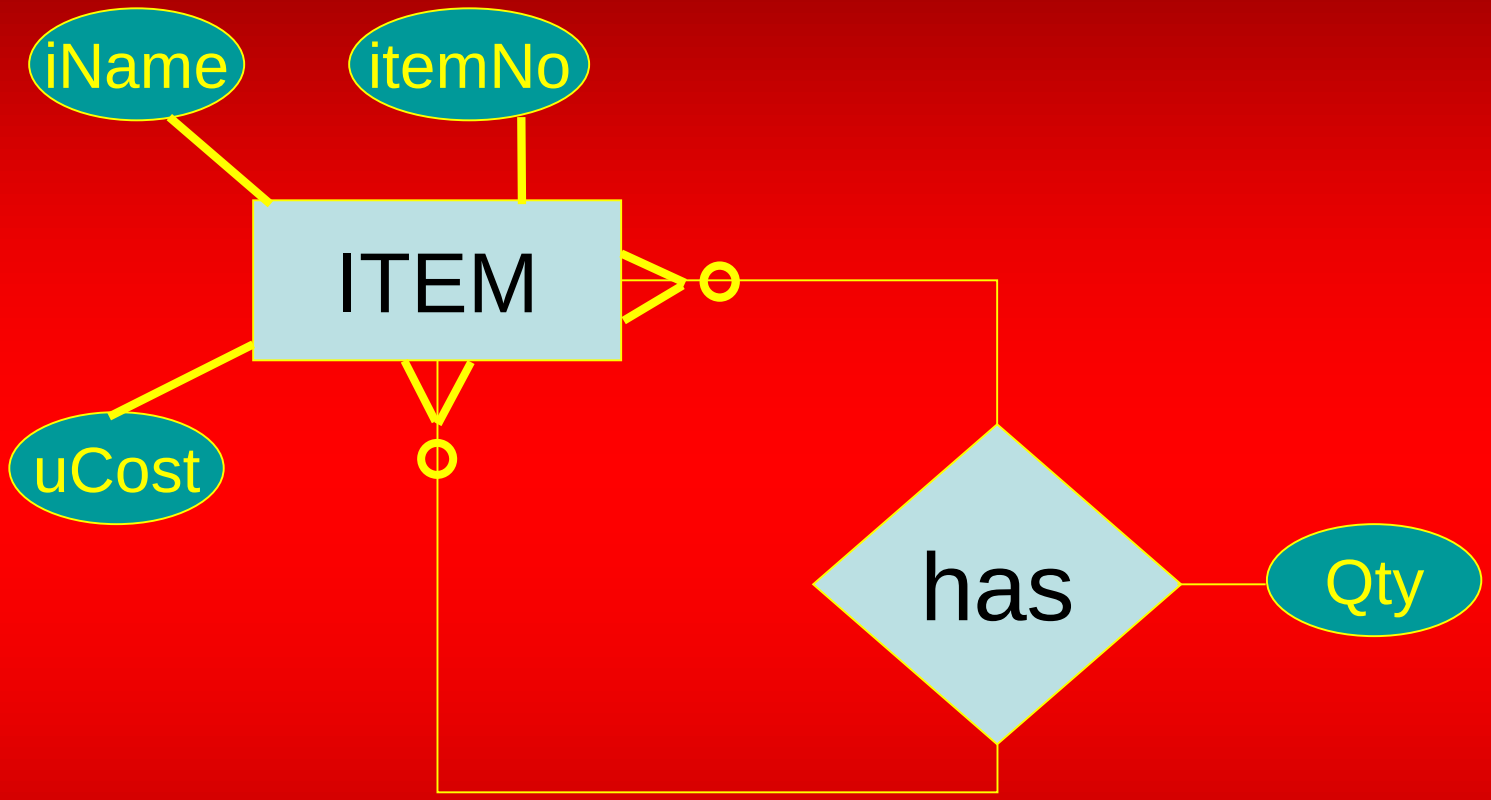
EMP(empId, empName, eQual, empId)

Unary Example



EMP(empId, empName, eQual, mgrId)

Unary Example



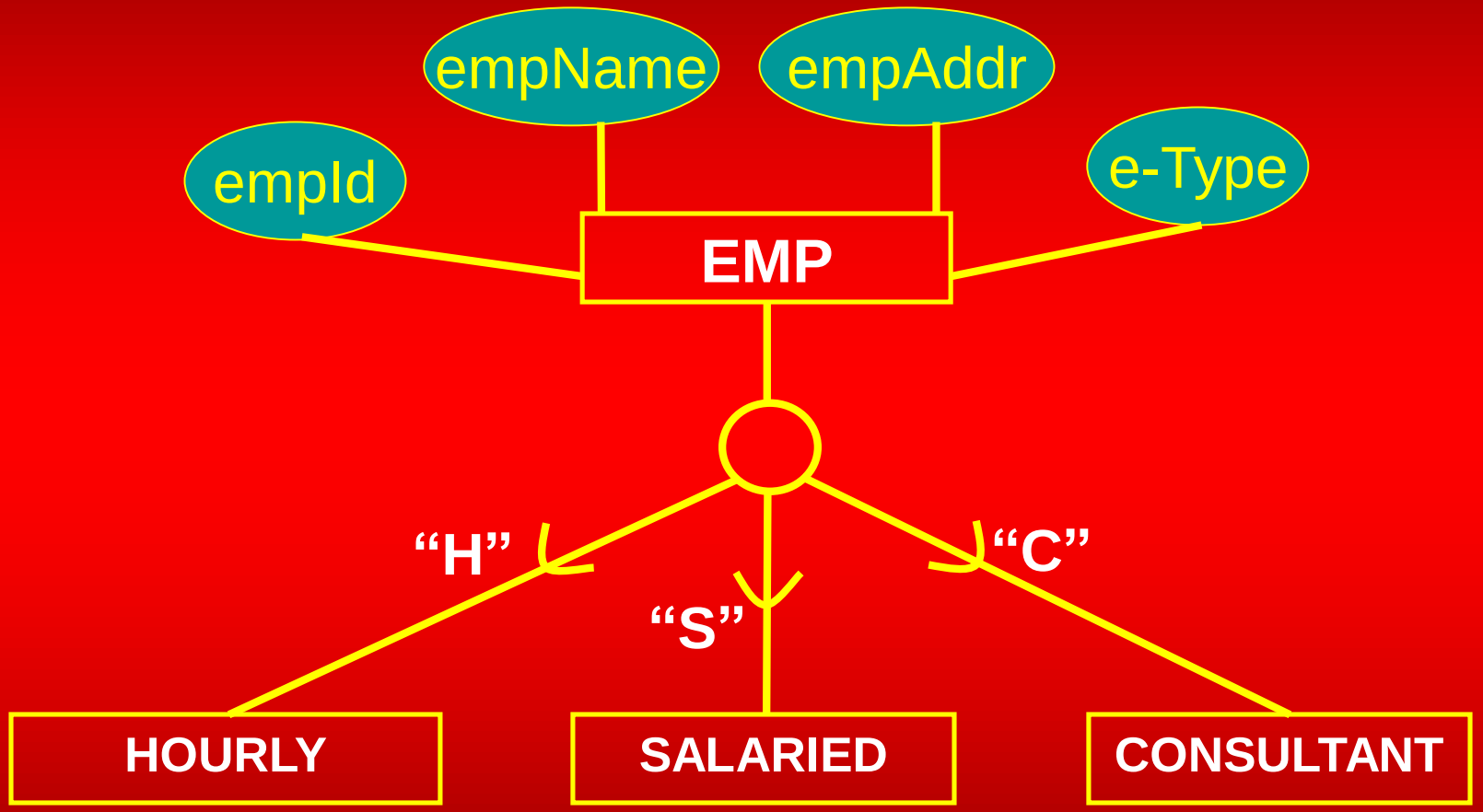
ITEM(<u>itemNo</u>, iName, uCost)

COMPONENT(<u>itemNo</u>, <u>CompNo</u>, Qty)

Super/Subtype Relationship

- Separate relations are created for each of the supertype and subtypes
- Assign relevant attributes
- Primary key of the supertype is included in all relations, that work for both the identity and the link

Super/Subtype example



Summary of Mapping E-R Diagram to Relational DM

DATA MANIPULATION LANGUAGES

Relational Algebra

- Relational algebra operations work on one or more relations to define another relation leaving the original intact.

Relational Algebra

- Both operands and results are relations, so output from one operation can become input to another operation.

Relational Algebra

- Allows expressions to be nested, just as in arithmetic. This property is called ***closure***.

Relational Algebra

- 5 basic operations in relational algebra: Selection, Projection, Cartesian product, Union, and Set Difference.

Relational Algebra

- These perform most of the data retrieval operations needed.
- Also have Join, Intersection, and Division operations, which can be expressed in terms of 5 basic operations.

Database Management System

Lecture - 17

Five Basic Operators

- Unary Operators
  1. Select
  2. Project

Five Basic Operators

- Binary Operators
  3. Union
  4. Set Difference
  5. Cartesian Product

Select

- Selects tuples that satisfy a given predicate
- Notation is Greek symbol sigma:

$$\sigma_{PREDICATE}(RELATION)$$

Select

- To process a selection,
  - Look at each tuple
  - See if we have a match (based on the condition)

Select

- The Degree of the resulting relation is the same as the degree of the relation

$$| \sigma | = | r(R) |$$

- σ is Commutative

$$\sigma_{c1} (\sigma_{c2}(R)) = \sigma_{c2} (\sigma_{c1}(R))$$

Selection Example

stId	stName	stAdr	prName	curSem
S1020	Sohail Dar	H#14, F/8-4,Islamabad.	MCS	4
S1038	Shoaib Ali	H#23, G/9-1,Islamabad	BCS	3
S1015	Tahira Ejaz	H#99, Lala Rukh Wah.	MCS	5
S1018	Arif Zia	H#10, E-8, Islamabad.	BIT	5

Selection Example

1. $\sigma_{\text{Curr_Sem} > 3}$ (STUDENT)
2. $\sigma_{\text{Studid} = \text{‘S1038’}}$ (STUDENT)

Selection Example Cont'd)

FacID	Fname	Address	Salary	Rank
F2345	Usman	H#Car409	21000	lecturer
F3456	Tahir	H#Bac100	23000	Asso Prof
F4567	Ayesha	H#Car301	27000	Asso Prof
F5678	Samad	H#Irv342	32000	Professor

Selection Example Cont'd)

1. $\sigma_{\text{Salary} > 27000}$ (FAC)
2. $\sigma_{(\text{Salary}>26000) \text{ and } (\text{Rank}=\text{'Asso Prof'})}$ (FAC)

Projection

- Unary operation that returns its argument relation with certain attributes left out.
- Any duplicate rows are eliminated.

Projection

- Notation

$$\Pi_{a1, a2, ..., ak} (R)$$

where a_1 ... a_k are attribute names and R is a relation name

Projection

- Works on a single relation R and defines a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates.

Projection

- Reduces duplicate columns created by cross product.
- Creates a New relation

Projection Example

FacID	Fname	Dept	Salary	Rank
F2345	Usman	CSE	21000	lecturer
F3456	Tahir	CSE	23000	Asso Prof
F4567	Ayesha	ENG	27000	Asso Prof
F5678	Samad	MNG	32000	Professor

Projection Example

- π FacID, Salary (Faculty)
- π Fname, Rank (Faculty)
- π Facid, Salary,Rank (Faculty)

Union

- Assuming R & S are union compatible…
- Union: R U S is the set of tuples in either R or S or both.

Union

- Since it is a set, there are no duplicate tuples
- Union is Commutative

$$R \cup S = S \cup R$$

Union Example

Course1

CID	ProgID	Cred_Hrs	CourseTitle
C2345	P1245	3	Operating Sytems
C3456	P1245	4	Database Systems
C4567	P9873	4	Financial Management
C5678	P9873	3	Money & Capital Market

Course2

CID	ProgID	Cred_Hrs	CourseTitle
C4567	P9873	4	Financial Management
C8944	P4567	4	Electronics

Union Example

Course 1 U Course 2

CID	ProgID	Cred_Hrs	CourseTitle
C2345	P1245	3	Operating Sytems
C3456	P1245	4	Database Systems
C4567	P9873	4	Financial Management
C5678	P9873	3	Money & Capital Market
C8944	P4567	4	Electronics

Intersection (∩)

- Assuming that R & S are union compatible
- Intersection: R ∩ S is the set of tuples in both R and S

Intersection (∩)

- Intersection is Commutative

$$R \cap S = S \cap R$$

Intersection (∩) Example

Course 1 ∩ Course 2

CID	ProgID	Cred_Hrs	CourseTitle
C4567	P9873	4	Financial Management

Difference (-)

- Difference: R – S is the set of tuples that appear in R but do not appear in S.

Difference (-) Example

Course1 – Course2

CID	ProgID	Cred_Hrs	CourseTitle
C2345	P1245	3	Operating Sytems
C3456	P1245	4	Database Systems
C5678	P9873	3	Money & Capital Market

Cartesian Product (X)

- Sets do not have to be union compatible.

$R(A_1, A_2, \ldots, A_N)$

X

$S(B_1, B_2, \ldots, B_M)$ is

Cartesian Product (X)

$Q(A_1, A_2, \ldots, A_{n,} B_1, B_2, \ldots, B_m)$

- If R has C tuples and S has D tuples, the result is C*D tuples.

Cartesian Product (X)

- Also called “cross product”
- Note that union, intersection and cross product are commutative and associative

Cartesian Product (X) Example

Course X Registration

Course

CID	CourseTitle
C3456	Database Systems
C4567	Financial Management
C5678	Money & Capital Market

Registration

SID	StudName
S101	Ali Tahir
S103	Farah Hasan

Cartesian Product (X) Example

CID	CourseTitle	SID	StudName
C3456	Database Systems	S101	Ali Tahir
C4567	Financial Management	S101	AliTahr
C5678	Money & Capital Market	S101	Ali Tahir
C3456	Database Systems	S103	Farah Hasan
C4567	Financial Management	S103	Farah Hasan
C5678	Money & Capital Market	S103	Farah Hasan

Join Operation

Lecture 17 ends here…

Database Management System

Lecture - 18

Types of Join

- A special form of cross product of two tables
- Different types
- Theta, Equi, Natural, Semi, Outer Joins are different types

Theta Join

We apply the condition through select on one relation and then only those rows are used in the cross product with the second relation

$$R \bowtie_{\theta} S$$

Theta Join Example

FACULTY

facId	facName	dept	salary	rank
F2345	Usman	CSE	21000	lecturer
F3456	Tahir	CSE	23000	Asso Prof
F4567	Ayesha	ENG	27000	Asso Prof
F5678	Samad	MNG	32000	Professor

COURSE

crCode	crTitle	fId
C3456	Database Systems	F2345
C4567	Financial Management	
C5678	Money & Capital Market	F4567
C3425	Introduction to Accounting	F2345

$(\sigma_{\text{rank = 'Asso Prof'}}(\text{FACULTY})) \times \text{COURSE}$

$(\sigma_{\text{rank = 'Asso Prof'}}(\text{FACULTY}))$ X COURSE

facId	facName	dept	salary	Rank	crCode	crTitle	fId
F3456	Tahir	CSE	23000	Asso Prof	C3456	Database Systems	F2345
F3456	Tahir	CSE	23000	Asso Prof	C4567	Financial Management	
F3456	Tahir	CSE	23000	Asso Prof	C5678	Money & Capital Market	F4567
F3456	Tahir	CSE	23000	Asso Prof	C3425	Introduction to Accounting	F2345
F4567	Ayesha	ENG	27000	Asso Prof	C3456	Database Systems	F2345
F4567	Ayesha	ENG	27000	Asso Prof	C4567	Financial Management	
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market	F4567
F4567	Ayesha	ENG	27000	Asso Prof	C3425	Introduction to Accounting	F2345

Equijoin

- Rows are joined on the basis of values of a common attribute between the two relations
- Rows having he same value in the common attribute are joined

Equijoin

- Common attributes appears twice in the output
- Common attribute with the same name is qualified with the relation name in the output

Equijoin Example

FACULTY

facId	facName	dept	salary	rank
F2345	Usman	CSE	21000	lecturer
F3456	Tahir	CSE	23000	Asso Prof
F4567	Ayesha	ENG	27000	Asso Prof
F5678	Samad	MNG	32000	Professor

COURSE

crCode	crTitle	fId
C3456	Database Systems	F2345
C4567	Financial Management	
C5678	Money & Capital Market	F4567
C3425	Introduction to Accounting	F2345

FACULTY ⋈ FACULTY.facId = COURSE.fId COURSE

Equijoin Example

FACULTY $\bowtie$ FACULTY.facId = COURSE.fId COURSE

facId	facName	dept	salary	Rank
F2345	Usman	CSE	21000	lecturer
F4567	Ayesha	ENG	27000	Asso Prof
F2345	Usman	CSE	21000	lecturer

crCode	crTitle	fId
C3456	Database Systems	F2345
C5678	Money & Capital Market	F4567
C3425	Introduction to Accounting	F2345

Equijoin Example

FACULTY $\bowtie_{\text{FACULTY.facId = COURSE.facId}}$ COURSE

FACULTY.facId	acName	dept	salary	Rank	crCode	crTitle	COURSE.facId
F2345	Usman	CSE	21000	lecturer	C3456	Database Systems	F2345
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market	F4567
F2345	Usman	CSE	21000	lecturer	C3425	Introduction to Accounting	F2345

Natural Join

- Also called simply the join, most general form of join
- Same as equijoin with common column appearing once

Natural Join Example

FACULTY

facId	facName	dept	salary	rank
F2345	Usman	CSE	21000	lecturer
F3456	Tahir	CSE	23000	Asso Prof
F4567	Ayesha	ENG	27000	Asso Prof
F5678	Samad	MNG	32000	Professor

COURSE

crCode	crTitle	facId
C3456	Database Systems	F2345
C4567	Financial Management	
C5678	Money & Capital Market	F4567
C3425	Introduction to Accounting	F2345

FACULTY $\bowtie$ FACULTY.facId, COURSE.facId COURSE

Natural Join Example

FACULTY $\bowtie$ FACULTY.facId, COURSE.facId COURSE

facId	facName	dept	salary	Rank	crCode	crTitle
F2345	Usman	CSE	21000	lecturer	C3456	Database Systems
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market
F2345	Usman	CSE	21000	lecturer	C3425	Introduction to Accounting

Types Of Joins

- Left Outer Join
- Right Outer Join
- Outer Join
- Semijoin

Left Outer Join

- Keep all of the tuples from the "left" relation
- Join with the right relation
- Pad the non-matching tuples with nulls

Left Outer Join Example

FACULTY LEFT JOIN COURSE

facId	facName	dept	salary	rank	crCode	crTitle	facId
F2345	Usman	CSE	21000	lecturer	C3456	Database Systems	F2345
F2345	Usman	CSE	21000	lecturer	C3425	Intro. To Accounting	
F3456	Tahir	CSE	23000	Asso Prof			
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market	F4567
F5678	Samad	MNG	32000	Professor			

Right Outer Join

Same as the left, but keep tuples from the “right” relation

Right Outer Join Example

FACULTY RIGHT OUTER JOIN COURSE

facId	facName	dept	salary	rank	crCode	crTitle	facId
F2345	Usman	CSE	21000	lecturer	C3456	Database Systems	F2345
					C4567	Financial Management	
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market	F4567
F2345	Usman	CSE	21000	lecturer	C3425	Intro. To Accounting	F2345

Outer Join

Same as left, but keep all tuples from both relations

Outer Join Example

FACULTY OUTER JOIN COURSE

facId	facName	dept	salary	rank	crCode	crTitle	facId
F2345	Usman	CSE	21000	lecturer	C3456	Database Systems	F2345
F2345	Usman	CSE	21000	lecturer	C3425	Intro. To Accounting	F2345
F4567	Ayesha	ENG	27000	Asso Prof	C5678	Money & Capital Market	F4567
F3456	Tahir	CSE	23000	Asso Prof			
F5678	Samad	MNG	32000	Professor			
					C4567	Financial Management	

Semijoin

- First take the natural join of two tables
- Then take the projection on the attributes of first table

FACULTY $\ltimes$ COURSE

Semijoin Example

FACULTY ⋉ COURSE

facId	facName	dept	salary	rank
F2345	Usman	CSE	21000	lecturer
F4567	Ayesha	ENG	27000	Asso Prof

Relational Calculus

Relational Calculus

- Relational calculus is a non-procedural formal data manipulation language.
- Use simply specifies what data should be retrieved.

Relational Calulus

- Comes in two forms
  1. Tuple- Oriented Relational Calculus
  2. Domain-Oreinted Relation Calculus.

Tuple-Oriented Relational Calculus

- Finding relational tuple s for which a predicate is true.
- A tuple variable is a variable that takes on only the tuples of some relations or relations as its RANGE of value.

Tuple-Oriented Relational Calculus

- Specify the range of tuple variable by statement

  RANGE OF S IS STUDENT

Tuple-Oriented Relational Calculus

- S is a tuple variable and STUDENT is the range.
- S represent the tuple of STUDENT

Tuple-Oriented Relational Calculus

- Express as

$$\{ S \mid P(S) \}$$

- Find he set of all tuples S such that P(S) is true, where P implies the predicate condition.

Tuple-Oriented Relational Calculus

- RANGE of R is STUDENT

  { R | R.Credits > 50}

- Find the studid, stuName, major, credits etc of all student having more than 50 credits.

Domain-Oriented Relational Calculus

- Use variables that take values from domain instead of tuple of relations.

Domain-Oriented Relational Calculus

- If P(x1,x2,…..,xn) stands for predicate with variables x1,x2,…..,xn then

Domain-Oriented Relational Calculus

{<x1,x2,….,xn>| P(x1,x2,….,xn))

- which means, the set of all domain variables x1,x2,….,xn for which predicate p(x1,x2,…,xn) is true.

Domain-Oriented Relational Calculus

- we test for membership condition, to determine whether values belong to a relation.

NORMALIZATION

Normalization

- A step by step process to produce more efficient and accurate database design
- Purpose is to produce an anomaly free design

Anomalies

- An inconsistent, incomplete or incorrect state of database
- Four types of anomalies are of concern here; Redundancy, insertion, deletion and updation

Normalization

- A strongly recommended step
- Normalized design makes the maintenance of database easier
- Normalization applied on each table of a DB design

Normalization

- Performed after the logical database design
- Informally also performed during conceptual DB design

Normalization Process

- Different forms or levels of normalization
- Called first, second, third and so on forms
- Each form has got certain conditions
- If a table fulfils the condition(s) for a normal form then the table is in that normal form

Normalized DB Design

- Process is applied on each table
- The minimum form in which all tables are in is called the normal form of the entire database
- Objective is to place the DB in highest form of normalization

Database Management System

Lecture - 18

Database Management System

Lecture - 19

Functional Dependency

- Normalization is based on functional dependencies (FDs)
- A type of relationship between attributes of a relation

Functional Dependency

Definition: If A and B are attributes of a relation R, then B is functionally dependent on A if each value of A in R is associated with exactly one value of B; written as A $\longrightarrow$ B

Functionally Dependency

- It does not mean that A derives B, although it may be the case sometime
- Means that if we know value of A then we can precisely determine a unique value of B

Functional Dependency

- Attribute of set of attributes on the left side are called determinant and on the right are called dependents
- Like R (a, b, c, d, e)

  a ⟶ b, c, d

  d ⟶ d, e

FD Example

STD(stId, stName, stAdr, prName, credits)

stId $\longrightarrow$ stName, stAdr, prName, credits

prName $\longrightarrow$ credits

FD Example from Table

stId	stName	stAdr	prName	prCrdts
S1020	Sohail Dar	I-8 Islamabad	MCS	64
S1038	Shoaib Ali	G-6 Islamabad	BCS	132
S1015	Tahira Ejaz	L Rukh Wah	MCS	64
S1015	Tahira Ejaz	L Rukh Wah	MCS	132
S1018	Arif Zia	E-8, Islamabad.	BIT	134

FDs and Keys

- We can determine the keys of a table seeing its FDs
- The determinant of an FD that determines all attributes of that table is the super key

FDs and Keys

- A minimal super key is the candidate key, so if a determinant of an FD determines all attributes of that relation then it is definitely a super key, and

FDs and Keys

- If there is no other FD where a subset of this determinant/SK is a super key, then it is a candidate key
- So FDs help to identify keys, how

FDs and Keys

EMP(eId, eName, eAdr, eDept, prId, prSal)

eId → eName, eDept, eAdr

eId, prId → prSal

STD(stId, stName, prName, adr, nic, cgpa)

stId → stName, prName, adr, nic, cgpa

nic → stName, prName, adr, stId, cgpa

Inference Rules

- Called inference axioms or armstrong axioms
- These are rules that establish certain FDs from a given set of FDs
- These rules are sound

Reflexivity

- If B is a subset of A then A $\rightarrow$ B, it also implies that A $\rightarrow$ A always hold, that is

  stName, stAdr $\rightarrow$ stName

Or stName $\rightarrow$ stName

Augmentation

- If we have A $\longrightarrow$ B then

  AC $\longrightarrow$ BC that is

  if stId $\longrightarrow$ stName then

  stId, stAdr $\longrightarrow$ stName, stAdr

Transitivity

- If A $\rightarrow$ B and B $\rightarrow$ C then A $\rightarrow$ C

that is

If stId $\rightarrow$ prName and prName $\rightarrow$ credits

Then

stId $\rightarrow$ credits

Additivity or Union

- If A $\longrightarrow$ B and A $\longrightarrow$ C then A $\longrightarrow$ BC

if empId $\longrightarrow$ eName and empId $\longrightarrow$ qual

Then we can write it as

empId $\longrightarrow$ eName, qual

Projectivity or Decomposition

- If A $\longrightarrow$ BC then A $\longrightarrow$ B and A $\longrightarrow$ C

  if empId $\longrightarrow$ eName, qual

  Then we can write it as

  empId $\longrightarrow$ eName and empId $\longrightarrow$ qual

Pseudotransitivity

- If A → B and CB → D then AC → D

  if stId → stName and

  stName, fName → stAdr

  Then we can write it as

  stId, fName → stAdr

Normal Forms

First Normal Form

- A relation is in first normal form iff every attribute in every tuple contains an atomic value
- There is no multivaued (repeating group) in the relation

First Normal Form

- One of the basic properties of relation
- Multiple values create problems in performing different operations, like, select or join
- Remember the treatment of multivalued attributes in the mapping process

First Normal Form

STD(stId, stName, stAdr, prName, bkId)

stId	stName	stAdr	prName	bkId
S1020	Sohail Dar	I-8 Islamabad	MCS	B00129
S1038	Shoaib Ali	G-6 Islamabad	BCS	B00327
S1015	Tahira Ejaz	L Rukh Wah	MCS	B08945, B06352
S1018	Arif Zia	E-8, Islamabad.	BIT	B08474

First Normal Form

stId	stName	stAdr	prName	bkId
S1020	Sohail Dar	I-8 Islamabad	MCS	B00129
S1038	Shoaib Ali	G-6 Islamabad	BCS	B00327
S1015	Tahira Ejaz	L Rukh Wah	MCS	B08945
S1015	Tahira Ejaz	L Rukh Wah	MCS	B06352
S1018	Arif Zia	E-8, Islamabad.	BIT	B08474

Mind the key please

Second Normal Form

Full functional dependency: an attribute B is fully functionally dependent on A if the B can be determined by whole of A not by any proper subset of A

Full Functional Dependence

Consider the relation

CLASS(crId, stId, stName, fId, room, grade)

crId, stId → stName, fId, room, grade

stId → stName

crId → fId, room

Database Management System

Lecture - 19

Database Management System

Lecture - 20

Second Normal Form

- A relation is in 2nd normal form iff it is in the first normal form and all non key attributes are fully functionally dependent on key, that is, there is no partial dependency

Second Normal Form

CLASS(crId, stId, stName, fId, room, grade)

crId, stId → stName, fId, room, grade

stId → stName

crId → fId, room

Anomalies

- Redundancy
- Insertion Anomaly
- Deletion Anomaly
- Updation Aomaly

Anomalies

crId	stId	stName	fId	room	grade
C3456	S1020	Suhail Dar	F2345	104	B
C5678	S1020	Suhail Dar	F4567	106	
C3456	S1038	Shoaib Ali	F2345	104	A
C5678	S1015	Tahira Ejaz	F4567	106	B

Second Normal Form

Relation is decomposed based on the FDs

CLASS(crId, stId, stName, fId, room, grade)

crId, stId → stName, fId, room, grade

stId → stName

crId → fId, room

STD(stId, stName)

COURSE(crId, fId, room)

CLASS(crId, stId, grade)

Second Normal Form

- Each of these tables is in second normal form
- Free of anomalies due to partial dependency

Third Normal Form

A table is in third normal form (3NF) iff it is in 2NF and there is no transitive dependency, that is, no non-key attribute is dependent on another non-key attribute

Transitive Dependency

STD(stId, stName, stAdr, prName, prCrdts)

stId → stName, stAdr, prName, prCrdts

prName → prCrdts

Anomalies

stId	stName	stAdr	prName	prCrdts
S1020	Sohail Dar	I-8 Islamabad	MCS	64
S1038	Shoaib Ali	G-6 Islamabad	BCS	132
S1015	Tahira Ejaz	L Rukh Wah	MCS	64
S1015	Tahira Ejaz	L Rukh Wah	MCS	64
S1018	Arif Zia	E-8, Islamabad.	BIT	134

Third Normal Form

- STD(stId, stName, stAdr, prName, prCrdts)
- stId → stName, stAdr, prName, prCrdts
- prName → prCrdts
- STD (stId, stName, stAdr, prName)
- PROGRAM (prName, prCrdts)

3NF Relations

- Each of the table is in 3NF
- Free of all anomalies

Boyce-Codd Normal Form

- A general form of 3NF
- Every relation in BCNF is in 3NF vice-versa is not always true
- 3NF is checked in steps, BCNF checked directly

BCNF

- A table is in BCNF if every determinant is a candidate key
- Situation when table in 3NF is not in BCNF
- A non-key determines a part of the composite primary key

BCNF

FACULTY(fId, dept, office, rank, dateHired)

fId, dept → office, rank, dateHired

office → dept

- Table is in 3NF, not in BCNF since the office is not a candidate key
- Generates multiple overlapping candidate keys so we have fId, office → dept, rank, dateHired

BCNF

- We decompose the table again to bring it into BCNF

FACULTY (fId, dept, office, rank, dateHired)

FACULTY(fId, office, rank, dateHred)

OFFICE(office, dept)

Higher Normal Forms

- After BCNF fourth, fifth and domain-key normal forms exist
- 4NF deals with multivalued dependency, fifth deals with possible lossless decompositions, DKNF reduces further chances of any possible inconsistency

Database Management System

Lecture - 20

Database Management System

Lecture - 21

Normalization Summary

- A step by step process to make DB design more efficient and accurate
- A strongly recommended activity performed after the logical DB design phase

Normalization Summary

- Un-normalized relations are more prone to errors or inconsistencies
- Normalization is based on the FDs
- FDs are not created rather identified by the designer/analyst

Normalization Summary

- Normalization forms exist up to 6NF, however, for most of the situations 3NF is sufficient
- Performed through Analysis or Synthesis process

Normalization Example

- Identify FDs
- Apply on the relevant tables; see if any normalization requirement is being violated, that is, causing some anomaly

Normalization Example

PROJNAME	PROJMGR	EMPID	HOURS	EMPNAME	BUDGET	STARTDATE	SALARY	EMPMGR	EMPDEPT	RATING
	Different Data as mentioned in the book...									

Some Facts

1. Each project has a unique name, but names of employees and managers are not unique
2. Each project has one manager, whose name is stored in PROJMGR

PROJNAME → PROJMGR

3. Many employees may be assigned to work on each project, and an employee may be assigned to more than one project. HOURS tells the number of hours per week that a particular employee is assigned to work on a particular project

PROJNAME, EMPID  HOURS

4. Budget stores the amount budgeted for a project, and STARTDATE gives the starting date for a project

PROJNAME ⟶ PROJMGR, BUDGET, STARTDATE

5. Salary gives the annual salary of an employee



6. EMPMGR gives the name of the employee's manager, who is not the same as the project manager

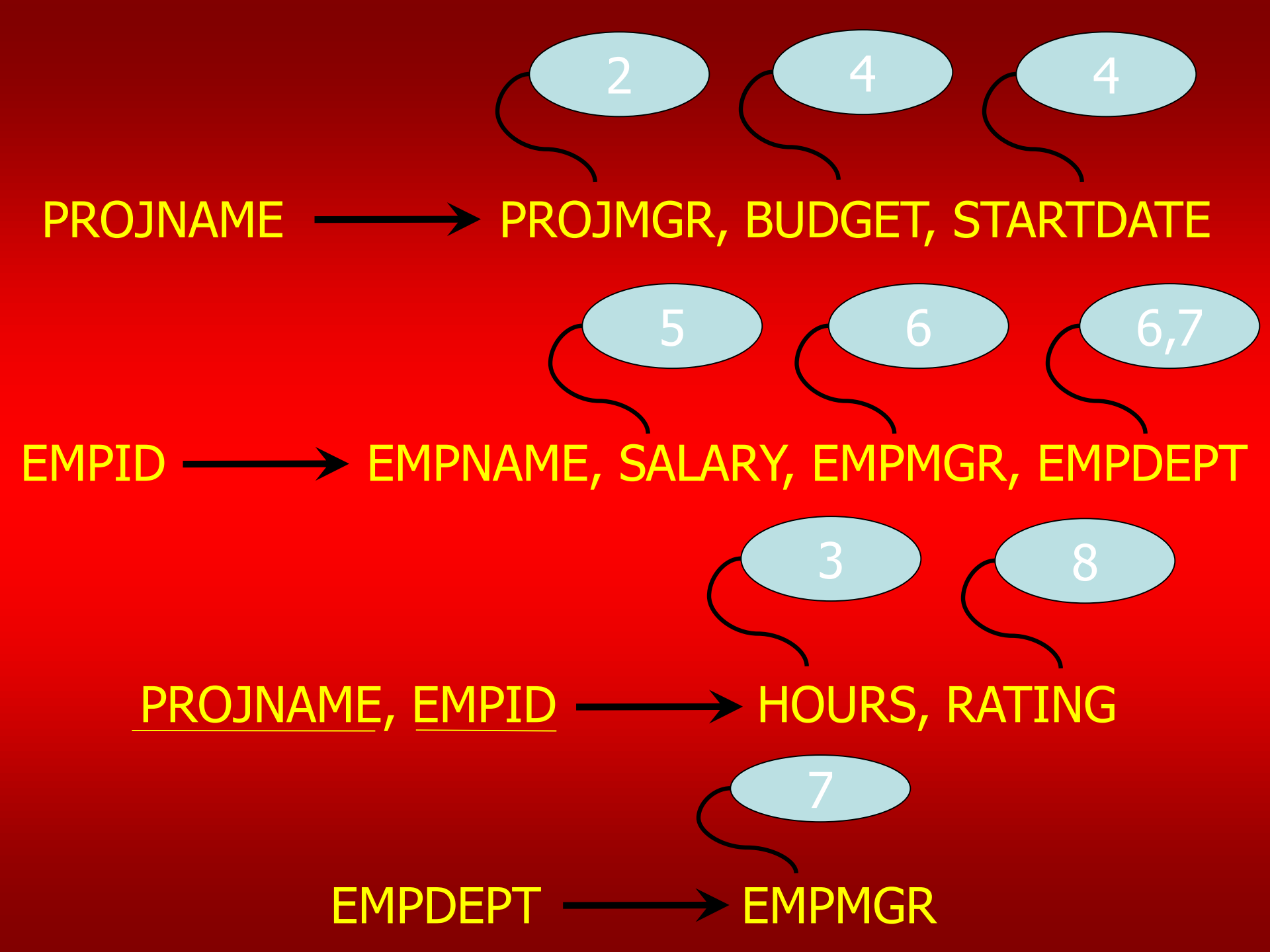


7. EMPDEPT gives the employee's department. Department names are unique. The employee's manager is the manager of the employee's department



8. RATING gives the employee's rating for a particular project. The project manager assigns the rating at the end of the employee's work on that project

PROJNAME, EMPID ⟶ RATING


2
4
4
PROJNAME
PROJMGR, BUDGET, STARTDATE
5
6
6,7
EMPID
EMPNAME, SALARY, EMPMGR, EMPDEPT
3
8
PROJNAME, EMPID
HOURS, RATING
7
EMPDEPT
EMPMGR

Original relation:

WORK ⟶ (PROJNAME, PROJMGR, EMPID, HOURS, EMPNAME, BUDGET, STARTDATE, SALARY, EMPMGR, EMPDEPT, RATING)

New relations:

PROJ (PROJNAME, PROJMGR, BUDGET, STARTDATE)

EMP (EMPID, EMPNAME, SALARY, EMPMGR, EMPDEPT)

WORK (PROJNAME, EMPID, HOURS, RATING)

PROJ (PROJNAME, PROJMGR, BUDGET, STARTDATE)

EMP (EMPID, EMPNAME, SALARY, EMPMGR, EMPDEPT)

WORK (PROJNAME, EMPID, HOURS, RATING)

PROJ (PROJNAME, PROJMGR, BUDGET, STARTDATE)

EMP (EMPID, EMPNAME, SALARY, EMPDEPT)

DEPT (EMPDEPT, EMPMGR)

WORK (PROJNAME, EMPID, HOURS, RATING)

Checking for BCNF

PROJ (PROJNAME, PROJMGR, BUDGET, STARTDATE)

PROJNAME ⟶ PROJMGR, BUDGET, STARTDATE

EMP (EMPID, EMPNAME, SALARY, EMPDEPT)

EMPID ⟶ EMPNAME, SALARY, EMPMGR, EMPDEPT

WORK (PROJNAME, EMPID, HOURS, RATING)

PROJNAME, EMPID $\longrightarrow$ HOURS, RATING

DEPT (EMPDEPT, EMPMGR)

EMPDEPT ⟶ EMPMGR

Physical Database Design

Objective

- Basic goal is data processing efficiency
- Transforms logical DB design into technical specifications for storing and retrieving data
- Does not include practically implementing the design however tool specific decisions are involved

Inputs Required

- Normalized relations
- Definitions of each attribute
- Descriptions of data usage
- Requirements for response time, data security, backup etc.
- Tool to be used

Decisions Involved

1. Choosing data types
2. Grouping attributes (although normalized)
3. Deciding file organizations
4. Selecting structures
5. Preparing strategies for efficient access

Database Management System

Lecture - 21

Database Management System

Lecture - 22

Data Volume and Usage Analysis

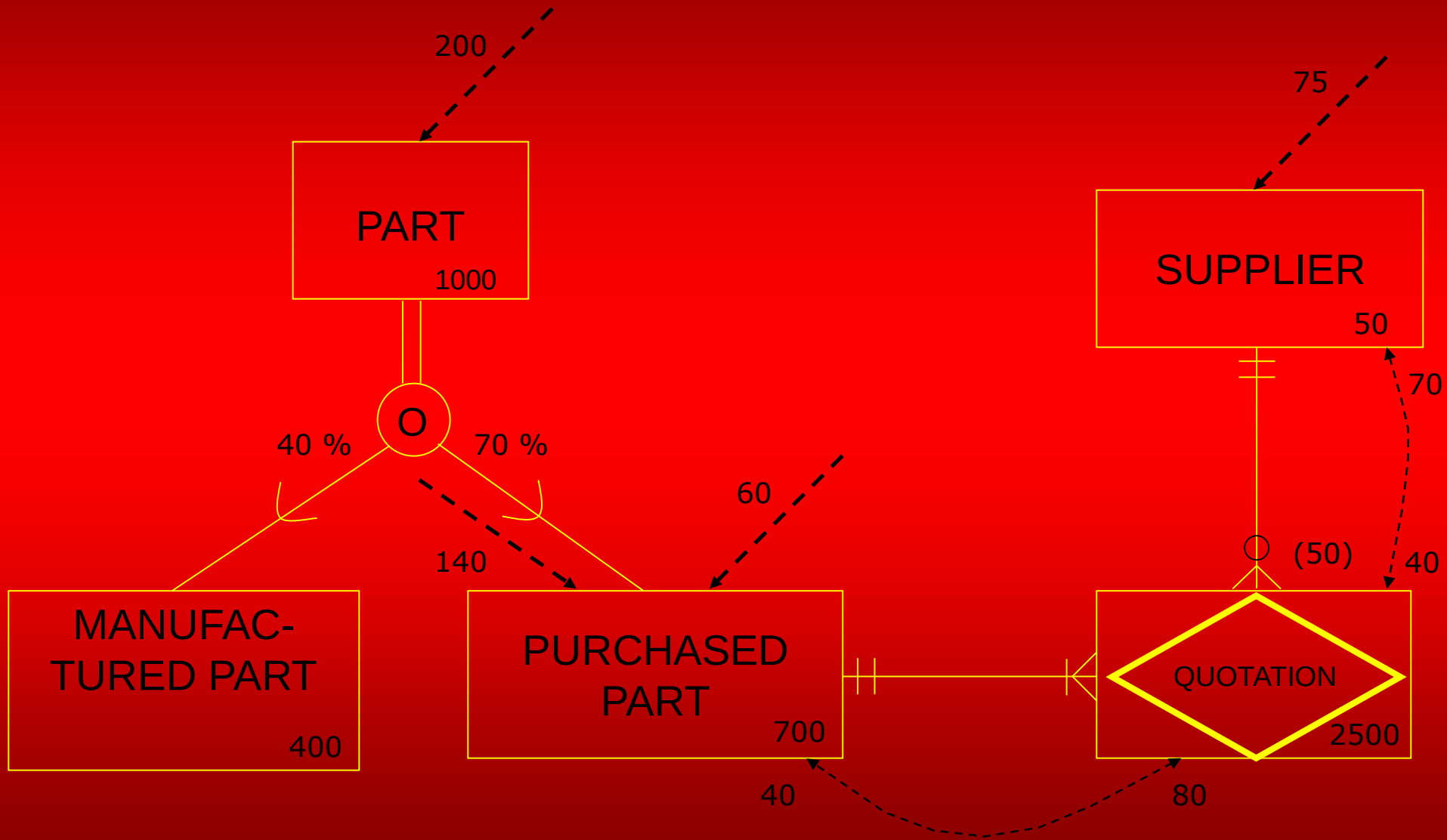
- Statistics about the size and usage of data plays critical role in data processing efficiency
- Final step of logical DB design or first step in physical DB design

Data Volume and Usage Analysis

- Statistics collected during analysis phase from the users of the system
- May not be accurate; give the tentative and relevant figures

Data Volume and Usage Analysis

Composite Usage Map



Designing Fields

- Field is smallest unit of application data; corresponds to a simple attribute
- Involves different decisions about fields

Choosing Data Type

- Data type is defined as set of values along with the operations that can be performed on them
- Precisely depends on the particular DBMS

Choosing Data Types

- Involves four objectives
  - Minimize storage space
  - Represent all possible values
  - Improve data integrity
  - Support all data manipulation

Coding Techniques

- Values of the attributes with small domains can be replaced by codes
- Codes can be stored in lookup table or can be hard coded, example

Coding Example

STUDENT

stId	stName	hobby
S1020	Sohail Dar	Reading
S1038	Shoaib Ali	Gardening
S1015	Tahira Ejaz	Reading
S1015	Tahira Ejaz	Movies
S1018	Arif Zia	Reading

Coding Example

STUDENT

stId	stName	hobby
S1020	Sohail Dar	R
S1038	Shoaib Ali	G
S1015	Tahira Ejaz	R
S1015	Tahira Ejaz	M
S1018	Arif Zia	R

HOBBY

code	Hobby
R	Reading
G	Gardening
M	Movies

Controlling Data Integrity

- Concerns the possible values that a field can assume
- First such control is enforced by the data type
- Some others are…

Controlling Data Integrity

- Default value
- Range control
- Null values
- Referential integrity
- Handling missing data

Database Management System

Lecture - 22

Database Management System

Lecture - 23

Physical Records and Denormalization

- In logical DB design we group things logically related, accessed through same PK
- In physical DB design fields are grouped as they are stored physically and accessed by DBMS

Denormalization

- In some cases for the efficiency purposes we have to denormalize the relations
- Process of transforming normalized relations into unnormalized physical record specifications

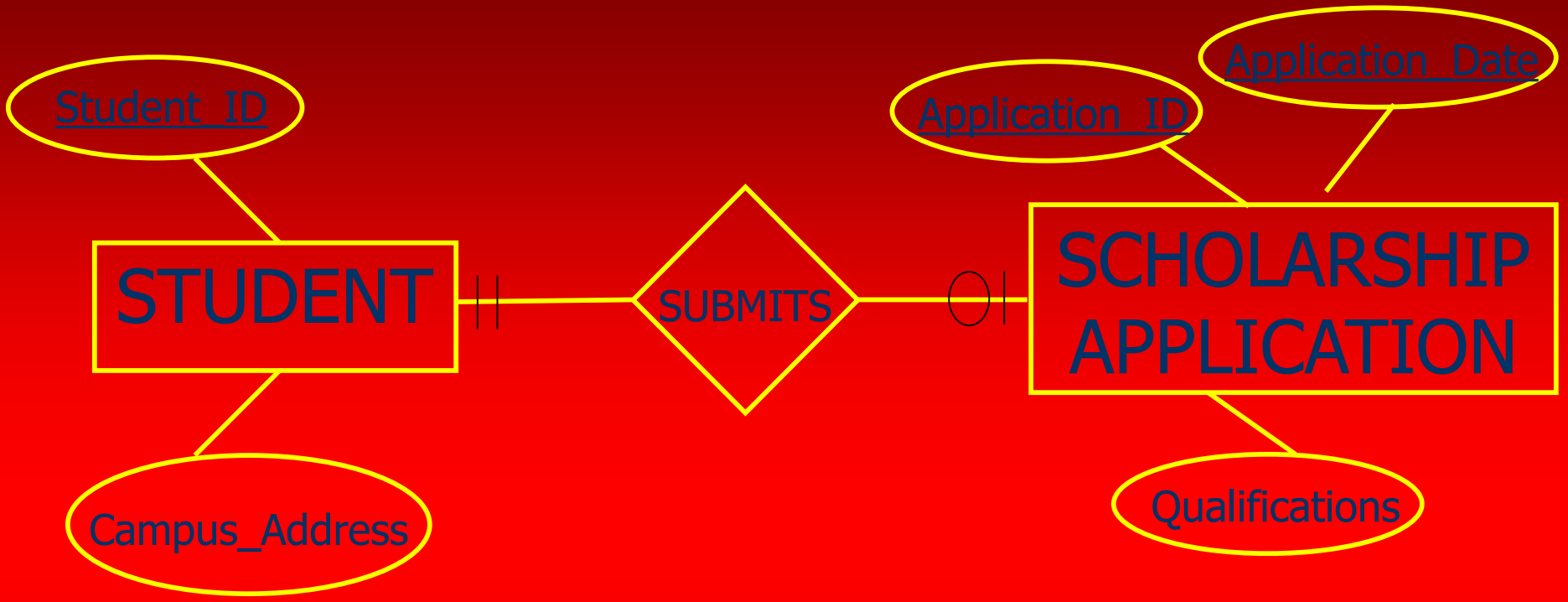
Denormalization

- In general, may decompose one logical relations into separate physical records, combine some, or do both
- See the situation, it is an option that you adopt purely for efficiency

Denormalization Situation 1

- Merge two ET into one with one to one relationship
- Even if one of the ETs is optional, so joining can lead to wastage of storage, however if two accessed together very frequently then merging might be a wise decision

One to One Relationship



STUDENT

Student_ID	Campus_Address

APPLICATION

Application_ID	Application_Date	Qualifications	Student_ID

Normalized Relations:

STUDENT

Student_ID	Campus_Address

APPLICATION

Application_ID	Application_Date	Qualifications	Student_ID

Denormalized Relations:

STUDENT

Student_ID	Campus_Address	Application_Date	Qualifications

Denormalization Situation 2

- Many to many binary relationships mapped to three relations
- Queries needing data from two participating ETs need joining of three relations, that is expensive

Denormalization Situation 2

- The relation created against relationship is merged with one of the relations created against participating ETs
- Reduces one join

Many to many relationship

- EMP(empId, eName, pjId, sal)
- PROJ(pjId, pjName)
- WORK(empId, pjId, dtHired, sal)

Many to many relationship

- EMP(empId, eName, pjId, sal)
- PROJ(empId, pjId, pjName, dtHired, sal)

Denormalization Situation 3

- Reference data: one to many situation when the ET on one side does not participate in any other relationship, then many side ET is appended with reference data rather than the FK

One to Many Relationships

- Student – Hobby(slide)

Partitioning

- Denormalization leads to merging different relations, whereas partitioning splits same relation into two
- Two types are possible horizontal and vertical

Horizontal Partitioning

- Table is split on the basis of rows
- The idea is that it is more efficient to process a table with small number of rows rather than a large table

Horizontal Partitioning

- Also helps in the maintenance of tables, like security, authorization, backup
- Even can be placed on different disks to reduce disk contention

Horizontal Partitioning

Some of the horizontal partitioning types:

- Range partitioning
- Hash partitioning
- List partitioning

Database Management System

Lecture - 23